Applied Soft Computing Journal 180 (2025) 113420

Contents lists available at ScienceDirect

Applied Soft Computing

journal homepage: www.elsevier.com/locate/asoc

Transforming security in internet of medical things with advanced deep learning-based intrusion detection frameworks

Nikhil Sharma *, Prashant Giridhar Shambharkar

Department of Computer Science & Engineering, Delhi Technological University, Bawana Road Rohini, Delhi 110042, India

HIGHLIGHTS

- Novel Deep Learning Models for IoMT Attacks: Introduced EmbedNet, ConvNet-SVM, and DeepSVM-Net models for intrusion detection.
- Advanced Data Preprocessing Techniques: Techniques include covariance matrix filtering, data augmentation, and normalization.
- Classification Performance Evaluation: Evaluated models against attacks like DoS, DDoS, and MITM for detection ability.
- Epochs and Batch Size Analysis: Analyzed effects of epochs and batch size on training loss, validation loss, and accuracy.
- Advanced Metrics Trade-off Analysis: Provided detailed trade-off analysis of metrics like MK, LR+, FOR, and FDR for model evaluation.
- Comparison with Existing Techniques: Proposed models show superior accuracy, PPV, TPR, and F1-Score compared to existing methods.

ARTICLE INFO

Keywords:
Cyber-Attacks
Deep Learning
Intrusion Detection System (IDS)
Internet of Medical Things (IoMT)
Security

ABSTRACT

The Internet of Things (IoT) revolutionizes industries like healthcare, agriculture, smart cities, and weather forecasting by enabling vast device networks to interact and transmit information. However, traditional network security methods are inadequate for IoT due to the limited storage and processing power of IoT devices. As IoT systems encounter numerous cyber threats, it is essential to develop innovative security approaches. Intrusion detection systems (IDS) are a popular mechanism for identifying and preventing systems from attacks. Therefore, this paper explores integrating deep learning (DL) models into IDS frameworks to enhance their capabilities. Deep learning is a subset of Artificial Intelligence (AI) that can extract complex patterns from data, adapt to new threats, and provide high accuracy and real-time detection. To provide the solution, this research presents novel deep learning models, including Embed-Net (Categorical Embedding Neural Network), the collaborative ConvNet-SVM model (combination of Convolutional Neural Network (CNN) and Support Vector Machine (SVM)), and the DeepSVM-Net (Deep Neural Network emulated by SVM), which are designed to detect network-based attacks on the Internet of Medical Things (IoMT). These models are evaluated using real-time benchmark datasets: ECU-IoHT, NF-BoT-IoT, and Wustl-HDRL-2024 by applying advanced preprocessing techniques that demonstrate superior performance by achieving an accuracy of 0.9990, 0.9959, and 0.9987with the lowest FNR of 0.0001, 0.0059, and 0.0014 by all three proposed models as compared to traditional methods. Furthermore, this paper conducted an ablation study which shows that our models have achieved outstanding performance with minimum training and validation losses of 0.0034 and 0.0008 for EmbedNet, 0.0018 and 0.0008 for ConvNet-SVM, and 0.0044 and 0.0016 for DeepSVM-Net. This research has enhanced IDS effectiveness, accuracy, and resilience by contributing significantly to cybersecurity, with the proposed models showing a remarkable improvement percentage ranging from 3.5 % to 7.5 % over the state-of-the-art.

1. Introduction

The Internet of Things (IoT) is a well-known technology that enables a vast network of devices and machines to interact and transmit information. IoT is extensively employed in sectors such as healthcare, agriculture, smart cities, and weather forecasting, which achieve remarkable advancements. According to a recent study, over 13.9 billion IoT devices are in use globally, with predictions indicating an increase to

* Corresponding author.
E-mail addresses: nikhilsharma_2k21phdco04@dtu.ac.in, nikhilsharma1694@gmail.com (N. Sharma), prashant.shambharkar@dtu.ac.in (P.G. Shambharkar).

https://doi.org/10.1016/j.asoc.2025.113420
Received 25 November 2024; Received in revised form 27 May 2025; Accepted 1 June 2025
Available online 4 June 2025

30.85 billion by 2025 [1]. This extensive network allows remote control of smart objects, data collection, and seamless data sharing. IoT networks comprise smart sensors for information gathering, software, and physical intelligent objects for computing and communication [2]. In healthcare, IoT solutions offer significant promise for improving medical treatments. Applications such as remote patient monitoring (RPM), electronic health records (EHR), and preventive care benefit from IoT's capabilities [3]. IoT transforms healthcare by making procedures more precise, streamlining decision-making, and fostering innovative frameworks. It allows us to capture real-time data, track motion, and monitor emergencies, which enhances remote patient care. Additionally, IoT improves communication within the smart healthcare ecosystem and generates valuable insights through data analytics [4]. IoT has revolutionized the field of smart healthcare systems by enabling different techniques, including:

- ***Remote monitoring system:*** IoT-driven remote monitoring facilitates ongoing communication with medical centres, thereby enhancing informed, timely, and effective decision-making across a spectrum of treatments and procedures.
- ***Telemedicine:*** The integration of IoT can significantly enhance remote medical procedures. Although the specific mechanisms may vary depending on the technique, IoT is poised to transmute telemedicine in the near future.
- ***Integrated Solutions:*** Multimodal developments can be contemplated in which medical IoT applications operate as part of a broader ecosystem that includes various other applications, such as logistics and transportation.
- ***Specialized attention:*** Recent advancements in IoT technology have significantly enhanced continuous and efficient healthcare solutions for both children and the elderly. These innovations provide real-time data through remote monitoring, enabling timely interventions and personalized care.
- ***Information Systems:*** IoT possesses the ability to assist in enhancing the efficiency of storing, accessing, and utilizing primary healthcare data through diverse methods.
- ***Medical records:*** IoT possesses the capability to assist healthcare systems by furnishing intricate patient data through consistent and thorough data collection methodologies.
- ***Mobile Assistance:*** Thanks to the extensive adoption of diverse devices, IoT has the potential to enable personalized healthcare services and enhance patients' accessibility.
- ***Knowledge Resources:*** In an IoT-enabled Medicare practice, professionals experience improved overall performance due to enhanced situational awareness, speedy response times, and continuous data gathering.

However, the integration of IoT in healthcare systems also introduces various cyber threats, including Denial of Service (DoS), Man-In-The-Middle (MITM) attacks, ransomware, sniffing, authentication issues, and risks to confidentiality and integrity [5]. These threats present major challenges to sensitive medical devices and patient data. Traditional network security methods are inadequate for IoT due to the limited storage and processing power of IoT devices [6]. As IoT systems face these cyber threats, innovative security approaches are crucial to enhance protection. IDS is a popular mechanism for detecting and preventing attacks on IoT networks. IDS differentiates between benign and malicious network traffic, preventing potential damage to the systems [7]. The significance of IDS in healthcare is highlighted by the increasing cyberattacks on healthcare systems. For instance, the "Wannacry" ransomware attack in May 2017 affected 48 healthcare services within the UK's National Health Service (NHS), which caused hospital immobilizations and surgery cancellations. The COVID-19 pandemic further impaired these challenges, with a fivefold increase in cyberattacks on health departments. Particularly, ransomware attacks have surged, targeting hospitals, clinics, and laboratories, often leading to severe operational disruptions and data breaches [8]. The increasing complexity of cyber threats necessitates advanced security measures [9]. While effective to an extent, traditional IDS struggle to manage pace with the evolving threat landscape. Therefore, integrating deep learning (DL) models into IDS frameworks has emerged as a promising solution. DL models extract complex patterns from data, adapting to new threats and providing high accuracy and real-time detection [10]. By using the strengths of DL, IDS can achieve significant improvements in identifying and mitigating cyber threats.

This research explores integrating DL models into IDS to enhance their capabilities. Specifically, it introduces novel deep learning models, including EmbedNet, ConvNet-SVM, and DeepSVM-Net, designed to detect network-based attacks on the Internet of Medical Things (IoMT). These proposed models are assessed using benchmark datasets: ECU-IoHT, NF-BoT-IoT, and Wustl-HDRL-2024. The proposed models demonstrate superior performance, which achieves high accuracy and low false negative rates compared to traditional methods. An ablation study further highlights the outstanding performance of these models, emphasizing their effectiveness, accuracy, and resilience in enhancing IDS for IoT networks. This comprehensive analysis underscores the potential of DL-enhanced IDS in safeguarding IoT systems, specifically in the essential healthcare field, against advanced cyber threats.

1.1. Motivation

The Smart Healthcare industry is promptly advancing, utilizing cutting-edge technology to enhance patient-centered services. However, this growth has also provided new opportunities for cyber attackers. To protect IoMT environments, numerous security measures have been developed, including authentication, access control, key management, encryption, and intrusion detection. This research specifically examines IDSs and the integration of AI techniques into these systems. IDSs are intended to detect malicious behaviours at both host and network levels. These malicious behaviours present serious threats to the security of Smart Healthcare, potentially causing data breaches, healthcare data alterations, unauthorized access, and life-threatening effects for patients.

The rapid increase in digital data and the growing complexity of cyber-attacks have made robust IDS more important than ever. Traditional methods and classical ML models are becoming less effective against the advanced and evolving nature of cyber threats. These older approaches often depend on predefined signatures or simple statistical models, which makes it hard to keep up with new and complex attack strategies. As a result, they struggle with accuracy, adaptability, and scalability in the fast-changing world of cyber security. DL, a branch of AI, has emerged as a powerful tool to overcome these challenges. By using multiple layers of neural networks, DL models have shown superior performance in various areas, such as image and speech recognition, natural language processing (NLP), and, more recently, cyber security [11]. DL models can automatically identify and learn complex patterns from large amounts of data, which makes them well-suited for enhancing IDS capabilities.

The motivation for this research is to demonstrate the advantages of DL over traditional methods and ML in the field of intrusion detection. DL models offer several key benefits:

- **Enhanced Feature Extraction:** DL models can identify complex, high-dimensional patterns in data that traditional methods often miss, which allows for the detection of intelligent and advanced attack signatures.
- **Adaptive Learning:** Unlike static models, DL frameworks can continuously learn and adapt from new data, which can help in improving their detection capabilities over time. This adaptability is essential in the ever-evolving cyber threat landscape.
- **High Accuracy and Reduced False Positives:** Deep neural networks (DNNs) enable IDS to distinguish between normal and

malicious activities better. This reduces a common issue of false positives found in the traditional systems.

- **Scalability:** DL models can efficiently handle and analyze large volumes of network traffic data, which makes them suitable for large-scale environments. This scalability ensures that IDS can maintain high performance even as network sizes and data volumes increase.
- **Real-time Detection:** The advanced architectures of DL models support real-time processing and threat detection, which allows immediate responses to potential security breaches.

1.2. Major contribution

The main contributions of this research are outlined as follows:

- **Design and development of Novel Deep Learning Models:** This study introduces several DL models, including the EmbedNet (A Categorical Embedding Neural Network), the collaborative ConvNet-SVM model (a combination of convolutional neural network (CNN) and support vector machine for Feature Extraction and Classification), and the DeepSVM-Net (A Deep Neural Network emulated by Support Vector Machine) model. These models are designed to identify network-based attacks on the IoMT. The EmbedNet model represents a significant advancement by utilizing a DL-driven embedding methodology to detect intrusions within the IoMT environment using the ECU-IOHT (D1), NF-BoT-IoT (D2), and WUSTL-HDRL-2024 (D3) real-time dataset.
- **Advanced Preprocessing and Feature Engineering Techniques:** The research applies advanced preprocessing and feature engineering techniques, including eliminating irrelevant columns using a covariance matrix to enhance the detection process, which helps reduce memory usage and inference time. Then, the Data augmentation technique is employed, which balances the dataset by providing adequate data for normal and attack samples. Furthermore, Features are categorized into continuous and categorical columns where continuous columns are normalized using techniques like the Standard Scaler and Power Transformer, while categorical columns are transformed using ordinal encoders. This ensures effective normalization and transformation of the data.
- **Classification for Performance Evaluation:** The models' performance is evaluated through a classification approach that differentiates between benign network traffic and malicious activities, which include attacks like Smurf attack, ARP Spoofing, Nmap PortScan, DoS, DDoS, MITM, Reconnaissance, and Ransomware. This method thoroughly assesses the models' ability to detect various types of network intrusions.
- **Analysis of Epochs and Batch Size Effects:** This research examines the effects of epochs and batch size on IoMT intrusion detection performance, specifically by analyzing training loss (TL), validation loss (VL), and accuracy. This research provides insights into optimizing DL model performance for intrusion detection by examining these parameters.
- **Advanced Metrics Trade-off and Analysis:** For the first time, this research provides an in-depth trade-off and analysis of advanced metrics for the proposed DL models, including metrics such as Markedness (MK), Informedness (BM), Positive Likelihood Ratio (LR+), Negative Likelihood Ratio (LR-), False Omission Rate (FOR), and False Discovery Rate (FDR). These metrics offer a comprehensive evaluation of the models' performance and reliability.
- **Comparison with Existing Techniques:** The proposed models are rigorously compared with existing methods on similar real-time and benchmark datasets. The results demonstrate that these models outperform existing techniques by achieving the highest accuracy, Positive predictive value (PPV), True positive rate (TPR), and F1-Score for detecting attacks while minimizing inference time.

This paper aims to thoroughly analyze the use of DL models in IDS frameworks and compare their performance with traditional techniques and ML models. Through rigorous experimentation and evaluation, we seek to demonstrate how DL can significantly advance the effectiveness, accuracy, and resilience of IDS by contributing to the broader field of cyber security.

Paper Organization: The rest of the paper is organized as follows: 2 presents a literature review comprising all the potential solutions provided by different researchers to detect attacks in the IoT ecosystem; 3 explains the need for DL approach for building IDS framework for IoMT, overview, and Workflow of Proposed DL Models Architecture including data preprocessing and feature engineering steps; 4 discusses Performance Evaluation Metrics and Experimentation setup, datasets, and Result achieved by different ML and DL models; 5 illustrates the performance evaluation of IoMT intrusion detection with varying Epochs and Batch Sizes followed by the conclusion in 6.

2. Literature review

The integration of Artificial Intelligence (AI) into various domains has led to significant advancements, particularly in cybersecurity, where it plays a crucial role in enhancing Intrusion Detection Systems (IDS). The proliferation of the Internet of Things (IoT) has expanded the attack surface, necessitating robust security measures. In this context, IDS serve as a fundamental mechanism for detecting malicious activities in network traffic. This section explores the evolution of IDS, categorizing existing methodologies into traditional IDS approaches, machine learning-based IDS techniques, deep learning-based IDS solutions, and challenges specific to Intrusion Detection in the Internet of Medical Things (IoMT).

2.1. Traditional IDS methods

Traditional IDS techniques primarily rely on rule-based and signature-based methods to detect intrusions. These approaches involve predefined attack signatures and heuristic rules to classify network traffic. While effective against known threats, traditional IDS struggle to detect novel and zero-day attacks due to their reliance on static rules [12]. Several studies have attempted to improve traditional IDS by optimizing feature selection and decision-making processes. Kabir et al. [13] proposed an IDS framework that integrates Least Square Support Vector Machine (LS-SVM) with optimal sampling techniques, significantly enhancing classification accuracy. Similarly, Aljawarneh et al. [14] developed a hybrid model that estimates intrusion thresholds by optimizing network transaction features, leading to improved detection performance. Zhou et al. [15] introduced a heuristic-based feature selection algorithm known as CFS-BA, which effectively reduces redundant features and improves classification efficiency. Despite these improvements, traditional IDS still suffer from high false positives and limited adaptability to evolving attack patterns.

2.2. Machine Learning-Based IDS approaches

Machine learning (ML) techniques have revolutionized IDS by enabling systems to learn patterns from data and detect anomalies without relying on predefined rules. ML-based IDS leverage algorithms such as decision trees, support vector machines, and ensemble methods to improve detection accuracy and reduce false positive rates.

Aldwairi et al. [16] applied the Restricted Boltzmann Machine (RBM) to detect network anomalies, demonstrating the potential of deep feature learning for IDS. Similarly, Abusitta et al. [17] proposed a cooperative IDS that integrates historical feedback, allowing the system to continuously refine its decision-making process, thereby reducing false positives. Gu et al. [18] further enhanced model performance by transforming logarithm marginal density ratios, improving the training data's quality. Additionally, Zhou et al. [19] introduced an ensemble

learning approach based on a modified adaptive boosting algorithm (M-AdaBoost-A), which significantly improved detection accuracy by leveraging multiple weak classifiers. While ML-based IDS have shown promising results, they often require extensive labeled data for training and may struggle with scalability in real-world IoT environments.

2.3. Deep learning-based IDS approaches

Deep learning (DL) has further advanced IDS by leveraging neural networks to extract complex patterns from network traffic. DL-based IDS solutions employ architectures such as autoencoders, convolutional neural networks (CNNs), and recurrent neural networks (RNNs) to enhance detection capabilities.

Kim and Park [20] developed a Hybrid Deep Auto-Encoder Q-network (DAEQ-N) that improves intrusion prediction accuracy by efficiently capturing latent representations of network data. Li et al. [21] proposed an autoencoder-based IDS that integrates random forest (RF) classifiers to enhance attack detection rates. Similarly, Qazi et al. [22] introduced a Stacked Non-Symmetric Deep Auto-Encoder (S-NDAE) combined with SVM, achieving an impressive accuracy of 99.65 %. In another study, Firat Kilincer et al. [23] developed a hybrid Recursive Feature Elimination (RFE) method, integrating it with a multilayer perceptron (MLP) to enhance anomaly detection.

To address privacy concerns in distributed environments, Mosaiyebzadeh et al. [24] introduced a federated learning framework incorporating Deep Neural Networks (DNN-FL) to perform anomaly detection while preserving data privacy. Additionally, Nazir et al. [25] proposed a Hybrid CNN-LSTM model that employs Principal Component Analysis (PCA) for data preprocessing, optimizing network traffic analysis for IoT security. These advancements highlight the effectiveness of deep learning in IDS but also emphasize the need for computationally efficient models that can operate in resource-constrained environments.

2.4. IoMT-specific IDS challenges

The Internet of Medical Things (IoMT) introduces unique security challenges due to the critical nature of healthcare data, stringent privacy regulations, and the resource-constrained environment of medical devices. Effective IoMT IDS must ensure low-latency detection to support real-time healthcare operations, maintain energy efficiency for battery-powered devices, and comply with privacy standards like HIPAA. These requirements make traditional and even some ML-based IDS inadequate, as they often prioritize accuracy over computational efficiency or fail to address healthcare-specific attack vectors, such as those targeting medical IoT protocols.

Several studies have proposed IoMT-tailored IDS frameworks, which directly inform the design of our proposed system. Ahmed et al. [26] introduced the ECU-IoHT dataset, which includes healthcare-specific attack samples (like ARP Spoofing and Nmap PortScan) which are used in our proposed IDS to evaluate models like EmbedNet, ConvNet-SVM, and DeepSVM-Net. This dataset's focus on IoMT cyber-attacks ensures our system addresses realistic threats. Shahzad et al. [27] proposed an integrated IoT healthcare platform emphasizing stakeholder collaboration, inspiring our system's design to incorporate multi-device data aggregation for robust detection. Alazab et al. [28] developed a Moth Flame Optimization with Decision Trees (MFO-DT) approach, achieving 97.8 % accuracy by optimizing feature selection, which aligns with our use of efficient feature extraction in ConvNet-SVM to reduce computational overhead. Similarly, Azimjonov and Kim [29] introduced a lightweight Stochastic Gradient Descent Classifier (SGDC) with ridge regression, achieving 92.69 % accuracy on IoT botnet datasets. This lightweight approach influenced our DeepSVM-Net's architecture to balance accuracy and efficiency in resource-constrained IoMT environments. Zhang et al. [30] presented a Maximum Information Coefficient (MIC)-based method for analyzing nonlinear feature relationships in network traffic, effectively reducing redundancy and noise.

Advanced DL techniques further address IoMT challenges. Mosaiyebzadeh et al. [24] employed federated learning with Deep Neural Networks (DNN-FL) on the ECU-IoHT dataset, ensuring privacy-preserving anomaly detection. This approach informs our consideration of decentralized training to protect sensitive patient data. Nazir et al. [25] proposed a Hybrid CNN-LSTM model with PCA preprocessing, achieving 99 % accuracy on N-BaIoT, which parallels our ConvNet-SVM's use of CNN for feature extraction and DeepSVM-Net's hybrid architecture for capturing temporal attack patterns. Aguru and Erukala [31] developed a stacked modified GRU (SM-GRU) model for DDoS detection, achieving 98.56 % F1-score, highlighting the need for time-critical detection, which our system addresses through optimized inference in EmbedNet. Kumar et al. [32] introduced an XAI-based Self-Attention LSTM (XAI-SALSTM), emphasizing decision transparency, which we incorporate via interpretable feature importance in DeepSVM-Net. Saheed et al. [33] proposed IoT-Defender with a modified genetic algorithm and LSTM, achieving 99.41 % accuracy, guiding our use of optimization techniques to enhance model efficiency. Chintapalli et al. [34] combined Osprey Optimization Algorithm with BiLSTM, achieving 99.98 % accuracy, reinforcing our focus on advanced feature selection to improve detection rates in resource-constrained settings.

Despite these advancements, IoMT IDS face challenges like dataset biases, high computational costs, and scalability issues in heterogeneous IoMT networks. Our proposed IDS addresses these by leveraging ECU-IoHT and NF-BoT-IoT datasets for robust evaluation, optimizing model architectures for low-latency inference, and exploring bias mitigation techniques like SMOTE, inspired by the need to handle imbalanced classes as seen in [31,33].

2.5. Relevance to proposed IoMT IDS

The surveyed DL tools and IoMT challenges directly shape our proposed IDS, which comprises EmbedNet, ConvNet-SVM, and DeepSVM-Net, evaluated on ECU-IoHT and NF-BoT-IoT datasets. The high accuracy of DL models like S-NDAE-SVM (99.65 %) [22] and Hybrid CNN-LSTM (99 %) [25] justifies our use of CNN-based feature extraction in ConvNet-SVM and hybrid architectures in DeepSVM-Net to capture complex attack patterns. The lightweight SGDC [29] and RFE-based MLP [23] inform our focus on computational efficiency, critical for IoMT devices with limited resources. For instance, DeepSVM-Net integrates SVM classification with DL features, reducing inference time compared to traditional CNNs, while EmbedNet employs compact embeddings to minimize energy consumption, addressing challenges highlighted in [24,33].

IoMT-specific challenges, such as low latency and privacy, are addressed by drawing on federated learning [24] for potential decentralized training and XAI [32] for transparent decision-making, ensuring compliance with healthcare regulations. Dataset biases, a key challenge in IoMT (e.g., under-represented Theft and Reconnaissance classes in NF-BoT-IoT), are mitigated using SMOTE and class weighting, inspired by [31,33], to improve minority class detection. The ECU-IoHT dataset's healthcare-specific attack samples [26] align with our system's goal of detecting IoMT-relevant threats like ARP Spoofing, while NF-BoT-IoT's diverse attack types ensure generalizability. By integrating these insights, our IDS achieves high accuracy of 99.94 % for DeepSVM-Net on Dataset 1 while addressing IoMT constraints, offering a robust, efficient, and privacy-aware solution for healthcare security.

The evolution of IDS has transitioned from traditional rule-based approaches to AI-driven solutions that leverage machine learning and deep learning. While traditional IDS methods remain effective for known threats, ML and DL-based IDS significantly improve anomaly detection, reducing false positives and enabling adaptive learning. However, IoMT-specific challenges, including resource constraints and data privacy concerns, necessitate the development of lightweight and

efficient IDS frameworks tailored for healthcare environments. Future research must focus on optimizing computational efficiency, enhancing model interpretability, and ensuring robust security solutions that comply with healthcare data protection regulations. Table 1 depicts the advantages and disadvantages of various proposed techniques for anomaly detection in IoMT systems.

Table 2 depicts the dataset samples type where ✓ indicates the presence of samples (like IE- IoT enabled traffic, HS- Healthcare sample, and AT- Attack samples) and × denotes the absence of samples in the dataset.

3. Proposed methodology

This section presents the proposed methodology, followed by the data preprocessing and feature engineering process, and overview and hyperparameter tuning of ML and DL approaches.

3.1. Problem statements and research gaps

The need for IDS utilizing DL approaches in the IoMT ecosystem introduces various challenges and research gaps. These issues are identified from the literature review, which are presented as follows:

- **Rapid growth of the IoMT devices:** The IoMT integrates various sensors, medical devices, and healthcare applications connected over the internet. This interconnectedness aims to enhance healthcare delivery, monitoring, and management, resulting in the widespread deployment of IoMT devices.
- **Increased Vulnerability to Cyber Threats:** IoMT devices often have limited computational capabilities and are frequently deployed with minimal security features. This vulnerability exposes IoMT networks to numerous cyber threats, including data breaches, unauthorized access, and malicious attacks that can disrupt healthcare services.
- **Inadequacies of Traditional IDS Solutions:** Traditional IDS, typically reliant on signature-based methods, fail to keep up with the dynamic and evolving nature of cyber threats targeting IoMT devices. These systems struggle to detect new, unknown attack vectors, leading to high rates of false positives and negatives.
- **Advancements in DL approaches for Cybersecurity:** DL has demonstrated significant potential in numerous domains, including cybersecurity, due to its ability to learn complex patterns and generalize from data. The application of DL in IDS can potentially address the limitations of traditional approaches by providing more accurate and adaptive threat detection capabilities.
- **Critical Need for IoMT-Specific IDS:** There is a critical need to develop specialized IDS solutions that leverage deep learning to secure IoMT environments. These systems must be capable of real-time monitoring, effective anomaly detection, and adaptability to the unique challenges presented by IoMT devices.
- **Limited Focus on IoMT in Existing IDS Research:** Current research on IDS predominantly focuses on general-purpose networks or specific environments like industrial IoT, with limited attention to the unique requirements of IoMT. There is a lack of comprehensive studies that address the specific security challenges and threat landscape of IoMT networks.
- **Inadequate IoMT-Specific Datasets:** The development and evaluation of DL-based IDS for IoMT are hindered by the scarcity of extensive, labelled datasets that accurately represent IoMT traffic and attack patterns. Existing datasets are often insufficient in capturing the diversity and complexity of IoMT environments, limiting the effectiveness of IDS models.
- **Scalability and Real-Time Detection Challenges:** Many proposed deep learning-based IDS solutions face challenges in scaling to real-time applications due to computational constraints and the need for rapid processing. Ensuring that DL models can operate efficiently in resource-constrained IoMT devices and provide timely threat detection remains an open research challenge.
- **Integration and Interoperability Issues:** Integrating DL-based IDS into existing IoMT frameworks requires addressing compatibility and interoperability issues with diverse medical devices and healthcare systems.
- **Adaptive Learning and Continuous Improvement:** While DL models offer high accuracy, maintaining their performance over time necessitates adaptive learning mechanisms incorporating new threat intelligence and adapting to evolving attack strategies.

3.1.1. Addressing research gaps through the proposed model

The proposed DL-based Intrusion Detection System (IDS) for IoMT effectively addresses the research gaps identified in the literature review. By employing advanced DL architectures, feature engineering techniques, and performance optimization strategies, our approach ensures robust and real-time intrusion detection in IoMT environments as shown in Table 3.

We designed an IDS framework using ML and DL models for the IoMT ecosystem from the identified research gaps, as depicted in Fig. 1. This framework provides a robust solution to accurately classify normal and malicious activities.

3.2. Data preprocessing

Data preprocessing is a critical initial phase in ML and DL workflows to enhance model performance. This research implemented widely used techniques such as Standard Scaler [36] and Simple Imputer for data preprocessing.

Standard Scaler is a prominent method for data normalization, frequently utilized before training ML or DL models. This technique standardizes features by removing the mean and scaling to unit variance. It normalizes the input dataset's value distribution so that the mean is 0 and the standard deviations are 1. This standardization is essential for optimizing the functionality and accuracy of various algorithms by ensuring that all input features contribute equally to the model training procedure. The standard scaler can be mathematically expressed as follows in Eq. (1):

For a given feature x in the dataset, the standardized value z is calculated as:

$$z = \frac{x - \mu}{\sigma} \tag{1}$$

Where, μ depicts mean and σ represents the standard deviation of the feature values. This technique involved several steps in the calculation of mean, standard deviation, and standardization of each feature value using Eqs. (2–4) and feature scaling in Eq. (5):

$$\mu = \frac{1}{N}\sum_{i=1}^{N} x_i \tag{2}$$

$$\sigma = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(x_i - \mu)^2} \tag{3}$$

$$z_i = \frac{x_i - \mu}{\sigma} \tag{4}$$

Where, for each i in $\{1, 2, \ldots\ldots, N\}$.

$$X_{\text{FeatureScaled}} = \frac{X_{\text{FeatureCurrentValue}} - \text{mean}(X_{\text{Feature}})}{\text{StandardDeviation}(X_{\text{Feature}})} \tag{5}$$

Where, $X_{\text{FeatureScaled}}$ represents the scaled feature value after the scaling process, $X_{\text{FeatureCurrentValue}}$ shows the unaltered current feature value; X_{Feature} encompasses all the dataset's feature columns in the datasets and

Table 1
The advantages and disadvantages of various proposed techniques for anomaly detection in IoMT systems.

Ref & Year	Method	Dataset	Significance	Result	Advantages	Disadvantages
Kabir et al. [13], 2018	LS-SVM	KDD 99	This paper utilizes a two-stage decision-making process to enhance the accuracy and efficiency of intrusion detection.	Ac= 97.64 %, Re= 90.89 %, F1 = 94.14 %	The two-stage approach, with subgroup sampling followed by LS-SVM application, is designed to handle large datasets efficiently. The method's capability to handle incremental datasets suggests scalability for dynamic and evolving network environments.	The effectiveness of the optimal allocation scheme depends on the variability within subgroups, which might be challenging to determine accurately. Despite efficiency in handling large datasets, the initial subgrouping and sampling process could introduce computational overhead, particularly in highly dynamic network environments.
Aldwairi et al. [16], 2018	RBM	ISCX dataset	This paper highlights the pressing need for adaptive methodologies in network security to counter increasing and evolving cyber-attacks.	Ac= 79.8 %, TPR= 84.7 %, TNR= 75 %	RBMs are capable of learning and adapting to new, previously unseen patterns of attacks, enhancing the robustness of the intrusion detection system.	While RBMs are effective, their scalability to extremely large and diverse network environments may present challenges, particularly in real-time applications.
Aljawarneh et al. [14], 2018	Hybrid classifiers such as Naive Bayes, AdaBoostM1, J48, REPTree, RandomTree	NSL-KDD	This paper introduces a hybrid model aimed at estimating the intrusion scope threshold degree by leveraging optimal network transaction features, thereby effectively mitigating computational and time complexity challenges.	Ac= 99.81 %	The author utilizes the Vote algorithm with Information Gain to filter data and select important features, enhancing the model's accuracy.	The model exhibits a high false negative rate, which could lead to undetected intrusions and potential security breaches.
Abusitta et al. [17], 2019	SDAE-IDS	KDD Cup 99	This paper utilizes a Denoising Autoencoder (DA) to handle incomplete feedback, enhancing the robustness and reliability of the IDS in a cooperative cloud-based environment.	Ac= 89.2 %	The ability to make decisions without waiting for complete feedback from all IDSs reduces delays and improves the system's real-time effectiveness.	Despite using GPU-enabled TensorFlow, the deep neural network and DA reconstruction processes could introduce significant computational overhead, especially in resource-constrained environments.
Gu et al. [18], 2019	DT-EnSVM, DT-EnSVM2	NSL-KDD, Kyoto 2006 + , KDD'99	This paper emphasizes the importance of high-quality training data, which is crucial for the accuracy and effectiveness of IDS.	Ac= 99.41 %, DR= 99.09 %, FAR= 0.31	This paper uses ensemble learning techniques to combine multiple SVM models, improving overall detection performance and reliability.	The effectiveness of the framework heavily relies on the quality of the training data, which may not always be available or easy to obtain.
Kim and Park [20], 2019	DAEQ-N	KDD'99	This paper highlights the potential of reinforcement learning (RL), specifically the deep Q-learning algorithm, to enhance real-time online network intrusion detection (NID) systems.	Different Result Values	The RL-powered approach allows the system to adapt in real-time to new and evolving cyber threats, maintaining high performance despite the changing nature of attacks.	The DL and RL components require significant computational power and memory, which might limit the deployment of this approach in resource-constrained environments.
Zhou et al. [15], 2020	CFS-BA-Ensemble approach	NSL-KDD, AWID, CIC-IDS2017	This paper addresses key challenges in existing IDS by proposing a new framework combining feature selection and ensemble learning to enhance detection accuracy and adaptability.	AC= 99.89 %, DR= 99.9 %, FAR= 0.12	The CFS-BA algorithm effectively reduces high-dimensional data, eliminating irrelevant and redundant features that could interfere with the classification process.	Ensemble methods typically require more computational resources for training and testing, which might be a limitation in resource-constrained environments.
Li et al. [21], 2020	AE-IDS	CSE-CIC-IDS 2018	This paper tackles prevalent challenges in IDS, such as the lack of labelled datasets, high computational overhead, and low accuracy, by implementing a novel approach to feature selection and grouping.	Different Result Values	The model's ability to adapt to various types of network traffic and attack patterns increases its robustness and effectiveness in diverse environments.	Despite reduced detection time, the initial training phase might still require significant computational resources due to the deep learning components.
Zhou et al. [19], 2020	M-AdaBoost-A	AWID, NSL-KDD	This paper introduced a unique ensemble system utilizing the M-AdaBoost-A algorithm, enhancing the	Ac= 99.99 %	The author utilizes particle swarm optimization to fine-tune the ensemble system, potentially increasing the robustness	While effective, the ensemble approach may face scalability challenges when applied to large

(*continued on next page*)

Table 1 (*continued*)

Ref & Year	Method	Dataset	Significance	Result	Advantages	Disadvantages
			detection of network intrusions.		and effectiveness of the intrusion detection.	datasets or in real-time scenarios.
R.M. et al. [70], 2020	PCA-GWO-based Deep Neural Network (DNN) classifier	Kaggle	The proposed model significantly improves intrusion detection accuracy by 15 % and reduces training time by 32 %, making it highly suitable for real-time IoMT security systems with faster alert mechanisms.	AC−0.999, RE−0.954, SPE−0.9992	15 % improvement in classification accuracy. 32 % reduction in time complexity, enhancing real-time detection.	Needs further evaluation for multiclass problem scenarios.
Ahmed et al. [26], 2021	Nearest Neighbor-Based Algorithms	ECU-IoHT	This paper provides a unique resource, ECU-IoHT, as no other publicly available datasets exist specifically for cybersecurity in the IoHT domain. Facilitates the analysis of attack behaviour and the development of robust countermeasures by making the dataset available to researchers.	Different Result Values	ECU-IoHT dataset is specifically designed to reflect cyberattacks on IoHT, providing a valuable resource for the community. Includes various attacks exploiting different vulnerabilities, offering a thorough basis for analysis.	The dataset may not cover all possible attack vectors or scenarios, potentially limiting its applicability to new or emerging threats. Development and sharing of datasets in the healthcare domain can be challenging due to privacy issues.
Shahzad et al. [27], 2021	A Two-Step Ontological Modeling	-	This paper proposed an integrated platform to unify various IoT-based smart healthcare services, enhancing collaboration among stakeholders.	-	The paper enhanced the sharing of knowledge and data across different healthcare services, leading to better-informed decision-making. The ontological approach can be extended to include new healthcare services and applications, making it scalable and adaptable to future developments.	The initial setup of the ontological model and its maintenance may demand significant effort and continuous updates. The effectiveness of the integration depends on the quality and accuracy of the data from various smart healthcare services.
Alazab et al. [28], 2022	MFO-DT	KDDCUPP9, NSL-KDD, and UNSW-NB15	This paper introduced cosine similarity as a method to binarize continuous MFO, overcoming limitations of the traditional sigmoid function and improving the search algorithm's effectiveness.	Ac= 97.8 %, F1–99 %, TPR= 99.6 %, FPR= 8.1	By selecting a minimal subset of features, the proposed method reduces computational complexity and resource requirements, making it suitable for real-time applications.	The effectiveness of the MFO algorithm can be sensitive to initial conditions and parameter settings, potentially impacting reproducibility and consistency across different scenarios.
Qazi et al. [22], 2022	S-NDAE-SVM	KDD CUP'99.	The study proposes an intelligent and efficient NIDS utilizing DL, addressing the critical challenge of detecting continuously emerging threats.	Ac = 99.65 %,Pr = 99.99 %,Re = 99.85 %, F1= 99.55 %	This paper utilized the TensorFlow library and GPU framework, ensuring efficient processing and scalability, which is critical for handling large-scale network data.	Implementing and tuning deep learning models, especially non-symmetric deep auto-encoders, can be complex and resource-intensive, requiring significant expertise in DL frameworks.
Patil et al. [71], 2022	ML ensemble methods	CICIDS−2017	The integration of ensemble methods with LIME enhances both the performance and interpretability of IDS, facilitating better trust and understanding in the detection process.	AC−0.9625, PR−0.89, RE−0.89, F1–0.89	High classification accuracy (96.25 %) using ensemble methods. Improved model interpretability with LIME. Effective in reducing false positives.	LIME might not address all aspects of model interpretability for complex models.
Firat Kilincer et al. [23], 2023	RFE, MLP, XGBRegressor	ECU-IoHT, TON-IoT, WUSTL-EHMS 2020	The authors address the critical need for robust cybersecurity measures in the rapidly growing IoMT, where devices are often vulnerable to attacks.	AC= 99.99 %, Pr= 99.99 %, Re= 99.99 %, F1= 99.99 %	RFE with LR and XGBRegressor ensures the selection of the most relevant features, enhancing model performance.	The comprehensive approach, including feature selection and model tuning, can be resource-intensive and potentially challenging to deploy in highly constrained environments.
Mosaiyebzadeh et al. [24], 2023	Deep Neural Network in Federated Learning	ECU-IoHT	The author used federated learning to enhance privacy in ML tasks, aligning with increasing privacy regulations and concerns.	Different Result Values	Federated learning ensures that data remains decentralized, protecting user privacy while enabling robust machine learning. Enhances the security of	Federated learning frameworks may face scalability challenges when deployed across a large number of IoHT devices with varying capabilities.

(*continued on next page*)

Table 1 (*continued*)

Ref & Year	Method	Dataset	Significance	Result	Advantages	Disadvantages
					IoHT environments by effectively identifying and mitigating potential security threats.	
Thulasi and Sivamohan [35], 2023	Multi Step Convolutional neural network Stacked Long short-term memory architecture (MSCSL)	ECU-IoHT, TON-IoT, SCADA IEC 60870–5–104	Targets weak points and gaps in the cyber-resilience of healthcare facilities, especially during crises.	Ac= 98.95 %, Pr= 98.5 %, Re= 96.3 %, F1= 98.74 %, MCC= 92 %,	This paper used LSO to optimize MSCSL hyperparameters, enhancing model efficiency. Combines the advantages of ReLU with LeLeLU for better data adaptability and lower computational complexity.	Scalability and real-world application in diverse healthcare settings may present challenges due to varying IoT device capabilities.
Azimjonov and Kim [29], 2024	SGDC	KDD-CUP−1999, BotIoT−2018, N-BaIoT−2021	Tackles the urgent need for efficient IDSs in the face of rising IoT-related botnet attacks.	Ac= 92.69 %, Pr= 99.68 %, Re= 82.01 %, F1= 89.99 %	Suitable for resource-constrained IoT devices due to lightweight algorithms. Adaptable for various IoT environments, promoting widespread application.	Hyperparameter tuning is complex and resource-intensive, requiring expert intervention. Fine-tuning on specific datasets might lead to overfitting, impacting real-world performance.
Zhang et al. [30], 2024	Maximum Information Coefficient (MIC)-Extreme Gradient Boosting (XGB)	WUSTL-EHMS−2020, CICIDS2017, ECU-IoHT	This paper introduced the MIC method to analyze nonlinear relationships among traffic features, addressing redundancy and noise issues.	Ac= 95.01 %, Pr= 94.94 %, Re= 95.01 %, F1= 94.64 %	The MIC method effectively reduces information redundancy and noise, leading to better detection performance. Incorporates patient biometric features, making the IDS highly relevant and effective for healthcare systems.	The MIC method and Extreme Gradient Boosting model may require significant expertise and computational resources to implement. High performance on specific datasets may lead to concerns about overfitting and generalizability to different or new data environments.
Nazir et al. [25], 2024	PCA, Hybrid CNN-LSTM	N-BaIoT, CICIDS−2017	This paper addressed the urgent need for advanced IoT threat detection solutions amid increasing cybersecurity threats.	Ac= 99 %	Effectively captures and analyzes security data, offering robust threat detection capabilities.	The combination of CNN and LSTM models, along with PCA and optimization techniques, might be resource-intensive, posing challenges for deployment in extremely resource-constrained devices.
Aguru and Erukala [31], 2024	SM-GRU	CICIoT2023, CICSSoS2019	This paper addressed the critical demand for effective IoT security, particularly against intensified multi-vector DDoS attacks.	Pr= 98.53 %, Sen= 97.74 %, Spe= 97.74 %, F1= 98.56 %, FAR= 0.023, FPR= 0.0225, MCC= 95.36 %	Designed for time-critical applications, making it suitable for deployment in resource-constrained healthcare environments.	While beneficial, a 2 % reduction in time consumption may not be substantial enough to address all time-critical constraints in healthcare systems.
Kumar et al. [32], 2024	XAI-SALSTM, SHAP	N-BaIoT, DTs	This paper addressed the critical challenges of transparency and accountability in AI decisions by integrating smart contracts and eXplainable AI (XAI).	Ac= 95.81 %	Employs a SALSTM network for superior attack detection performance. Digital Twins (DTs) simulate attack detection operations, offering a platform for analysis and development of IDS under various scenarios.	The use of SALSTM networks and the generation of DTs datasets can demand substantial computational resources, potentially limiting deployment in resource-constrained environments. The complexity of the proposed framework may pose scalability challenges for large-scale or real-time applications.
Saheed et al. [33], 2024	GA-LSTM	BoT-IoT, UNSW-NB15, and N-BaIoT	This paper addressed the significant security challenges posed by the integration of IoT devices in these smart environments, particularly focusing on misconfigurations and vulnerabilities. Innovative Appr	Ac= 99.41 %, DR= 99.78 %, Pr= 98.50 %, FAR= 2.56 %	MGA optimizes feature selection, making the LSTM model more efficient by focusing on relevant features, crucial for resource-constrained IoT devices. Designed to be lightweight, the model can be deployed on edge servers, enabling real-time detection of cyber-attacks.	While designed to be lightweight, the deployment across diverse and large-scale IoT networks could present scalability challenges. Many IoT devices lack hardware security measures, potentially limiting the effectiveness of even advanced IDS frameworks like IoT-Defender.

(*continued on next page*)

Table 1 (*continued*)

Ref & Year	Method	Dataset	Significance	Result	Advantages	Disadvantages
Chintapalli et al. [34], 2024	OOA-modified Bi-LSTM	N-BaIoT, CICIDS−2017, ToN-IoT	This paper addressed the challenges of processing the massive amounts of data generated by IoT systems, ensuring security and reliability. The Osprey Optimization Algorithm (OOA) is introduced for effective feature selection, improving detection accuracy and efficiency.	Ac= 99.98 %, Re= 99.94 %, Spe= 99.90 %, F1 = 99.89 %	The use of the ELU activation function in the Bi-LSTM network accelerates the learning process by handling gradients more effectively. It provides a framework that offers high detection accuracy and better interpretability, making it easier to understand and trust the IDS's decisions.	DL models like Bi-LSTM with advanced activation functions can be resource-intensive, potentially limiting their deployment on resource-constrained IoT devices. Continuous updates and maintenance are required to keep the IDS effective against evolving cyber threats, adding to operational overhead.
Lazrek et al. [72], 2024	Recursive Feature Elimination (RFE) with ML and Ridge regression in DL models	WUSTL-EHMS dataset	The proposed framework enhances the security of IoMT systems by effectively detecting anomalies and cyber-attacks, thereby safeguarding patient data and ensuring robust healthcare services.	Training Accuracy = 0.99 Test Accuracy = 0.9785, PR−0.9650, RE−0.8629, AUC −0.9292, MCC−0.8402, FAR−0.03	The efficient feature selection through RFE reduces the complexity and improves the performance of the model.	Implementation in real-world may face challenges related to integration with existing IoMT systems and operational constraints.
Ioannou et al. [73], 2024	Three-stage IDS for MIoT	Network emulation (KALI penetration testing) with CICEV2023	The proposed system effectively detects both known and unknown attacks in MIoT networks, while preserving privacy and minimizing energy consumption through FL.	AC−0.9998	The model detects anomaly (99.7 %) efficiently with One-Class SVM. Low resource utilization with FL and ensure privacy and quick updates	Limited attack types in the dataset; future research is needed for scalability.

mean () represents the average value of each feature.

The Simple imputer addresses missing values within the dataset. This technique replaces the missing entries with statical values such as mean, median, or the most frequent value with a specified constant. It can be expressed as follows from the Eqs. (6–8):

To begin with, Eq. (6) is used to calculate the mean imputation where $\varkappa_i$ shows an observed value, $\widehat{x}$ represents imputed value, and depicts μ mean of the observed values. In addition, median imputation is calculated using Eq. (7), where $median(x)$ represents the median of the observed values. Furthermore, mode imputation is calculated using Eq. (8) where $mode(x)$ denoted the most frequent value among the observed values.

$$\widehat{x} = \mu \tag{6}$$

$$\widehat{x} = median(x) \tag{7}$$

$$\widehat{x} = mode(x) \tag{8}$$

Power Transformer [37] and Standard Scaler were used to scale the numerical data in this research. These approaches are part of a family of monotonic and parametric transformations designed to convert skewed features into a more normal distribution, which often utilizes logarithmic transformations to achieve a Gaussian-like data distribution. Power Transformer is particularly effective in addressing issues related to heteroscedasticity and other conditions where the assumption of normality is essential for accurate modelling. By applying these transformations, the data's variance becomes more constant, improving the performance and reliability of the subsequent ML or DL models. Eq. (9) represented the mathematical expression of the Power transformer. Here, a parameter represented as λ is determined during the fitting procedure; the input data, denoted as y, is transformed to be more Gaussian-like by selecting the optimal value of λ.

$$y(\lambda) = \begin{cases} \dfrac{((y+1)^{\lambda}-1)}{\lambda}, if y \geq 0 and\ \lambda \neq 0 \\ \log(y+1), if y \geq 0 and\ \lambda = 0 \\ -\dfrac{((-y+1)^{2-\lambda}-1)}{(2-\lambda)}, if y < 0 and\ \lambda \neq 2 \\ -\log(-y+1), if y < 0 and\ \lambda = 2 \end{cases} \tag{9}$$

3.3. Feature engineering

The ML and DL approaches employed in this work tackle data to create novel variables that were not part of the original training set. This enhancement involves operations such as mutation, combination, addition, and deletion to advance the dataset features and enhance the efficacy and efficiency of proposed models. This helps to achieve remarkable accuracy and performance. The feature selection mechanism is a key strategy that aims to lessen the dimensionality of the input variables by retaining only the most relevant features and eradicating the noise from the data.

3.3.1. Covariance matrix and feature selection

A Covariance matrix heatmap has been employed to visualize the linear correlations among various variables in the dataset. It helps to classify patterns and relationships between the data variables. Based on domain expertise, the variables with high correlations were considered redundant and excluded. Specifically, columns like Dir, SrcAddr, DstAddr, and DstMac were removed. In the covariance matrix, the correlation coefficient r_{xy} between two variables, x and y, is calculated using Eq. (10).

$$r_{xy} = \frac{\sum(x_i - \overline{x})(y_i - \overline{y})}{\sqrt{\sum(x_i - \overline{x})^2 \sum(y_i - \overline{y})^2}} \tag{10}$$

Where the mean values of x and y are represented by $\overline{x}$ and $\overline{y}$ respectively. This coefficient helps to identify which columns to drop

Table 2
Presence or absence of dataset sample types across multiple studies.

Dataset Samples	[13]	[14]	[15]	[16]	[17]	[18]	[19]	[20]	[21]	[22]	[23]	[24]	[25]	[26]	[27]	[30]	[32]	[33]	[34]	[35]
IE- IoT enabled traffic	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	-	✓	✓	✓	✓	✓
HS- Healthcare sample	×	×	×	×	×	×	×	×	×	×	✓	✓	×	✓	✓	✓	×	×	×	×
AT- Attack samples	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	-	✓	✓	✓	✓	✓

Table 3
Overview of how the proposed model addresses the identified research gaps in IoMT-Based intrusion detection systems.

Research Gap	Proposed Model Solution
Rapid growth of the IoMT devices	The proposed DL models (EmbedNet, ConvNet-SVM, DeepSVM-Net) are designed to efficiently process large-scale IoMT data, ensuring scalability and adaptability in IoMT environments.
Increased Vulnerability to Cyber Threats	Models are trained on real-time IoMT-specific datasets (ECU-IoHT, NF-BoT-IoT, WUSTL-HDRL−2024) to detect various cyber threats, including data breaches, unauthorized access, and malicious attacks.
Inadequacies of Traditional IDS Solutions	Unlike signature-based IDS, deep learning models detect complex attack patterns, reducing false positives and negatives, and adapting to evolving threats in IoMT environments.
Advancements in Deep Learning Approaches for Cybersecurity	EmbedNet, ConvNet-SVM, and DeepSVM-Net employs DL for accurate and adaptive threat detection, providing a significant improvement over traditional method.
Critical Need for IoMT-Specific IDS	The proposed models are specifically designed for IoMT environments, distinguishing between benign traffic and cyber threats like DoS, DDoS, MITM, and others.
Limited Focus on IoMT in Existing IDS Research	The research focuses on IoMT-specific datasets and models, addressing the unique security needs of IoMT rather than general-purpose networks or industrial IoT environments.
Inadequate IoMT-Specific Datasets	The use of multiple real-time datasets (ECU-IoHT, NF-BoT-IoT, WUSTL-HDRL−2024) provides comprehensive data for training DL models, overcoming the limitations of insufficient IoMT datasets.
Scalability and Real-Time Detection Challenges	The models are optimized for real-time detection by analyzing batch sizes and epochs to reduce memory usage and inference time, making them suitable for resource-constrained IoMT devices.
Integration and Interoperability Issues	DL architectures are designed to be modular and can be easily integrated into existing IoMT frameworks, ensuring compatibility with diverse medical devices and healthcare systems.
Adaptive Learning and Continuous Improvement	Adaptive learning mechanisms are incorporated to allow the models to update threat intelligence and continuously improve detection capabilities, ensuring long-term effectiveness.

based on high correlation.

3.3.2. Ordinal encoders

The ordinal encoders [38] were employed to convert categorical columns, such as Flgs and SrcMac, into integer values that reflect the order of the labels. It follows two major encoding processes i.e. assign each unique category c_i an integer value e_i as shown in Eq. (11). In another process, replace each category with its corresponding integer rank. For a categorical feature x_{cat} with categories $\{A, B, C\}$ where $rank(A) = 1, rank(B) = 2, rank(C) = 3$, the encoded feature x_{enc} becomes $\{1, 2, 3\}$.

$$e_i = rank(c_i) \tag{11}$$

Subsequently, categorical columns, such as Flgs and SrcMac, were encoded using an Ordinal Encoder. This method converts each label into an integer value, preserving the order of the labels in the encoded data.

3.3.3. Dataset splitting and cross-validation

The dataset was divided into training and testing subsets with an 80 −20% split to accurately evaluate the performance of the ML and DL models. Additionally, performance has been considered with a 70 −30%

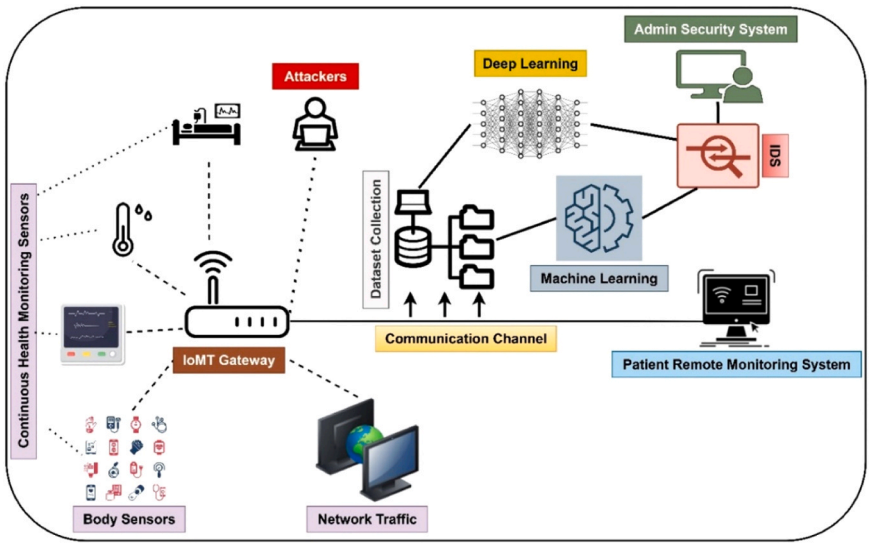

Fig. 1. IoMT architecture.

split for comparison using ML models. K-fold cross-validation with 10-fold was employed on the training dataset to illustrate the adaptability in performance across different folds.

3.3.4. Solution to data imbalance problem

The dataset posed an imbalanced challenge, where standard samples comprised approximately 88 % of the data. To address this, the Synthetic Minority Oversampling Technique (SMOTE) [39] was applied during the training phase to balance the dataset by generating new synthetic samples for the minority class. The fit_resample method from the imblearn library [40] was used to apply the SMOTE algorithm to the input data, denoted as X and y. Here, X is a $2D$ array-like structure representing the features and y is a $1D$ array-like structure representing the target labels. As expressed in Eq. (12), this resampling process ensured a more balanced dataset and improved the model's ability to identify minority class instances.

$$X\_resampled, \quad y\_resampled = smote.fit\_resample(X, \quad y) \tag{12}$$

The quantity of resampled data generated by SMOTE is influenced by the parameters provided to the SMOTE object. By default, SMOTE creates synthetic samples for the minority class until it achieves a balanced class distribution and results in an equal number of samples for both minority and majority classes. However, the degree of oversampling can be adjusted by setting the sampling_strategy parameter. For instance, if sampling_strategy is set to 0.5, SMOTE will oversample the minority class to reach 50 % of the majority class size. Consequently, the minority class will have 50 % as many samples as the majority class.

3.4. Overview of ML models and hyperparameter tuning

This paper evaluates different pre-trained machine-learning (ML) algorithms, such as LR (linear regression), XGB (Extreme Gradient Boosting), DT (Decision Tree), GBT (Gradient-Boosted Trees), KNN (K-nearest neighbour), SVM (Support Vector Machine), and RF (Random Forest), on three datasets: ECU-IoHT, NF-BaIoT, and Wustl HDRL 2024 to recognize attacks in the IoMT ecosystem. Table 4 depicts the ML classifier description with optimized hyperparameter tuning values. We employed 10-fold cross-validation for pre-trained ML model evaluation. In this process, each model underwent training with a range of parameter values to minimize the impact of data spitting and avoid overfitting.

3.5. Discussion of proposed DL approaches with training parameters

This section highlights the working architecture of deep learning (DL)-enabled proposed models, i.e. EmbedNet model, ConvNet-SVM Model, and DeepSVM (DeepSVM-Net) model, to detect intrusion in the IoMT-based ecosystem. Table 5 depicts the setup Parameters values of DL approaches, which are explained as follows:

3.5.1. EmbedNet (A Categorical Embedding Neural Network)

The EmbedNet model presented in this work provides a comprehensive solution for handling categorical and continuous features (Spatial Features) in DL, particularly in classification modelling. Unlike traditional approaches that handle these two types of features separately, the EmbedNet technique integrates them seamlessly within a unified neural network architecture, which captures complex relationships and interactions more effectively. The components of this model include Embedding layers for categorical features, batch normalization for continuous features, concatenation of features, linear layers with dropout and batch normalization (BNorm), and a final output layer as shown in Fig. 2, which are explained as follows:

i) Embedding layers for categorical features

For categorical features (x_{cat}) with cardinality (C) and embedding dimension (d), the embedding layer transforms the categorical feature into a dense vector as expressed in Eq. (13):

$$e_{cat} = Embedding(\quad x_{cat} \quad) \in \mathbb{R}^d \tag{13}$$

Which further mapped each categorical value to an index i such that $i \in \{1, 2, \ldots\ldots, C\}$, the index i is used to retrieve the corresponding vector embedding vector e_i from the embedding matrix E where $e_i = E[i]$.

ii) Batch Normalization for Continuous Features

For continuous features $x_{cont} \in \mathbb{R}^m$, batch normalization standardizes the inputs, which is expressed using Eq. (14):

$$x_{cont\_norm} = BatchNorm(x_{cont}) \tag{14}$$

This process involves the calculation of mean, variance, and normalization, which is expressed by using Eqs. (15–17)

$$\mu_{batch} = \frac{1}{m}\sum_{i=1}^{m} x_i \tag{15}$$

$$\sigma^2_{batch} = \frac{1}{m}\sum_{i=1}^{m} (x_i - \mu_{batch})^2 \tag{16}$$

Table 4
ML classifier description with optimized hyperparameter tuning values.

Models	Definitions	K-Fold	Hyperparameter	Optimized Values	Result Range
LR	Logistic Regression models the relationship between a dependent variable and independent variables using a linear function.	10	Max_iteration Random_state	100 8	0–1.0
XGB	XGBoost builds an ensemble of decision trees using gradient boosting for high-speed predictive modeling.	10	N_estimators Max_depth	1000 8	0–1.0
DT	Decision Tree splits data into branches based on feature values to determine outcomes.	10	Max_depth	8	0–1.0
GBT	Gradient Boosting Trees iteratively improve predictive accuracy by correcting previous errors.	10	Max_depth subsample Max_leaf_node N_estimators	8 0.5 1000 1000	0–1.0
KNN	K-Nearest Neighbors classifies data points based on the majority class among their closest neighbors.	10	Leaf_size N_estimators	100 2	0–1.0
SVM	Support Vector Machine classifies data by finding the optimal hyperplane to maximize class separation.	10	gamma	auto	0–1.0
RF	Random Forest constructs multiple decision trees and aggregates their outputs for accuracy.	10	Random_state N_estimators Max_leaf_node	1 1000 1000	0–1.0

Table 5
The setup parameters values of DL approaches.

Parameters	Values
Epochs	100
Embedding dimension	16
Optimizer	Adam
Batch Size	64, 128, 256
Activation Function	ReLU, TanH
Learning rate	0.0001

$$x_{cont\_norm} = \frac{x_{cont} - \mu_{batch}}{\sqrt{\sigma^2_{batch} + \in}} \tag{17}$$

iii) Concatenation of features

The transformed categorical and continuous features are concatenated to form a combined feature vector, which is expressed by Eq. (18):

$$h_0 = [e_{cat1}, e_{cat2}, e_{cat3}, \ldots\ldots, e_{catn}, x_{cat1}] \tag{18}$$

Here, h_0 represents the initial hidden state combining all processed features.

iv) Linear Layers with dropout and batch normalization

The combined feature vector h_0 is passed through a series of linear layers with dropout and batch normalization, as expressed in Eq. (19), to improve learning and prevent overfitting.

$$h_{i+1} = Dropout(BatchNorm(ReLU(W_i h_i + \quad b_i))) \tag{19}$$

Firstly, a linear transformation is applied where the vector is multiplied by the weight matrix W_i, and a bias vector b_i is added, which results in ai as expressed in Eq. (20). The output a_i is then passed through the ReLU activation function, which introduces non-linearity by setting all negative values to zero, as depicted in Eq. (21). This activated output hi is then standardized using batch normalization, which normalizes the output to improve training stability and convergence as depicted in Eq. (22) Finally, dropout is applied where a fraction of the neurons are randomly set to zero based on the dropout rate p, as represented in Eq. (23). This process reduces the risk of overfitting by preventing the model from becoming too dependent on any particular set of neurons.

$$a_i = W_i h_i + \quad b_i \tag{20}$$

Where, $W_i \in \mathbb{R}^{d_i \times d_{i-1}}$ and $b_i \in \mathbb{R}^{d_i}$ are the weights and biases of the i^{th} layer.

$$h_i \quad = ReLU(\quad a_i \quad) \tag{21}$$

$$\text{Where}, ReLU(x) \quad = \max(0, x)$$

$$h_i^{norm} = BatchNorm(h_i) \tag{22}$$

$$h_{i+1} = Dropout(h_i^{norm}, p) \tag{23}$$

v) Final output layer

The final output of the neural network performs a linear transformation followed by a sigmoid activation function to provide the prediction for binary classification as expressed in Eq. (24). In the linear transformation phase, the weight matrix W_{final} multiplied with the last hidden layer output h_{last} and the bias vector b_{final} is added to it as depicted in Eq. (25).

$$\hat{y} = \sigma(\quad W_{final} h_{last} \quad + \quad b_{final}) \tag{24}$$

$$a_{final} = W_{final} h_{last} + b_{final} \tag{25}$$

$$\text{Where}, W_{final} \in \mathbb{R}^{1 \quad \times d_{last}} \quad \text{and} b_{final} \in \mathbb{R}^{1 \quad \times d_{last}}$$

Here, W_{final} is a matrix with dimensions $1 \times d_{last}$, and b_{final} is scalar. This linear transformation produces the input for the sigmoid function (a_{final}) which then passes through the sigmoid activation function as expressed in Eq. (26) to yield the final prediction as $\hat{y} = \sigma(\ a_{final}\ )$. The sigmoid function maps the final prediction output to a probability between $0 and 1$, making it suitable for binary classification tasks to predict intrusion in the IoMT environment. Algorithm 1 explains the working architecture of the Embedding neural network model.

$$\sigma(\quad x \quad) = \frac{1}{1 + e^{-x}} \tag{26}$$

Algorithm 1. EmbedNet (A Categorical Embedding Neural Network)

```
Input:
    - cat_dim: Dimension of categorical features
    - cont_dim: Dimension of continuous features
    - cat_cardinality: Array of cardinalities of categorical features
    - embd_dim: Dimension of embeddings for categorical features
    - layers_dim: Array of dimensions for hidden layers (default: [64, 128])
    - output_dim: Dimension of the output (default: 1)
    - drop_prob: Dropout probability (default: 0.5)
    - x: Input data tensor
Output:
    - x: Output prediction of attack classification
Initialization:
Define the function to initialize layers
void initialize_layers(string activation, string initialization, list layers);
Define a function to create linear layers with dropout and batch normalization
list linear_dropout_bn(string activation, string initialization, bool use_batch_norm, int in_units, int out_units, float dropout);
class EmbedNet extends nn.Module {
    // Constructor for the EmbedNet  class
    void __init__(int cat_dim, int cont_dim, list<int> cat_cardinality, int embd_dim, list<int> layers_dim=[64, 128], int output_dim=1, float drop_prob=0.5) {
        // Call parent class constructor
        super(EmbedNet , self).__init__();
        // Initialize an empty list of layers
        list layers = [];
        // Create embedding layers for categorical features
        self.embedding_layers = nn.ModuleList();
        for (int i = 0; i < cat_cardinality.length; i++) {
            self.embedding_layers.append(nn.Embedding(cat_cardinality[i],embd_dim, padding_idx=0));
        }
        // Calculate the number of input units for the first linear layer
        int current_units = embd_dim * cat_dim + cont_dim;
        // Add linear layers with dropout and batch normalization
        for (int i = 0; i < layers_dim.length; i++) {
            list temp_layers = linear_dropout_bn("ReLU", "kaiming", true, current_units, layers_dim[i], drop_prob);
            for (int j = 0; j < temp_layers.length; j++) {
                layers.append(temp_layers[j]);
            }
            current_units = layers_dim[i];
        }
            // Combine linear layers into a sequential model
            self.linear_layers = nn.Sequential(layers);
            self.cont_norm = nn.BatchNorm1d(cont_dim);
            // Create a head layer with dropout and a final linear layer
            self.head = nn.Sequential(nn.Dropout(drop_prob), nn.Linear(current_units, output_dim));
            // Initialize head layer
            initialize_layers("ReLU", "kaiming", self.head);
    }
        // Forward pass method
        tensor forward(dict x) {
            // Separate continuous and categorical data
            tensor continuous_data = x["continuous"];
            tensor categorical_data = x["categorical"];
            // Embed categorical data
            tensor[] embedded_categorical_data = [];
            for (int i = 0; i < self.embedding_layers.length; i++) {
                tensor embedded = self.embedding_layers[i](categorical_data[:, i]);
                embedded_categorical_data.append(embedded);
            }
            // Concatenate embedded categorical data
            tensor concatenated_embeddings = torch.cat(embedded_categorical_data, 1);
            // Normalize continuous data and concatenate with embeddings
            tensor normalized_continuous_data = self.cont_norm(continuous_data);
            tensor combined_data = torch.cat([concatenated_embeddings, normalized_continuous_data], 1);
            // Pass data through linear layers
            tensor output = self.linear_layers(combined_data);
            // Pass through the head layer and apply sigmoid activation
            output = self.head(output);
            return torch.sigmoid(output);
    }
}
```

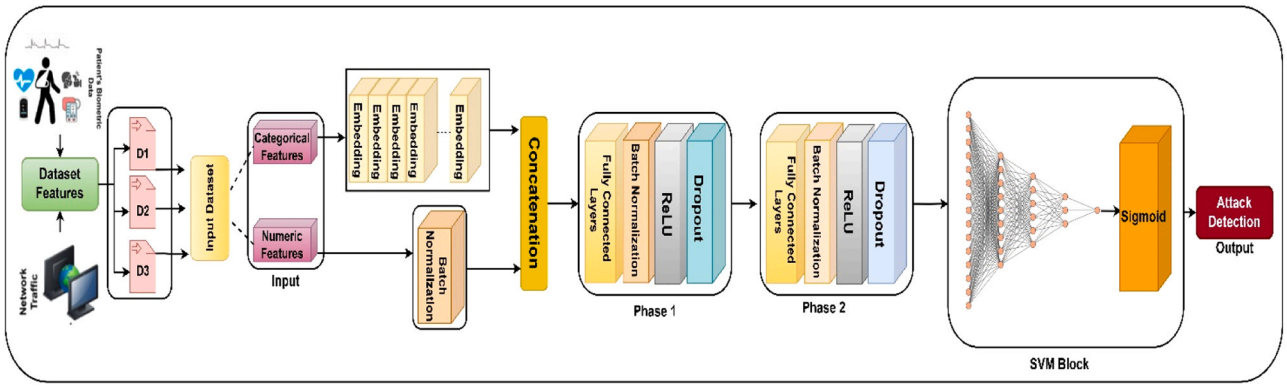

Fig. 2. Working architecture of EmbedNet model.

3.5.2. ConvNet-SVM

The convolutional neural network (CNN) and support vector machine (ConvNet-SVM) model is another proposed DL model representing a novel approach to detecting intrusion in the IoMT environment. Unlike the traditional SVM model, which relies on manual feature engineering, ConvNet-SVM employs CNNs to automatically extract relevant features from raw input data. This model is specially designed to handle high dimensional input data, which is typical in network traffic and system logs used for intrusion detection. This model provides a robust framework for accurately identifying malicious activities in the network traffic and system logs that are used for intrusion detection by automatically extracting relevant features from the raw input data.

The architecture of this model consists of several key components: convolutional layers followed by batch normalization, hyperbolic tangent activation functions (tanh), a dropout layer for regularization, adaptive average pooling, a fully connected layer (FCL), and Sigmoid Activation Function (SAF) for classification as shown in Fig. 3.

i) Convolutional Layers

The convolutional layers in this model play a vital role in feature extraction. These layers apply convolutional filters to the input data and obtain local designs and structures. By identifying these patterns, the ConvNet-SVM model can effectively detect intrusions in the IoMT environment.

As expressed in Eq. (27), the convolutional operation encompasses sliding a filter or kernel over the input data to produce feature maps.

$$Z_{i,j,k} = \tanh\left(\sum_{m-1}^{M}\sum_{n-1}^{N} X_{i+m-1,j+n-1} W_{m,n,k} + b_k\right) \tag{27}$$

Where, $Z_{i,j,k}$ represents the output feature map at the position (i,j) for the k-th filter, $X_{i,j}$ depicts the input data at position (i, j), $W_{m,n,k}$ denotes the k-th convolutional filter at position (m,n), shows the bias term for the k-th filter, and tanh(.) depicts a hyperbolic tangent activation function.

The convolutional operation is made by sliding the filter over the input data. It performs element-wise multiplication, sums the results, adds the bias term, and finally applies the tanh activation function to produce the features map.

The network begins with a dropout layer to avoid overfitting by randomly setting a fraction of the input units to zero training. A hyperbolic tangent activation function is used, which scales the output between −1 and 1 and promotes non-linearity in the model. The architecture comprises of three convolutional layers ($self.conv1, self.conv2, self.conv3$) with corresponding BN layers ($self.bn1, self.bn2, self.bn3$). the first convolutional layer transforms the input x with one channel into six feature maps using a kernel of size 3 followed by BN to stabilize the learning process. The output of the first convolutional layer is expressed using Eq. (28).

$$x_1 = Tanh(BN1(Conv1(x))) \tag{28}$$

Where, Conv1 depicts convolution operation, and BN1 denotes Batch Normalization.

This process is repeated for the subsequent layers where the number of feature maps is increased to 12, and the in_features (input features) dimension is to default 40. It is represented using Eqs. (29–30).

$$x_2 = Tanh(BN2(Conv2(x_2))) \tag{29}$$

$$x_3 = Tanh(BN3(Conv3(x_3))) \tag{30}$$

The final convolutional layer is followed by an adaptive average pooling layer, which lessens the dimensionality of each feature map to a single value. Eq. (31) effectively summarises the information.

$$x_{pool} = Pool(x_3) \tag{31}$$

A residual connection is added to integrate the original input x into the final feature representation, as depicted in Eq. (32).

$$x_{residual} = x_{pool} + x \tag{32}$$

The resulting feature vector is passed through an FCL to provide the final output, which is activated by a sigmoid function to map the output to the range $[0,1]$ as depicted in Eq. (33).

$$\hat{y} = \sigma(FC(x_{residual})) \tag{33}$$

ConvNet-SVM model architecture effectively combines the strengths of convolutional layers for feature extraction and SVM-like linear decision boundaries for classification, which makes it suitable for complex sequential data analysis.

ii) Dropout Layer

ConvNet-SVM includes a dropout layer that randomly sets a fraction of the activation to zero during training to avoid overfitting. This helps to guarantee the model generalizes well to unseen data. The mathematical formulation of the dropout layer is expressed in Eq. (34).

$$h^{(l)} = dropout(a^{(l)}, p) \tag{34}$$

Where, $a^{(l)}$depicts the activations from the previous layer, $h^{(l)}$ represents the outputs after applying dropout with rate p, and $d^{(l)}$denotes mask generated from a Bernoulli distribution with parameter $1-p$ as expressed in Eq. (35).

$$d^{(l)} \sim Bernoulli(1 - p) \tag{35}$$

iii) Adaptive Average Pooling

ConvNet-SVM model uses an adaptive average pooling layer to handle inputs of varying lengths. It ensures a fixed-size feature map output regardless of the input size. This is particularly useful in intrusion detection, where network traffic can vary in length and size, which is calculated using Eq. (36).

$$P = \frac{1}{HW}\sum_{i=1}^{H}\sum_{j=1}^{W} Z_{ij} \tag{36}$$

Where P represents the Pooled Feature, and H and W denote the height and width of the feature map.

This operation involves summing all values in the feature map and averaging them to produce a fixed-size vector.

iv) FCL and SAF

The final layer of the ConvNet-SVM model is an FCL followed by a SAF, which outputs the probability of the input being a malicious activity, which is expressed in Eq. (37).

$$\widehat{y} = \sigma(\ W_{final}\ \bullet P\ +\ b_{final}\ ) \tag{37}$$

Where, W_{final} depicts the weights of the FCL, P shows the pooled feature vector, b_{final} represents the bias term and $\sigma(x)$ denotes the SAF.

This layer produces a probability score, which makes it suitable for binary classification, such as determining whether network traffic is normal or indicative of an intrusion. Algorithm 2 explains the working architecture of the ConvNet-SVM model.

Algorithm 2. ConvNet-SVM

```
    Input:
        - in_features: Number of input features (default: 40)
        - out_features: Number of output features (default: 1)
        - x: Input data tensor
    Output:
        - x: Output prediction of attack classification
    Initialization:
        Define a function ConvNet-SVM _init to initialize the layers of the ConvNet-SVM model.
        Parameters:
        - in_features: Number of input features
        - out_features: Number of output features
void ConvNet-SVM_init(float x[], int in_features, int out_features) {
    Initialize layers
    int drop=nn.Dropout();
    int tanh=nn.Tanh();
    int conv1=nn.Conv1d(1, 6, 3, padding=1);
    int bn1=nn.BatchNorm1d(6);
    int conv2=nn.Conv1d(6, 12, 3, padding=1);
    int bn2=nn.BatchNorm1d(12);
    int conv3=nn.Conv1d(12, in_features, 3, padding=1);
    int bn3=nn.BatchNorm1d(in_features);
    int pool=nn.AdaptiveAvgPool1d(1);
    int fc=nn.Linear(in_features, out_features);
    }
    Forward Pass:
    Begin forward pass through the ConvSVM network.
void ConvSVM(float x[], int in_features, int out_features) {
    Convolutional layers
    for (int i = 0; i < n_hid_layer; ++i) {
        x = tanh(conv1(x));
        x = tanh(conv2(x));
        x = drop(bn1(x));
        x = drop(bn2(x));
        x = drop(bn3(x));
        x = pool(x);
    }
    // Fully connected layer
    for (int i = 0; i < n_hid_layer; ++i) {
        x += nn;
    }
    }
```

3.5.3. DeepSVM-net (a deep neural network emulated by support vector machine)

DeepSVM-Net is a neural network designed to emulate the behaviour of a Support Vector Machine (SVM) with a DL approach. This hybrid model uses the strengths of both neural networks and traditional SVM, which provides a robust solution for classification purposes in the context of intrusion detection in the IoMT environment. The network initialization (init) begins with the init_weights function, which applies Xavier uniform initialization to the weights of linear layers. This method is essential for maintaining the variance of the gradients during back-propagation, which is crucial for stable and efficient training of the DL model. The Xavier initialization sets the weights W using the Eq. (38).

$$W\ \sim\ u\ (\ -\frac{\sqrt{6}}{\sqrt{n_{in}\ -n_{out}}}\ ,\frac{\sqrt{6}}{\sqrt{n_{in}\ +n_{out}}}\ ) \tag{38}$$

Where, n_{in} and n_{out} are the number of input and output units in the weight set, respectively. The biases are initialized to a small constant value (0.01) to ensure a slight positive bias at the start of training.

The architecture of DeepSVM-Net, as shown in Fig. 4, includes an input layer, a middle-hidden layer, and an output layer. This structure is suitable for detecting anomalies and intrusions in the IoMT environment by learning complex patterns in the data. The model is designed using the sequential neural network container, which allows for easy stacking of layers. The forward pass of the model applies these layers sequentially to the input data. This model architecture combines multiple hidden layers with non-linear activations and weights initialization, which

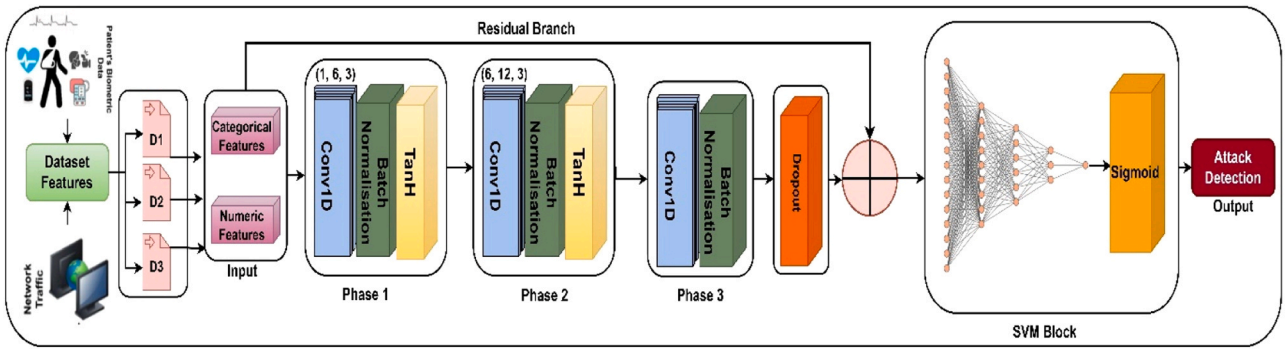

Fig. 3. Working architecture of ConvNet-SVM model.

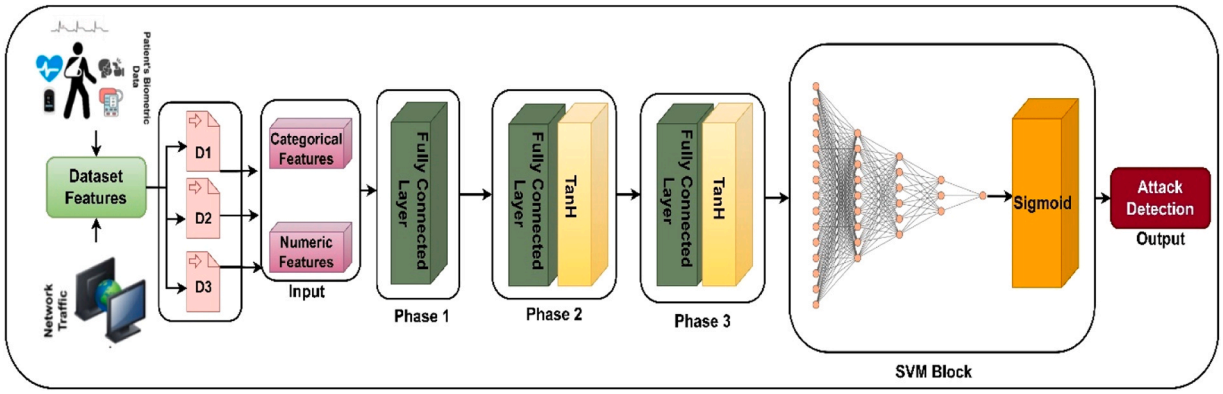

Fig. 4. Working architecture of DeepSVM-Net model.

depicts complex patterns in the data and provides a remarkable model for intrusion detection in the IoMT.

i) Input Layer

The input layer transforms the input features into a hidden representation of size, which is expressed using Eq. (39).

$$h_1 = Tanh(W_1 x + b_1) \tag{39}$$

Where, x is the input vector of features, W_1 represents the weight matrix of the first layer, b_1 denotes the bias vector, *Tanh* depicts the hyperbolic tangent activation function, and h_1 represents the output of the first layer.

ii) Hidden Layers

The hidden layers consist of stacked, FCL with Tanh activation functions. For the i-the hidden layer, the transformation is given by Eq. (40):

$$h_{i+1} = Tanh(\quad W_{i+1} \quad h_1 + \quad b_{i+1}) \tag{40}$$

Where, $i = 1, 2, \ldots\ldots, n_{hid}$, h_{i+1} represents the output of the $(i + 1)$-th layer, h_{i+1} shows the weight matrix of the $(i + 1)$-th layer, and b_{i+1} depicts the bias vector.

This repeated application of linear transformation and non-linear activations permits the network to learn complex patterns and representations in the data. This is crucial for identifying subtle anomalies indicative of security breaches in IoMT devices.

iii) Output Layer

The output layer maps the last hidden representation to the output with a single linear layer, which is followed by a SAF to produce a probability score for binary classification, which can be expressed using Eq. (41).

$$\hat{y} = (\quad W_{out} h_{n_{hid}} + \quad b_{out}) \tag{41}$$

Where, W_{out} represents the weight matrix and bias vector of the output layer, σ depicts the sigmoid function.

The final output $\hat{y}$ represents the probability of the input being classified as an intrusion. The model can effectively flag potential security threats by setting an appropriate threshold that enhances the reliability and security of IoMT systems. This research highlights the efficacy of the hybrid model in improving predictive performance and generalization capabilities, which are essential for maintaining security in dynamic and complex IoMT environments. Algorithm 3 explains the working architecture of the DeepSVM-Net model.

Algorithm 3. DeepSVM-Net

```
Input:
    - in_features: Number of input features
    - hidden_size: Size of the hidden layers (default: 32)
    - out_features: Number of output features (default: 1)
    - n_hid_layer: Number of hidden layers (default: 3)
    - x: Input data tensor
Output:
    - x: Output prediction of attack classification
Initialization:
    Define a function init_weights to initialize the weights and biases of the linear layers.
        - For each linear layer in the model:
            - Initialize weights using Xavier uniform initialization.
            - Initialize biases to a constant value (0.01).
void init_weights(float weights[], float biases[], int sizes[], int num_layers) {
    Initialize weights using Xavier uniform initialization
    for (int i = 0; i < num_layers; ++i) {
        Assuming Xavier initialization for weights
        float variance = 2.0 / (sizes[i] + sizes [i + 1]);
        for (int j = 0; j < sizes[i] * sizes [i + 1]; ++j) {
            weights [i * sizes[i + 1] + j] = sqrt(variance) * (float)rand() / RAND_MAX;
        }
            Initialize biases to 0.01
        biases[i] = 0.01;
    }
}
    SVM Initialization:
        Initialize SVM object with specified input and output features, hidden layer size, and number of hidden layers.
            - Initialize activation function as hyperbolic tangent (Tanh).
float svm(float x[], int in_features, int hidden_size, int out_features, int n_hid_layer) {
        Construct the layers of the model
    int sizes[n_hid_layer + 2];  // Sizes of layers including input and output
    sizes[0] = in_features;
    sizes[n_hid_layer + 1] = out_features;
    for (int i = 1; i <= n_hid_layer; ++i) {
        sizes[i] = hidden_size;
    }
        Create weight and bias arrays
    float weights[(n_hid_layer + 1) * hidden_size * hidden_size];
    float biases[n_hid_layer + 1];
    //Initialize weights and biases
    init_weights(weights, biases, sizes, n_hid_layer + 1);
    // Create the model layers
    float layers[(n_hid_layer + 1) * hidden_size + out_features];
         for (int i = 0; i <= n_hid_layer; ++i) {
        // Input layer
        for (int j = 0; j < sizes[i + 1] * sizes[i + 2]; ++j) {
            layers[i * sizes[i + 1] + j] = (float)rand() / RAND_MAX;
        }
        // Hidden layers
        for (int j = 0; j < sizes[i + 1] * sizes[i + 2]; ++j) {
            layers[i * sizes[i + 1] + j] = (float)rand() / RAND_MAX;
        }
        // Output layer
        for (int j = 0; j < sizes[i + 1] * sizes[i + 2]; ++j) {
            layers[i * sizes[i + 1] + j] = (float)rand() / RAND_MAX;
        }
    }
    // Forward Pass:
    // Begin forward pass through the SVM network.
    for (int i = 0; i < n_hid_layer; ++i) {
        // Pass input tensor x through the model
        // Apply the sigmoid activation function to the output tensor
        layers[i] = sigmoid(layers[i]);
    }
    // Return the final prediction result of the attack classification
    return layers[n_hid_layer + 1];
}
End Algorithm
```

Table 6
Training parameters of the proposed DL approaches.

Training Parameters	DL Models					
	EmbedNet		ConvNet-SVM		DeepSVM-Net	
Initial state or attributes	layers_dim drop_prob output_dim	64, 128 0.5 1	in_features out_features	40 1	in_features, out_features hidden_size n_hid_layer	 1 32 3
Learning Rate	0.0001		0.0001		0.0001	
Optimizer	Adam		Adam		Adam	
AdaptiveAvgPool1d	×		✓		×	
Batch Size	64, 128, 256		64, 128, 256		64, 128, 256	
Fully Connected Layer	✓		×		✓	
BatchNorm1d	✓		✓		×	
Conv1D	×		✓		×	
Activation Function	ReLU		TanH		TanH	
SVM Block	✓		✓		✓	
Dropout	✓		✓		×	
Residual Branch	×		✓		×	

These DL-based models have been evaluated using three different benchmark datasets like ECU-IoHT, NF-BoT-IoT, and WUSTL-HDRL-2024 dataset. Out of these three datasets, two datasets, i.e. ECU-IoHT and WUSTL-HDRL-2024 datasets, are generated by combining the IoT traffic with healthcare features samples. Table 6 depicts the training parameters of the proposed DL approaches.

Fig. 5 depicts the workflow of the Proposed DL model architecture. In this research, we designed three deep learning models: EmbedNet, ConvNet-SVM, and DeepSVM-Net, which are evaluated on three different real-time datasets: ECU-IoHT, NF-BoT-IoT, and Wustl-HDRL-2024. These datasets are comprised of normal IoT-based traffic flow and attack samples. To begin with, the data preprocessing phase employed several techniques, including power transformers, simple imputation, and standard scaling, to clean and standardize the raw data for effective model training. Feature engineering played a crucial role in this phase, involving methods such as the SMOTE to address class imbalance and using a covariance matrix to identify and remove irrelevant features. These steps were integral in enhancing the training process and improving the models' ability to detect attacks. In addition to data preprocessing, the dataset was split into training and testing sets using 70–30 and 80–20 distributions. This organized approach allowed for a thorough evaluation of the ML and DL models across different training and testing scenarios. This research incorporated DL models, including EmbedNet, ConvSVM-Net, and DeepSVM-Net, which were specifically designed to advance network-based attack detection in IoMT ecosystem. The effectiveness of these models was rigorously assessed through binary classification methods aimed at distinguishing between benign network traffic and various threats, including Smurf attacks, ARP Spoofing, Nmap PortScan, DoS, DDoS, MITM, Reconnaissance, and Ransomware. The evaluation metrics demonstrated that the DL models achieved superior performance in attack detection, as demonstrated by enhanced accuracy, PPV, TPR, and F1 scores while maintaining minimal inference time. The use of advanced data augmentation techniques, feature segregation strategies, and systematic column reduction via covariance matrix analysis optimized memory usage and reduced prediction time. However, the integration of these DL models into the framework underscores their efficacy in strengthening network security for IoMT environments.

The proposed models i.e., EmbedNet, ConvNet-SVM, and DeepSVM-Net differ in both structure and theoretical foundation, offering distinct advantages for intrusion detection in IoMT environments. EmbedNet is specifically designed to integrate categorical and continuous spatial features within a deep learning framework, employing embedding layers, batch normalization, and dropout mechanisms to enhance feature representation. Unlike traditional approaches that separately process categorical and numerical data, EmbedNet unifies them, making it particularly effective for analyzing diverse IoMT data sources. ConvNet-SVM, in contrast, combines the feature extraction power of CNNs with the classification strength of SVMs, making it highly suitable for structured network traffic data. CNNs capture spatial correlations and hierarchical patterns in network logs, while SVMs provide robust decision boundaries, improving generalization in intrusion detection. Lastly, DeepSVM-Net bridges neural networks and SVM principles, incorporating SVM-based margin maximization within a deep learning structure. This hybrid approach ensures the model not only learns hierarchical feature representations but also benefits from SVM's capacity to distinguish between complex decision boundaries, making it highly effective for detecting subtle attack patterns. These models collectively address IoMT-specific challenges such as heterogeneous data formats, varying attack signatures, and real-time anomaly detection, thereby enhancing security and resilience in IoMT systems.

4. Performance evaluation metrics and experimental result

This section provides in-depth applicability of ML classifier and proposed DL models on three benchmark real-time datasets. This section includes subsections like experimental setup and overview, dataset description, and performance evaluation metrics.

4.1. Experimental setup

The experiment was conducted on a Windows 11 system with 32 GB DDR5 RAM and 13th Generation Intel Core i7-13620H Processor operating at 4.9 GHz. Python libraries such as PyTorch, which is well known for its deep neural network capabilities, and Scikit-learn libraries [41], which provide a robust software environment for training and testing ML and DL models, were used within the Visual Studio Code (VSCode) platform.

4.2. Dataset description

In this research, the DL models have been trained and tested using three real time datasets i.e., ECU-IoHT, NF-BoT-IoT, and WUSTL-HDRL-2024. Out of these three datasets, two are IoMT-based datasets, which comprise both network and healthcare samples, which are explained as follows:

4.2.1. ECU-IoHT

We evaluated our deep learning model using the ECU-IoHT (Edith Cowan University - Internet of Health Things) dataset [26], which includes network flow activity and cyber-attack samples in the healthcare sector. This dataset has been generated using the Windows 10 operating system, Kali Linux, a mobile Wi-Fi hotspot, a wireless network adapter, and a Bluetooth adapter. These are all interconnected devices that

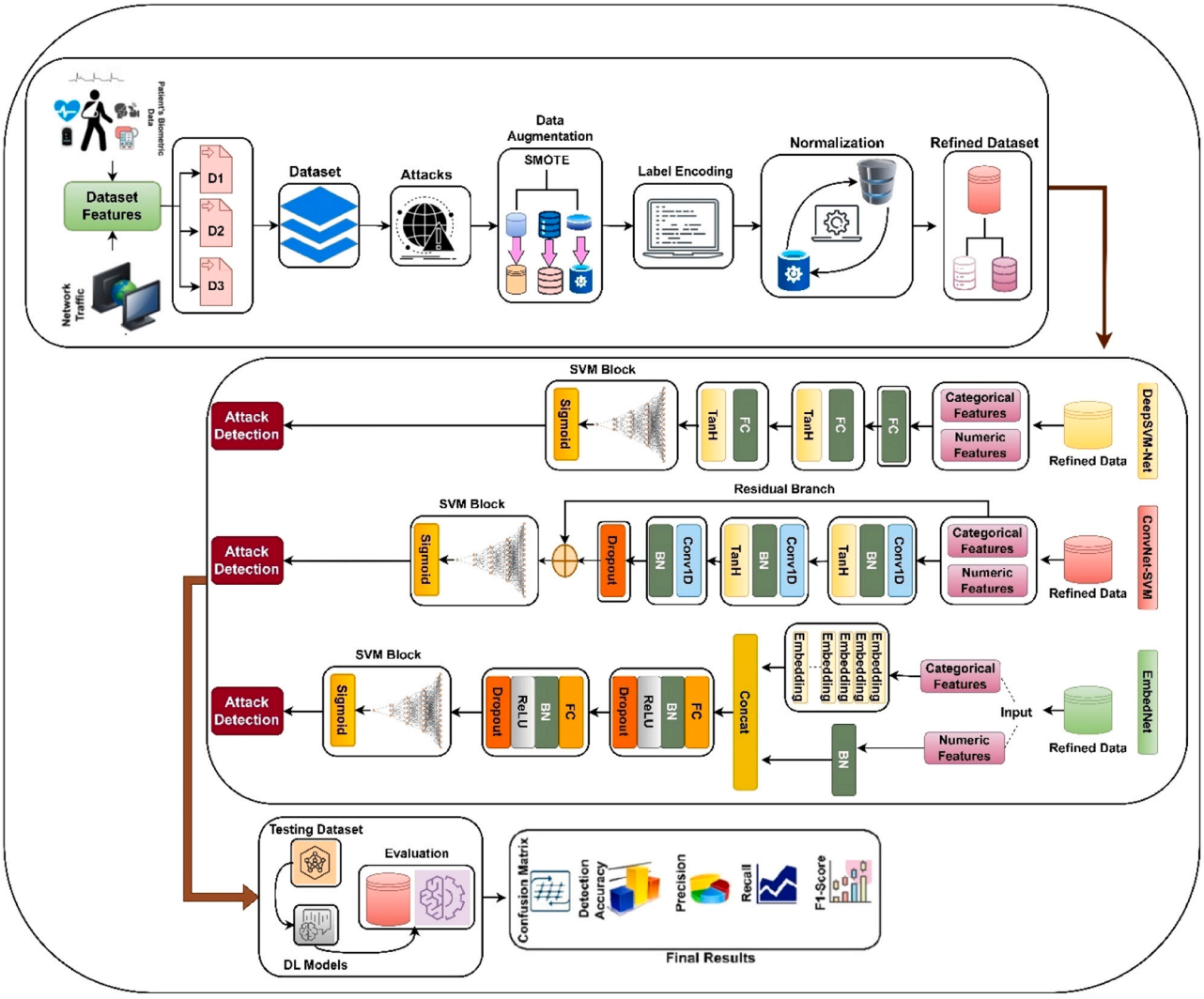

Fig. 5. Workflow of proposed DL models architecture.

provide internet access for the hosts. The setup also incorporates a healthcare kit named MySignals, which is equipped with various sensors that monitor and record patients' physiological data, including heart rate (HR), blood pressure (BP), and body temperature (BT). These sensor data are then transmitted to users' cloud storage. The ECU-IoHT dataset comprises seven key network data features: source (Src), destination (Dst), protocol, and specific attack types. It contains 23,453 instances of regular network activity and numerous cyber-attack instances, as depicted in Table 7. We classify these attacks into four categories: Smurf attacks, Address Resolution Protocol (ARP) spoofing, Network Mapper (Nmap) port scans, and Denial-of-Service (DoS) attacks.

4.2.2. NF-BoT-IoT

The NF-BoT-IoT (NetFlow version of the UNSW-Bot-IoT dataset) dataset [42] is an IoT NetFlow-based dataset derived from the BoT-IoT dataset. It was constructed by extracting features from publicly accessible pcap files and categorizing flows based on their respective attack types. This dataset comprises 600,100 data flows, where 97.69 % are classified as attack instances and 2.31 % as benign, as depicted in Table 8. It encompasses four distinct attack categories: Theft, Reconnaissance, DoS, and DDoS attack.

4.2.3. WUSTL-HDRL-2024

The WUSTL-HDRL-2024 dataset's [43] testbed has been designed to emulate a dynamic 5 G network environment. This dataset addresses the need for comprehensively capturing datasets with a wide range of interactions and security threats within 5 G networks. It includes six essential components: a 5 G Core, Local Network, MEC (Multi-access Edge Computing) servers, UE (User Equipments), Insider Attacker, and Routing system. An Ubuntu 20.04 computer employs Simu5G to simulate 5 G network operations, which expands its capabilities to include multiple MECs and external hosts so that Data can originate from the 5 G Core. The local network uses stateful IP/ICMP translation (SIIT) to bridge IPv6 and IPv4 communications and resolve compatibility issues within the simulated 5 G environment. MEC servers are critical in managing edge computing tasks, monitoring network traffic, and detecting intrusions. UEs with various operating systems connect directly to the 5 G network or via the Local Network, which presents diverse end-user scenarios. An Insider Attacker machine executes a series of attacks to evaluate the network's defensive strategies. Finally, a central Router oversees data management in the testbed, ensuring the development of a comprehensive and unpredictable attack dataset for thorough security analysis. The dataset includes four categories of security breaches: Man-in-the-Middle (MiTM) attacks, Distributed Denial of Service (DDoS) attacks, Ransomware, and Buffer Overflow attacks.

To begin with, MiTM attacks involve manipulating or intercepting data exchanged between UEs and MEC systems, which risks data confidentiality and integrity. DDoS attacks inundate MECs with large requests and disrupt their regular operations. Ransomware encrypts critical files and demands payment for decryption, whereas Buffer

Overflow attacks exploit software vulnerabilities to execute unauthorized code. Table 9 depicts the features of the WUSTL HDRL 2024 dataset and the total number of samples in Table 10.

4.3. Evaluation metrics

This subsection describes performance metrics used to assess the performance of ML and DL models for detecting intrusion in the IoMT environment. Table 11 provides a detailed description of evaluation metrics, value range and their significance in identifying attacks and normal traffic samples using datasets.

Ac- Accuracy, PPV- Positive predictive value, TPR- True positive rate, F1- F1-Score, TNR, True negative rate, MCC- Matthews correlation coefficient, NPV- Negative predictive value, ROC_AUC-Area Under the Receiver Operating Characteristic Curve, FDR- False discovery rate, FPR- False positive rate, FNR- False negative rate, FOR- False omission rate, LR+ - Positive likelihood ratio, LR- - Negative likelihood ratio, MK-Markedness, BM- Informedness.

4.4. Experimental overview

This subsection explores the effectiveness of machine learning (ML) and deep learning (DL) models in detecting intrusions within the IoT ecosystem. The performance of these models has been assessed using standard metrics, including accuracy, PPV, TPR, F1-score, TNR, MCC, NPV, and ROC-AUC. Following the initial performance evaluation, an error analysis was also conducted employing metrics such as FOR, FDR, FPR, FNR, likelihood ratios (LR+ and LR-), MK, and BM. The experimental flow of the research is illustrated in Fig. 6. The experimental framework utilized three datasets containing IoT traffic flows with cyber-attack samples: ECU-IoHT, NF-BoT-IoT, and WUSTL-HDRL-2024. The data preprocessing phase employed several techniques, including power transformers, simple imputation, and standard scaling, to clean and standardize the raw data for effective model training. Feature engineering played a crucial role in this phase, involving methods such as SMOTE to address class imbalance and using a covariance matrix to identify and remove irrelevant features. These steps enhanced the training process and the model's ability to detect attacks. In addition to data preprocessing, the dataset was split into training and testing sets using 70–30 and 80–20 distributions. This organized approach thoroughly evaluated the ML and DL models across different training and testing scenarios. This research incorporated several DL models, including EmbedNet, ConvSVM-Net, and DeepSVM-Net, specifically designed to advance network-based attack detection in the IoMT ecosystem. The EmbedNet approach is proposed as a novel embedding technique introduced in this research, which is utilized for classifying attacks in IoT-based networks. The effectiveness of these models was rigorously assessed through binary classification methods aimed at distinguishing between benign network traffic and various threats, including Smurf attacks, ARP Spoofing, Nmap PortScan, DoS, DDoS, MITM, Reconnaissance, and Ransomware. The evaluation metrics demonstrated that the DL models achieved superior performance in attack detection, as demonstrated by enhanced accuracy, PPV, TPR, and F1 scores while maintaining minimal inference time. Advanced data augmentation techniques feature segregation strategies and systematic column reduction via covariance matrix analysis optimized memory usage and reduced prediction time. However, integrating these DL models into the framework underscores their efficacy in strengthening network security for IoMT environments.

Table 7
Description of data features in ECU-IoHT.

Dataset Samples	Description	Count	After SMOTE
Smurf Attack	A DoS attack where large numbers of ICMP echo requests are sent to IP broadcast addresses, overwhelming the target with responses.	77,920	77,920
ARP Spoofing	An attack that links the attacker's MAC address with the IP address of a legitimate network device, allowing interception of data.	2359	77,920
Nmap PortScan	A technique to discover open ports and services on a target system by sending various packets and analyzing responses.	6836	77,920
DoS Attack	An attack that makes a service unavailable by overwhelming it with illegitimate requests, exhausting resources or bandwidth.	639	77,920
Benign	Normal, non-malicious network traffic represents typical communication within the network.	23,453	23,453

Table 8
Description of data features in NF-BoT-IoT.

Dataset Samples	Description	Count	After SMOTE
Theft	Attacks focused on acquiring sensitive information, such as through data theft or keylogging.	1909	4,70,655
Reconnaissance	Techniques used to gather information about network hosts are also referred to as probing activities.	4,70,655	4,70,655
DoS	DoS attacks are intended to overwhelm system resources, rendering services unavailable.	56,833	4,70,655
DDoS	DDoS attacks that involve multiple sources aiming to disrupt service.	56,844	4,70,655
Benign	Non-malicious traffic represents normal network activity.	13,859	13,859

4.5. Performance analysis of machine learning models

This subsection presents the performance analysis of ML models on the three real-time datasets which are explained as follows:

4.5.1. Performance analysis of ML models for 10-fold cross-validation

Table 12 compares ML models' performance for 10-fold cross-validation using pre-trained ML models: LR, XGB, DT, GBT, KNN, SVM and RF after hyperparameter tuning. All models have achieved remarkable average accuracy in detecting attacks on three different datasets: ECU_IoHT (D1), NF-BoT-IoT (D2), and WUSTL-HDRL-2024 (D3). Out of these models, the XGB approach achieved the highest performance on every dataset with an average accuracy of 0.9979 (D1), 0.9999 (D2), and 0.9998 (D3), whereas the LR model achieved the lowest accuracy of 0.9394 on D2.

4.5.2. Performance evaluation of ML models using diverse IoT-enabled datasets

This subsection offers a performance assessment of ML Models using Different IoT-enabled datasets as follows:

(i) Performance analysis using ECUIoHT dataset (Dataset-1)

The performance of the ML models has been assessed using diverse metrics, as shown in Table 13, which helps to provide a comprehensive grasp of the pre-trained models on the 70–30 and 80–20 dataset split. XGB model achieved the highest accuracy of 0.9979, PPV of 0.9961, TPR of 0.9913, F1-Score of 0.9937, and TNR of 0.9961 in 0.0017 s. In contrast, the LR model achieved the lowest accuracy of 0.9778 in detecting attacks using the ECU_IoHT dataset. The other models, DT, GBT, KNN, SVM, and RF, also achieved a remarkable accuracy of 0.9940, 0.9967, 0.9952, 0.9929, and 0.9977, respectively. DT takes less prediction time of 0.0031 s to detect attacks in the IoMT environment.

In addition to the balanced performance, the XGB model attained high F1-Score and MCC values of 0.9937 and 0.9871, respectively. The other models, like SVM and RF, also demonstrate balanced performance

Table 9
Description of data features in WUSTL-HDRL-2024.

Column A	Description	Column B	Description
Dir	Direction of the network flow	RTime	Real-time at which the network flow occurred.
Flgs	Flags indicating the state or features of the network flow.	Packet_num	Sequential number of the packet in the flow.
SrcAddr	Source IP address from which the network traffic originates.	scputimes_user	CPU time spent in user mode.
DstAddr	Destination IP address to which the network traffic is directed.	scputimes_nice	CPU time spent in user mode with low priority.
Sport	Source port number used in the network flow.	scpustats_ctx_switches	Number of context switches.
Dport	Destination port number used in the network flow.	scpustats_interrupts	Number of interrupts handled.
SrcBytes	Number of bytes sent from the source.	scpustats_soft_interrupts	Number of software interrupts handled.
DstBytes	Number of bytes sent to the destination.	scpustats_syscalls	Number of system calls made.
SrcLoad	Load on the source during the network flow.	svmem_total	Total physical memory available.
DstLoad	Load on the destination during the network flow.	svmem_available	Available physical memory.
SrcGap	Gap or delay in the source transmission.	svmem_percent	Percentage of memory used.
DstGap	Gap or delay in the destination transmission.	svmem_used	Amount of memory used.
SIntPkt	Interval between packets sent from the source.	svmem_free	Amount of free memory.
DIntPkt	Interval between packets sent to the destination.	svmem_active	Amount of active memory.
SIntPktAct	Actual interval between packets from the source.	svmem_inactive	Amount of inactive memory.
DIntPktAct	Actual interval between packets to the destination.	svmem_buffers	Amount of memory used for buffers.
SrcJitter	Variability in packet arrival time from the source.	svmem_cached	Amount of memory used for cache.
DstJitter	Variability in packet arrival time to the destination.	svmem_shared	Amount of memory shared between processes.
sMaxPktSz	Maximum packet size sent from the source.	svmem_slab	Amount of memory used by the kernel data structures.
dMaxPktSz	Maximum packet size sent to the destination.	ram_usage_warning	Indicator for high RAM usage warning.
Dur	Duration of the network flow.	sswap_total	Total swap memory available.
Trans	Number of transmissions during the network flow.	sswap_used	Amount of swap memory used.
TotPkts	Total number of packets in the network flow.	sswap_free	Amount of swap memory free.
TotBytes	Total number of bytes in the network flow.	sswap_percent	Percentage of swap memory used.
network_load	Load on the network during the flow.	sswap_sin	Swap memory swapped in from disk.
Loss	Packet loss during the network flow.	sswap_sout	Swap memory swapped out to disk.
pLoss	Percentage of packet loss during the network flow.	sdiskusage_total	Total disk space available.
pSrcLoss	Percentage of packet loss from the source.	sdiskusage_used	Amount of disk space used.
pDstLoss	Percentage of packet loss to the destination.	sdiskusage_free	Amount of free disk space.
Rate	Transmission rate of the network flow.	sdiskusage_percent	Percentage of disk space used.
SrcMac	MAC address of the source.	Boot_Time_with_date	Boot time of the system with date.
DstMac	MAC address of the destination.	DTime	Time duration of the network flow.
IMEI	International Mobile Equipment Identity, relevant for mobile network flows.	Attack_categories	Categories of attacks identified in the network flow.

Table 10
Total number of samples in WUSTL-HDRL-2024.

Samples	Types	Samples Values	After SMOTE
Total no. of attack samples	DDoS	9971	132884
	MiTM	1672	132884
	Ransomware	528	132884
	Buffer_Overflow	68	132884
Total no. of normal samples	Benign	132884	132884

with an F1-Score above 0.98 across both splits. Moreover, depending on the class distribution in our dataset, metrics like PPV, TPR, FNR and FDR play crucial roles in class imbalance distribution. The SVM model accomplished the highest PPV value of 0.9998, followed by DT (0.9993) and XGB (0.9961). It also gained the lowest FDR (0.0002), followed by the GBT model (0.0007). While most models have low FPR across both splits, some models like KNN and DT might have higher FNR, as lower TPR values indicate.

(ii) Performance analysis using NF-BoT-IoT dataset (Dataset-2)

The performance of the ML models has been assessed using the NF-BoT-IoT dataset to understand the pre-trained models' effectiveness comprehensively. Table 14 shows that the XGB model achieved the highest accuracy of 0.9999, PPV of 0.9998, TPR of 0.9996, F1-Score of 0.9997, and TNR of 0.9998 in 0.9515 s. In contrast, the LR model achieved the lowest accuracy of 0.9394 in detecting attacks using the NF-BoT-IoT dataset.

The other models, DT, GBT, KNN, SVM, and RF, also achieved a remarkable accuracy of 0.9962, 0.9998, 0.9996, 0.9975, and 0.9998, respectively. Notably, the DT had the shortest prediction time of 0.0103 s for attack detection in the IoMT environment, whereas KNN and SVM took a higher prediction time of 30.972 s. In addition, KNN and

Table 11
Performance evaluation metrics.

Metrics	Description	Significance	Expression	Value Range
Ac	The proportion of true results (both true +ve and true -ve) among the total number of cases examined.	It measures the overall correctness of the IDS in identifying both intrusions and normal activities, offering a broad performance overview.	$Ac = \frac{TP + TN}{TP + TN + FP + FN}$	0–1
PPV	The proportion of true +ve among all +ve results predicted by the model.	It evaluates the proportion of correctly identified intrusions out of all predicted intrusions, ensuring the IDS minimizes false alarms.	$PPV = \frac{TP}{TP + FP}$	0–1
TPR	The proportion of actual +ve correctly identified by the model.	It indicates the IDS's ability to detect actual intrusions, emphasizing the importance of catching all possible threats.	$TPR = \frac{TP}{TP + FN}$	0–1
F1	The harmonic mean of Precision and Recall, providing a balance between the two.	It balances precision and recall, providing a single metric that considers both false +ve and false -ve, crucial for optimizing IDS configurations.	$F1 = \frac{2 \times TPR \times PPV}{TPR + PPV}$	0–1
TNR	The proportion of actual -ve correctly identified by the model.	It assesses the IDS's capability to correctly identify normal activities, reducing the likelihood of flagging benign activities as intrusions.	$TNR = \frac{TN}{TN + FP}$	0–1
MCC	A balanced measure that accounts for true and false +ve and -ve, giving a correlation between observed and predicted binary classifications.	It offers a balanced measure of the IDS's performance, accounting for true and false +ve and -ve, thus reflecting the overall prediction quality.	$MCC = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN) \quad (TN + FP)(TN + FN)}}$	−1–1
NPV	The proportion of true -ve among all -ve results predicted by the model.	It measures the proportion of correctly identified normal activities out of all predicted normal activities, ensuring the IDS accurately recognizes benign behaviours.	$NPV = \frac{TN}{TN + FN}$	0–1
ROC_AUC	A measure of the model's ability to distinguish between classes, with higher values representing better performance.	It evaluates the IDS's ability to distinguish between intrusions and normal activities, with higher values indicating better discrimination capabilities.	$\text{ROC\_AUC} = \int_0^1 TPR \; d(FPR)$	0.5–1
FDR	The proportion of false +ve among all +ve results predicted by the model.	It measures the proportion of false +ve among all predicted intrusions, helping to reduce unnecessary alerts that can overwhelm security personnel.	$FDR = \frac{FP}{FP + TP}$	0–1
FPR	The proportion of actual -ve incorrectly classified as +ve by the model.	It indicates how often normal activities are incorrectly classified as intrusions, crucial for minimizing disruption to legitimate users.	$FPR = \frac{FP}{FP + TN}$	0–1
FNR	The proportion of actual +ve was incorrectly classified as -ve by the model.	It measures the proportion of intrusions missed by the IDS, highlighting the importance of minimizing undetected threats.	$FNR = \frac{FN}{FN + TP}$	0–1
FOR	The proportion of false -ve among all -ve results predicted by the model.	It evaluates the proportion of intrusions among all predicted normal activities, ensuring the IDS does not overlook potential threats.	$FOR = \frac{FN}{FN + TN}$	0–1
LR+	The ratio of the probability of a +ve test result in patients with the condition to the probability in patients without the condition.	It quantifies how much more likely a +ve result is to indicate an intrusion rather than normal activity, enhancing the IDS's diagnostic power.	$LR+ = \frac{TPR}{FPR}$	> 1
LR-	The ratio of the probability of a -ve test result in patients with the condition to the probability in patients without the condition.	It assesses how much less likely a -ve result is to indicate an intrusion, helping to confirm the absence of threats.	$LR- = \frac{TPR}{FPR}$	< 1
MK	The difference between PPV and FDR.	It reflects the difference between precision and FDR, ensuring the IDS's predictions are reliable and meaningful.	$MK = \text{Pr} + NPV - 1$	−1–1
BM	The difference between TPR and FPR reflects the probability that predictions are informed.	It indicates the probability that the IDS's predictions are informed and not random, providing a measure of its overall effectiveness.	$BM = TPR + TNR - 1$	−1–1

SVM models have underperformed in terms of TPR, and F1-Score compared to other approaches. This might indicate limitations in handling the specific classification tasks.

In error analysis, the XGB model attained the lowest FDR (0.0002), FPR (0.0001), FNR (0.0003), and FOR (0.0004) values, followed closely by the GBT model FDR value at 0.0002, FPR value at 0.0001, FNR value at 0.0004, and FOR value at 0.0005. Furthermore, XGB accomplished the highest MK, BM, and LR+ values of 0.9994, 0.9994, and 9996, respectively, along with a lower LR- rate of 0.0003. These standard metrics underscore the XGB model's ability to accurately detect and classify attacks in the IoMT environment by minimizing false positives and false negatives. It indicates its robustness and reliability in critical security applications.

(iii) Performance analysis using the WUSTL-HDRL-2024 dataset (Dataset-3)

The performance of the ML models has been assessed using the WUSTL-HDRL-2024 dataset to understand the pre-trained models' effectiveness comprehensively.

Table 15 shows that the XGB model achieved the highest accuracy of 0.9998, PPV of 0.9999, TPR of 0.9998, F1-Score of 0.9998, and TNR of 0.9999 in 0.0215 s. It is followed closely by the models: LR (0.9997), DT (0.9972), GBT (0.9980), KNN (0.9998), SVM (1.0) and RF (0.9996) accuracy score. Notably, the LR and DT approach had the shortest prediction time of 0.0009 s for attack detection in the IoMT environment. In contrast, KNN had a higher prediction time of 1.7376 s than other classifiers. Moreover, Error metrics such as FDR, FPR, FNR, and FOR are crucial for understanding the model's limitations. XGB demonstrated the lowest error rates, with FDR and FPR as low as 0.0002 and 0.0001, respectively, in the 80–20 split, which indicates the minimal false discoveries and positives rate. SVM also reported low error rates, which emphasizes its accuracy and reliability. Furthermore, the LR+ and LR- reflect the diagnostic power of the models. XGB demonstrated the highest LR+ of 9997.9 and the lowest LR- of 0.0002, highlighting its discriminative solid capability. Finally, the XGB model attained the highest MK and BM score of 0.9994, proving its superior ability to accurately detect and classify attacks in the IoMT environment by minimizing false positives and false negatives.

The comprehensive evaluation of these models indicates that XGB,

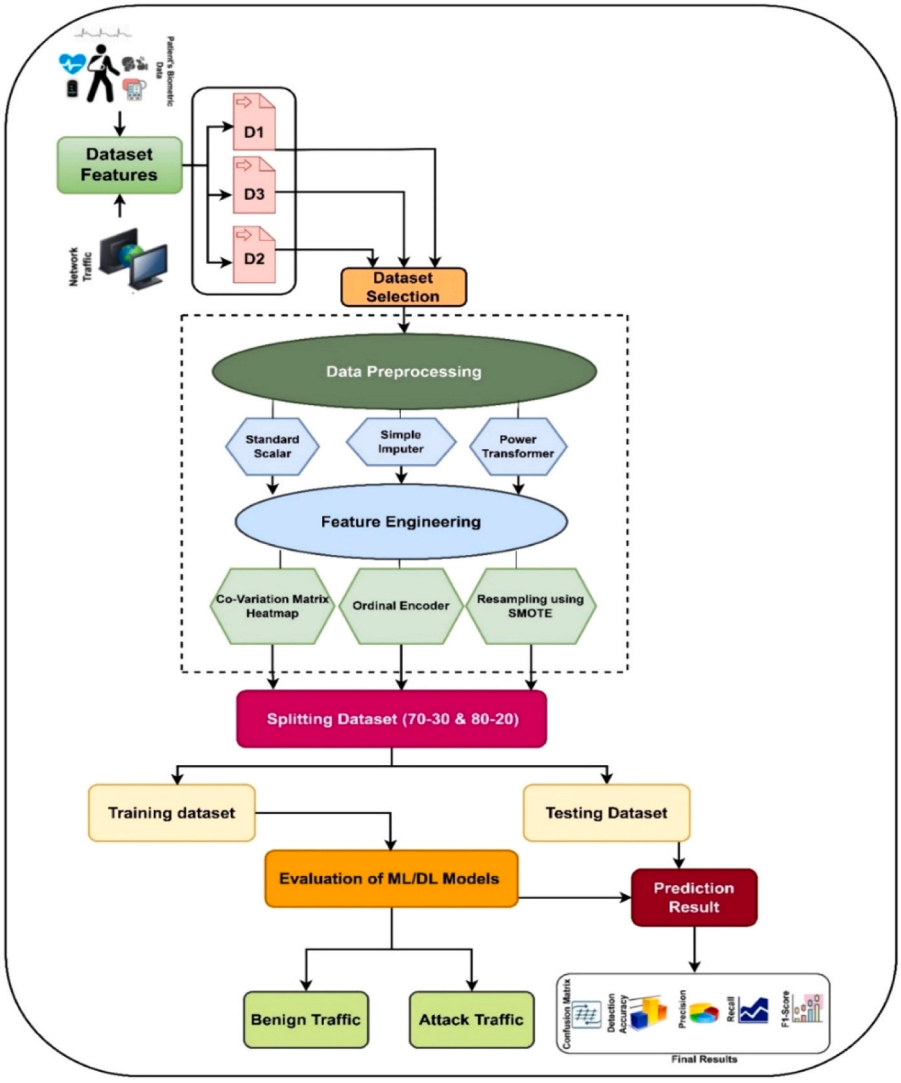

Fig. 6. Experimental flowchart of proposed framework for detecting attacks in IoMT environment.

an SVM, consistently outperformed others across most metrics. They demonstrated remarkable accuracy, PPV, TPR, F1 scores, and low error rates, which made them highly reliable for detecting attacks in the IoMT environment. Their robust performance across different data splits highlights their suitability for real-world applications where accuracy and reliability are paramount.

4.6. Performance analysis of deep learning models using different IoT-enabled datasets

This subsection comprehensively explains the deep learning model using standard metrics with error rate analysis.

4.6.1. Performance analysis using ECU_IoHT dataset (Dataset-1)

The comparative analysis of the EmbedNet, ConvNet-SVM, and DeepSVM-Net models on the ECU-IoHT dataset highlights the performance of each model across different attack classes. DeepSVM-Net demonstrates superior accuracy (ACC), positive predictive value (PPV), true positive rate (TPR), and F1-score across all classes, achieving an average accuracy exceeding 99.8 % and an almost perfect TPR of 1.000 for Class 4 (DoS Attack). ConvNet-SVM follows closely, maintaining high predictive performance but slightly lower than DeepSVM-Net. In contrast, EmbedNet, while still effective, shows relatively lower accuracy and prediction quality, particularly for Class 2 (ARP Spoofing) and Class 3 (Nmap PortScan). The Receiver Operating Characteristic - Area Under Curve (ROC_AUC) scores further confirm DeepSVM-Net's superiority, consistently reaching values above 0.998 across all classes. However, a key trade-off emerges in prediction time (P.T.), where DeepSVM-Net exhibits the highest computational overhead, particularly for Classes 3 and 4, with P.T. exceeding 4.7 s. ConvNet-SVM also incurs significant latency, peaking at 3.89 s for Class 4. Conversely, EmbedNet achieves the lowest prediction times, particularly excelling in Class 0 (Benign) with a minimal latency of 0.2038 s. This highlights a practical concern: while DeepSVM-Net offers the best predictive performance, its computational demand may hinder real-time applications, whereas EmbedNet provides a faster but slightly less accurate alternative. The findings suggest that selecting an intrusion detection model depends on the balance between prediction accuracy and processing efficiency, with DeepSVM-Net being optimal for high-security scenarios and EmbedNet being preferable for real-time applications requiring low latency. Table 16 presented the Qualitative analysis of Proposed DL models on ECU-IoHT (D1).

Among the models, DeepSVM-Net consistently outperforms the other models, achieving the highest Matthews Correlation Coefficient (MCC) and Markedness (MK) across all attack classes, signifying strong predictive capability. The Negative Predictive Value (NPV) is nearly perfect for DeepSVM-Net, reaching 1.000 for Class 4 (DoS Attack), indicating an exceptionally low probability of false negatives. Additionally, False

Table 12
Performance comparison of ML models for 10-fold cross-validation.

Model	Dataset		0	1	2	3	4	5	6	7	8	9	Avg. Score
LR	**ECU_IoHT**	AC	0.9785	0.9790	0.9750	0.9765	0.9775	0.9780	0.9770	0.9760	0.9795	0.9760	0.9778
		PPV	0.9800	0.9810	0.9770	0.9780	0.9790	0.9800	0.9790	0.9780	0.9810	0.9780	0.9791
		TPR	0.9770	0.9780	0.9730	0.9750	0.9760	0.9765	0.9750	0.9740	0.9785	0.9740	0.9757
		F1	0.9785	0.9795	0.9750	0.9765	0.9775	0.9780	0.9770	0.9760	0.9795	0.9760	0.9778
	NF-BoT-IoT	AC	0.9395	0.9391	0.9394	0.9396	0.9395	0.9388	0.9394	0.9396	0.9394	0.9394	0.9394
		PPV	0.9204	0.9241	0.9235	0.9211	0.9222	0.9253	0.9220	0.9217	0.9227	0.9217	0.9225
		TPR	0.9610	0.9596	0.9602	0.9596	0.9600	0.9618	0.9603	0.9598	0.9587	0.9606	0.9602
		F1	0.9403	0.9415	0.9415	0.9400	0.9407	0.9432	0.9407	0.9404	0.9404	0.9408	0.9409
	WUSTL	AC	0.9996	0.9997	0.9997	0.9997	0.9997	0.9997	0.9997	0.9997	0.9997	0.9997	0.9997
		PPV	0.9997	0.9998	0.9998	0.9998	0.9998	0.9998	0.9998	0.9998	0.9998	0.9998	0.9998
		TPR	0.9995	0.9996	0.9996	0.9996	0.9996	0.9996	0.9996	0.9996	0.9996	0.9996	0.9996
		F1	0.9996	0.9997	0.9997	0.9997	0.9997	0.9997	0.9997	0.9997	0.9997	0.9997	0.9997
XGB	**ECU_IoHT**	AC	0.9979	0.9981	0.9979	0.9977	0.9979	0.9980	0.9981	0.9978	0.9981	0.9979	0.9979
		PPV	0.9962	0.9966	0.9955	0.9968	0.9961	0.9964	0.9951	0.9963	0.9963	0.9958	0.9961
		TPR	0.9919	0.9908	0.9908	0.9917	0.9914	0.9904	0.9902	0.9929	0.9907	0.9920	0.9913
		F1	0.9941	0.9937	0.9931	0.9942	0.9937	0.9934	0.9926	0.9946	0.9935	0.9939	0.9937
	NF-BoT-IoT	AC	0.9999	0.9999	0.9999	0.9998	0.9998	0.9998	0.9999	0.9998	0.9999	0.9998	0.9999
		PPV	0.9998	0.9997	0.9997	0.9997	0.9997	0.9997	0.9997	0.9998	0.9997	0.9998	0.9998
		TPR	0.9995	0.9994	0.9995	0.9996	0.9997	0.9997	0.9996	0.9998	0.9996	0.9996	0.9996
		F1	0.9997	0.9996	0.9996	0.9997	0.9997	0.9997	0.9997	0.9998	0.9997	0.9997	0.9997
	WUSTL	AC	0.9998	0.9998	0.9998	0.9998	0.9998	0.9998	0.9998	0.9998	0.9998	0.9998	0.9998
		PPV	0.9998	0.9999	0.9999	0.9999	0.9999	0.9999	0.9999	0.9999	0.9999	0.9999	0.9999
		TPR	0.9997	0.9998	0.9998	0.9998	0.9998	0.9998	0.9998	0.9998	0.9998	0.9998	0.9998
		F1	0.9998	0.9998	0.9998	0.9998	0.9998	0.9998	0.9998	0.9998	0.9998	0.9998	0.9998
DT	**ECU_IoHT**	AC	0.9939	0.9942	0.9940	0.9939	0.9938	0.9942	0.9942	0.9937	0.9943	0.9940	0.9940
		PPV	0.9996	0.9997	0.9994	0.9991	0.9989	0.9990	0.9994	0.9992	0.9997	0.9995	0.9993
		TPR	0.9891	0.9873	0.9878	0.9883	0.9877	0.9871	0.9868	0.9895	0.9875	0.9887	0.9880
		F1	0.9943	0.9935	0.9936	0.9937	0.9932	0.9930	0.9930	0.9943	0.9936	0.9941	0.9936
	NF-BoT-IoT	AC	0.9962	0.9960	0.9960	0.9961	0.9961	0.9960	0.9962	0.9968	0.9961	0.9967	0.9962
		PPV	0.9929	0.9937	0.9940	0.9934	0.9937	0.9936	0.9929	0.9946	0.9932	0.9951	0.9937
		TPR	0.9985	0.9988	0.9989	0.9981	0.9986	0.9989	0.9987	0.9988	0.9984	0.9985	0.9986
		F1	0.9957	0.9962	0.9965	0.9957	0.9962	0.9963	0.9958	0.9967	0.9958	0.9968	0.9962
	WUSTL	AC	0.9971	0.9972	0.9973	0.9972	0.9972	0.9973	0.9971	0.9973	0.9973	0.9972	0.9972
		PPV	0.9973	0.9974	0.9975	0.9974	0.9974	0.9975	0.9973	0.9975	0.9975	0.9974	0.9974
		TPR	0.9969	0.9970	0.9971	0.9970	0.9970	0.9971	0.9969	0.9971	0.9971	0.9970	0.9970
		F1	0.9971	0.9972	0.9973	0.9972	0.9972	0.9973	0.9971	0.9973	0.9973	0.9972	0.9972
GBT	**ECU_IoHT**	AC	0.9971	0.9975	0.9937	0.9976	0.9971	0.9972	0.9970	0.9952	0.9973	0.9976	0.9967
		PPV	0.9942	0.9942	0.9874	0.9946	0.9933	0.9937	0.9929	0.9899	0.9950	0.9947	0.9930
		TPR	0.9916	0.9910	0.9905	0.9914	0.9909	0.9904	0.9902	0.9926	0.9908	0.9920	0.9911
		F1	0.9929	0.9926	0.9890	0.9930	0.9921	0.9920	0.9916	0.9912	0.9929	0.9934	0.9921
	NF-BoT-IoT	AC	0.9998	0.9999	0.9998	0.9998	0.9998	0.9998	0.9998	0.9998	0.9999	0.9998	0.9998
		PPV	0.9998	0.9996	0.9997	0.9998	0.9996	0.9998	0.9998	0.9999	0.9998	0.9998	0.9998
		TPR	0.9996	0.9993	0.9993	0.9998	0.9996	0.9995	0.9995	0.9996	0.9996	0.9996	0.9995
		F1	0.9997	0.9995	0.9995	0.9998	0.9996	0.9997	0.9996	0.9997	0.9997	0.9997	0.9997
	WUSTL	AC	0.9980	0.9980	0.9980	0.9980	0.9980	0.9981	0.9980	0.9980	0.9981	0.9980	0.9980
		PPV	0.9981	0.9982	0.9982	0.9982	0.9982	0.9983	0.9982	0.9982	0.9983	0.9982	0.9982
		TPR	0.9979	0.9980	0.9980	0.9980	0.9980	0.9981	0.9980	0.9980	0.9981	0.9980	0.9980
		F1	0.9980	0.9981	0.9981	0.9981	0.9981	0.9982	0.9981	0.9981	0.9982	0.9981	0.9981
KNN	**ECU_IoHT**	AC	0.9952	0.9952	0.9953	0.9951	0.9952	0.9952	0.9956	0.9950	0.9953	0.9950	0.9952
		PPV	0.9978	0.9990	0.9985	0.9982	0.9980	0.9979	0.9971	0.9982	0.9983	0.9973	0.9980
		TPR	0.9882	0.9869	0.9877	0.9877	09881	0.9869	0.9864	0.9898	0.9875	0.9889	0.9878
		F1	0.9930	0.9929	0.9931	0.9929	0.9930	0.9923	0.9917	0.9940	0.9929	0.9931	0.9929
	NF-BoT-IoT	AC	0.9996	0.9996	0.9995	0.9995	0.9995	0.9995	0.9996	0.9995	0.9996	0.9996	0.9996
		PPV	0.9999	0.9999	0.9998	0.9999	0.9998	0.9998	0.9999	0.9999	0.9997	0.9998	0.9999
		TPR	0.9984	0.9981	0.9985	0.9985	0.9987	0.9986	0.9983	0.9989	0.9985	0.9986	0.9985
		F1	0.9991	0.9990	0.9992	0.9992	0.9992	0.9992	0.9991	0.9994	0.9991	0.9992	0.9992
	WUSTL	AC	0.9997	0.9997	0.9997	0.9997	0.9998	0.9998	0.9998	0.9998	0.9999	0.9998	0.9998
		PPV	0.9971	0.9954	0.9969	0.9947	0.9951	0.9951	0.9965	0.9972	0.9971	0.9920	0.9957
		TPR	0.9992	0.9982	0.9982	0.9993	0.9985	0.9989	0.9979	0.9955	0.9982	0.9972	0.9981
		F1	0.9982	0.9968	0.9976	0.9970	0.9968	0.9970	0.9972	0.9963	0.9976	0.9946	0.9969
SVM	**ECU_IoHT**	AC	0.9929	0.9931	0.9930	0.9927	0.9929	0.9932	0.9932	0.9926	0.9931	0.9928	0.9929
		PPV	0.9999	0.9998	0.9997	0.9998	0.9998	1.0	0.9999	0.9998	0.9997	0.9998	0.9998
		TPR	0.9869	0.9851	0.9859	0.9856	0.9860	0.9849	0.9842	0.9881	0.9885	0.9870	0.9859
		F1	0.9934	0.9924	0.9928	0.9929	0.9924	0.9920	0.9920	0.9939	0.9927	0.9934	0.9928
	NF-BoT-IoT	AC	0.9974	0.9974	0.9974	0.9974	0.9975	0.9974	0.9974	0.9974	0.9976	0.9974	0.9975
		PPV	0.9956	0.9960	0.9960	0.9958	0.9953	0.9957	0.9957	0.9956	0.9952	0.9956	0.9956
		TPR	0.9995	0.9993	0.9995	0.9993	0.9995	0.9995	0.9996	0.9995	0.9992	0.9996	0.9995
		F1	0.9976	0.9977	0.9978	0.9976	0.9974	0.9976	0.9976	0.9975	0.9972	0.9976	0.9976
	WUSTL	AC	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0
		PPV	1.0	0.9989	1.0	1.0	1.0	0.9989	1.0	0.9996	1.0	0.9993	0.9996
		TPR	0.9996	1.0	0.9996	0.9996	1.0	1.0	1.0	1.0	0.9989	1.0	0.9997
		F1	0.9998	0.9994	0.9998	0.9998	1.0	0.9994	1.0	0.9998	0.9994	0.9996	0.9997
RF	**ECU_IoHT**	AC	0.9978	0.9979	0.9978	0.9976	0.9977	0.9977	0.9978	0.9975	0.9979	0.9976	0.9977
		PPV	0.9944	0.9953	0.9948	0.9953	0.9952	0.9953	0.9942	0.9945	0.9951	0.9949	0.9949

(*continued on next page*)

Table 12 (*continued*)

Model	Dataset		0	1	2	3	4	5	6	7	8	9	Avg. Score
		TPR	0.9923	0.9905	0.9915	0.9919	0.9912	0.9905	0.9900	0.9926	0.9910	0.9921	0.9914
		F1	0.9933	0.9929	0.9932	0.9936	0.9932	0.9929	0.9921	0.9935	0.9930	0.9935	0.9931
	NF-BoT-IoT	AC	0.9998	0.9999	0.9998	0.9998	0.9998	0.9998	0.9999	0.9998	0.9999	0.9998	0.9998
		PPV	0.9998	0.9997	0.9998	0.9997	0.9996	0.9997	0.9997	0.9999	0.9997	0.9998	0.9998
		TPR	0.9995	0.9993	0.9995	0.9996	0.9995	0.9995	0.9994	0.9995	0.9997	0.9995	0.9995
		F1	0.9996	0.9995	0.9996	0.9997	0.9996	0.9996	0.9996	0.9997	0.9997	0.9997	0.9996
	WUSTL	AC	0.9996	0.9996	0.9996	0.9996	0.9996	0.9996	0.9996	0.9996	0.9996	0.9996	0.9996
		PPV	0.9997	0.9997	0.9997	0.9997	0.9997	0.9997	0.9997	0.9998	0.9997	0.9997	0.9997
		TPR	0.9995	0.9995	0.9995	0.9995	0.9995	0.9995	0.9995	0.9995	0.9995	0.9995	0.9995
		F1	0.9996	0.9995	0.9996	0.9997	0.9996	0.9996	0.9996	0.9997	0.9997	0.9997	0.9996

Discovery Rate (FDR), False Positive Rate (FPR), and False Omission Rate (FOR) are significantly lower for DeepSVM-Net compared to EmbedNet and ConvNet-SVM, ensuring highly reliable threat identification. In contrast, EmbedNet demonstrates slightly weaker MCC and MK values, particularly in Class 2 (ARP Spoofing) and Class 3 (Nmap PortScan), suggesting more frequent misclassifications in these attack categories. The Likelihood Ratios (LR+ and LR-) further emphasize DeepSVM-Net's superiority in intrusion detection. Positive likelihood ratios (LR+) are significantly higher for DeepSVM-Net, exceeding 489.235 for Class 4, meaning a higher probability of correctly identifying attacks. Conversely, the negative likelihood ratio (LR-) is nearly zero for DeepSVM-Net, confirming its robustness in minimizing missed detections. While ConvNet-SVM also performs well, particularly in Class 1 (Smurf Attack) and Class 4 (DoS Attack), it has slightly higher FDR and FPR than DeepSVM-Net. Finally, EmbedNet, though computationally efficient, lags in BM (Informedness) and ROC_AUC scores, making it a less reliable choice for high-stakes cybersecurity applications. These results indicate that while DeepSVM-Net is the most precise and informed classifier, its computational complexity may make ConvNet-SVM a viable alternative where speed is prioritized. Table 17 presented the Quantitative analysis of Proposed DL models on ECU-IoHT (D1).

4.6.2. Performance analysis using NF-BoT-IoT dataset (Dataset-2)

The comparative analysis of EmbedNet, ConvNet-SVM, and DeepSVM-Net on the NF-BoT-IoT dataset reveals distinct differences in their classification performance for Benign, Theft, Reconnaissance, DoS, and DDoS attacks. Among the three models, ConvNet-SVM achieves the highest accuracy (ACC) across all classes, reaching 0.9975 for Class 1 (Theft) and 0.9968 for Class 3 (DoS), indicating its strong generalization ability. The precision (PPV) and recall (TPR-R) values are also highest for ConvNet-SVM, suggesting a better balance between correctly identifying attacks and minimizing false positives. DeepSVM-Net, while slightly lower in accuracy, maintains consistent performance across all attack classes, with high True Negative Rate (TNR), ensuring reliable attack detection. EmbedNet, though effective, lags behind in reconnaissance detection (Class 2), where its TPR-R is 0.9879, indicating a higher rate of misclassification compared to the other models. The ROC_AUC scores confirm the robustness of ConvNet-SVM, achieving 0.9973 for Class 1 and 0.9967 for Class 3, making it the most reliable classifier for IoT-based network security. Interestingly, DeepSVM-Net exhibits the lowest Prediction Time (P.T.), with only 0.5894 for Class 2 and 0.7195 for Class 4, making it the fastest model for real-time threat detection. In contrast, EmbedNet and ConvNet-SVM have significantly higher P.T. values, particularly in Class 1 and Class 4, indicating higher computational costs. The overall results suggest that ConvNet-SVM is the most accurate and balanced model, but DeepSVM-Net is preferable for time-sensitive applications. Thus, depending on the deployment scenario, a trade-off between accuracy and speed needs to be considered for optimal IoT security. Table 18 presented the Qualitative analysis of Proposed DL models on NF-BoT-IoT (D2).

Among the models, ConvNet-SVM achieves the highest Matthews Correlation Coefficient (MCC) across all classes, indicating superior prediction reliability, with values reaching 0.9939 for Class 1 (Theft) and 0.9931 for Class 3 (DoS). Additionally, ConvNet-SVM demonstrates the lowest False Discovery Rate (FDR) and False Omission Rate (FOR), ensuring minimal misclassifications. DeepSVM-Net exhibits slightly lower MCC values but remains competitive, with balanced performance across all attack categories. Notably, EmbedNet shows a decline in MCC and Negative Predictive Value (NPV) for Class 2 (Reconnaissance), highlighting its relative weakness in detecting reconnaissance threats compared to the other models. ConvNet-SVM also achieves the highest Positive Likelihood Ratio (LR+), reaching 1268.44 for Class 1 and 1002.76 for Class 3, reinforcing its high confidence in positive classifications. In contrast, DeepSVM-Net records the lowest LR+ , particularly in Class 2 (198.74), indicating weaker discriminative power for reconnaissance detection. However, DeepSVM-Net compensates with consistently low False Positive Rate (FPR) and False Negative Rate (FNR), ensuring stable detection performance. The Bookmaker Informedness (BM) and Markedness (MK) scores further confirm ConvNet-SVM's dominance, particularly in Classes 1 and 3, where it achieves values exceeding 0.9937, indicating strong classifier reliability. While EmbedNet maintains reasonable accuracy, its lower BM and MK values suggest comparatively weaker predictive power. Overall, ConvNet-SVM emerges as the most effective model for IoT attack detection, while DeepSVM-Net offers a computationally efficient alternative with slightly reduced accuracy. Table 19 presented the Quantitative analysis of Proposed DL models on NF-BoT-IoT (D2).

4.6.3. Performance analysis using WUSTL-HDRL-2024 dataset (Dataset-3)

In this sub-section, the comparative analysis of EmbedNet, ConvNet-SVM, and DeepSVM-Net for detecting different types of cyberattacks in IoT networks reveals that ConvNet-SVM outperforms the other models across all performance metrics. It achieves the highest accuracy (ACC), precision (PPV), recall (TPR-R), F1-score, and True Negative Rate (TNR), consistently exceeding 0.999 for benign and buffer overflow classes, demonstrating its superior classification capability. EmbedNet also performs well, with slightly lower but competitive values across all classes, particularly for Man-in-the-Middle (MiTM) and DDoS attacks. In contrast, DeepSVM-Net records the lowest classification performance, especially for MiTM (ACC = 0.9957) and DDoS (ACC = 0.9949), indicating its relative weakness in distinguishing these attacks. The Receiver Operating Characteristic - Area Under Curve (ROC_AUC) values follow a similar trend, reinforcing ConvNet-SVM's robustness in attack detection. An analysis of prediction time (P.T.) highlights the trade-off between accuracy and computational efficiency. ConvNet-SVM exhibits significantly higher prediction times compared to EmbedNet and DeepSVM-Net, with values reaching 0.4901 s for benign traffic and 0.4753 s for buffer overflow attacks. This suggests that while ConvNet-SVM achieves the best detection performance, it requires more computational resources, which could impact real-time applications. On the other hand, EmbedNet and DeepSVM-Net maintain lower prediction times, particularly for buffer overflow (0.0251 and 0.0412, respectively), making them more suitable for latency-sensitive environments. Thus, ConvNet-SVM is the preferred choice when accuracy is the priority, whereas

Table 13
Performance comparison of ML models using ECU-IoHT (D1).

Metrics	LR		XGB		DT		GBT		KNN		SVM		RF	
	70–30	80–20	70–30	80–20	70–30	80–20	70–30	80–20	70–30	80–20	70–30	80–20	70–30	80–20
ACC	0.9755	0.9778	0.9980	0.9979	0.9939	0.9940	0.9947	0.9967	0.9953	0.9952	0.9928	0.9929	0.9977	0.9977
PPV-P	0.9785	0.9791	0.9957	0.9961	0.9992	0.9993	0.9894	0.9930	0.9979	0.9980	0.9999	0.9998	0.9947	0.9949
TPR-R	0.9735	0.9757	0.9914	0.9913	0.9878	0.9880	0.9912	0.9911	0.9880	0.9878	0.9860	0.9859	0.9912	0.9914
F1	0.9760	0.9778	0.9935	0.9937	0.9935	0.9936	0.9903	0.9921	0.9929	0.9929	0.9929	0.9928	0.9930	0.9931
TNR-S	0.9822	0.9788	0.9957	0.9961	0.9992	0.9993	0.9893	0.9930	0.9979	0.9981	0.9999	0.9998	0.9947	0.9949
MCC	0.9505	0.9543	0.9871	0.9875	0.9871	0.9874	0.9806	0.9842	0.9859	0.9860	0.9860	0.9859	0.9860	0.9863
NPV	0.9683	0.9783	0.9914	0.9913	0.9879	0.9881	0.9912	0.9911	0.9880	0.9879	0.9861	0.9861	0.9912	0.9914
ROC_AUC	0.9803	0.9778	0.9935	0.9937	0.9935	0.9937	0.9903	0.9921	0.9929	0.9929	0.9929	0.9929	0.9930	0.9931
FDR	0.0213	0.0211	0.0043	0.0039	0.0008	0.0007	0.0106	0.0070	0.0021	0.0020	0.0001	0.0002	0.0053	0.0051
FPR	0.0178	0.0217	0.0042	0.0038	0.0007	0.0006	0.0106	0.0069	0.0020	0.0018	0.0001	0.0001	0.0052	0.0050
FNR	0.0263	0.0241	0.0085	0.0086	0.0121	0.0119	0.0087	0.0088	0.0119	0.0121	0.0139	0.0140	0.0087	0.0085
FOR	0.0318	0.0232	0.0086	0.0087	0.0121	0.0119	0.0088	0.0089	0.0209	0.0121	0.0139	0.0139	0.0088	0.0086
LR+	54.792	50.24	236.047	260.86	1411.14	1646.67	93.50	143.63	494	548.77	9860	9859	190.61	198.28
LR-	0.0266	0.0240	0.0085	0.0086	0.0121	0.0119	0.0087	0.0088	0.0119	0.0121	0.0139	0.0140	0.0087	0.0085
MK	0.9468	0.9566	0.9871	0.9874	0.9871	0.9874	0.9806	0.9841	0.9859	0.9859	0.9860	0.9859	0.9859	0.9863
BM	0.9562	0.9552	0.9871	0.9874	0.9870	0.9873	0.9805	0.9841	0.9859	0.9859	0.9859	0.9857	0.9859	0.9863
P.T. (Sec.)	0.0012	0.0017	0.7778	0.7553	0.0017	0.0031	0.8998	1.6364	3.0905	4.3795	7.6569	9.7627	3.8599	3.4396

Table 14
Performance comparison of ML models on NF-BoT-IoT (D2).

Metrics	LR		XGB		DT		GBT		KNN		SVM		RF	
	70–30	80–20	70–30	80–20	70–30	80–20	70–30	80–20	70–30	80–20	70–30	80–20	70–30	80–20
ACC	0.9393	0.9394	0.9999	0.9999	0.9964	0.9962	0.9998	0.9998	0.9995	0.9996	0.9975	0.9975	0.9998	0.9998
PPV-P	0.9226	0.9225	0.9998	0.9998	0.9940	0.9937	0.9998	0.9998	0.9999	0.9999	0.9956	0.9956	0.9998	0.9998
TPR-R	0.9606	0.9602	0.9996	0.9996	0.9985	0.9986	0.9995	0.9995	0.9986	0.9985	0.9994	0.9995	0.9996	0.9995
F1	0.9413	0.9409	0.9997	0.9997	0.9963	0.9962	0.9997	0.9997	0.9992	0.9992	0.9975	0.9976	0.9997	0.9996
TNR-S	0.9195	0.9193	0.9998	0.9998	0.9940	0.9937	0.9998	0.9998	0.9999	0.9999	0.9956	0.9956	0.9998	0.9998
MCC	0.8809	0.8803	0.9995	0.9994	0.9925	0.9924	0.9994	0.9994	0.9985	0.9984	0.9951	0.9952	0.9994	0.9993
NPV	0.9590	0.9585	0.9996	0.9996	0.9985	0.9986	0.9995	0.9995	0.9986	0.9985	0.9994	0.9995	0.9996	0.9995
ROC_AUC	0.9401	0.9397	0.9997	0.9997	0.9962	0.9962	0.9997	0.9997	0.9992	0.9992	0.9975	0.9976	0.9997	0.9996
FDR	0.0774	0.0775	0.0002	0.0002	0.0060	0.0063	0.0002	0.0002	0.0001	0.0001	0.0044	0.0044	0.0002	0.0002
FPR	0.0804	0.0806	0.0001	0.0001	0.0059	0.0062	0.0001	0.0001	0.0001	0.0001	0.0043	0.0043	0.0001	0.0001
FNR	0.0393	0.0397	0.0003	0.0003	0.0014	0.0013	0.0004	0.0004	0.0013	0.0014	0.0005	0.0004	0.0003	0.0004
FOR	0.0410	0.0415	0.0004	0.0004	0.0015	0.0014	0.0005	0.0005	0.0014	0.0015	0.0006	0.0005	0.0004	0.0005
LR+	11.947	11.913	9996	9996	169.23	161.06	9995	9995	9986	9985	232.41	232.44	9996	1999
LR-	0.0427	0.0431	0.0003	0.0003	0.0014	0.0013	0.0004	0.0004	0.0013	0.0014	0.0005	0.0004	0.0003	0.0004
MK	0.8816	0.8810	0.9994	0.9994	0.9925	0.9923	0.9993	0.9993	0.9985	0.9984	0.9950	0.9951	0.9994	0.9993
BM	0.8801	0.8795	0.9994	0.9994	0.9925	0.9923	0.9993	0.9993	0.9985	0.9984	0.9950	0.9951	0.9994	0.9993
P.T. (Sec.)	0.0035	0.0009	1.2453	0.9515	0.0032	0.0103	2.2155	2.5028	37.436	30.972	23.888	30.972	6.6051	8.8624

Table 15
Performance comparison of ML models on WUSTL-HDRL-2024 (D3).

Metrics	LR		XGB		DT		GBT		KNN		SVM		RF	
	70–30	80–20	70–30	80–20	70–30	80–20	70–30	80–20	70–30	80–20	70–30	80–20	70–30	80–20
ACC	0.9994	0.9997	0.9991	0.9998	0.9985	0.9972	0.9977	0.9980	0.9998	0.9998	1.0	1.0	0.9994	0.9996
PPV-P	0.9949	0.9998	0.9950	0.9999	0.9950	0.9974	0.9850	0.9982	0.9958	0.9957	0.9995	0.9996	0.9950	0.9997
TPR-R	0.9949	0.9996	0.9950	0.9998	0.9900	0.9970	0.9850	0.9980	0.9987	0.9981	0.9999	0.9997	0.9990	0.9995
F1	0.9949	0.9997	0.9950	0.9998	0.9920	0.9972	0.9850	0.9981	0.9973	0.9969	0.9997	0.9997	0.9970	0.9996
TNR-S	0.9995	0.9999	0.9995	0.9999	0.9995	0.9976	0.9985	0.9982	0.9958	0.9957	0.9995	0.9996	0.9995	0.9998
MCC	0.9944	0.9994	0.9930	0.9997	0.9902	0.9947	0.9838	0.9961	0.9946	0.9938	0.9995	0.9994	0.9970	0.9992
NPV	0.9995	0.9998	0.9995	0.9999	0.9990	0.9973	0.9985	0.9981	0.9987	0.9981	0.9999	0.9997	0.9999	0.9997
ROC_AUC	0.9995	0.9998	0.9975	0.9999	0.9990	0.9973	0.9968	0.9981	0.9973	0.9969	0.9997	0.9997	0.9993	0.9996
FDR	0.0051	0.0002	0.005	0.0001	0.005	0.0026	0.015	0.0018	0.0042	0.0043	0.0005	0.0004	0.005	0.0003
FPR	0.0005	0.0001	0.0005	0.0001	0.0005	0.0024	0.0015	0.0018	0.0041	0.0042	0.0004	0.0003	0.0005	0.0002
FNR	0.0051	0.0004	0.005	0.0002	0.0099	0.0030	0.015	0.0020	0.0012	0.0018	0.0001	0.0002	0.001	0.0005
FOR	0.0005	0.0002	0.0005	0.0001	0.001	0.0027	0.0015	0.0019	0.0054	0.0062	0.0005	0.0006	0.0001	0.0003
LR+	1989.8	9998.5	1990	9997.9	1980	403.13	656.67	566.67	243.58	237.64	2499.75	3332.33	1998	4997.5
LR-	0.0051	0.0004	0.005	0.0002	0.0099	0.0029	0.015	0.0020	0.0012	0.0018	0.0001	0.0002	0.001	0.0005
MK	0.9944	0.9996	0.9945	0.9998	0.9940	0.9947	0.9835	0.9963	0.9945	0.9938	0.9994	0.9993	0.9949	0.9992
BM	0.9944	0.9995	0.9945	0.9997	0.9895	0.9946	0.9835	0.9963	0.9945	0.9938	0.9994	0.9993	0.9985	0.9993
P.T.	0.0010	0.0009	0.0104	0.0215	0.0011	0.0009	0.0191	0.0174	1.8901	1.7376	0.3775	0.3819	0.6451	0.6317

Table 16
Qualitative analysis of proposed DL models on ECU-IoHT (D1).

Metrics	EmbedNet					ConvNet-SVM					DeepSVM-Net				
	Classes					Classes					Classes				
	0	1	2	3	4	0	1	2	3	4	0	1	2	3	4
	Benign	Smurf Attack	ARP Spoofing	Nmap PortScan	DoS	Benign	Smurf Attack	ARP Spoofing	Nmap PortScan	DoS	Benign	Smurf Attack	ARP Spoofing	Nmap PortScan	DoS
ACC	0.9857	0.9921	0.9873	0.9914	0.9932	0.9927	0.9956	0.9931	0.9947	0.9960	0.9989	0.9992	0.9985	0.9990	0.9994
PPV	0.9820	0.9908	0.9842	0.9889	0.9915	0.9857	0.9941	0.9869	0.9930	0.9953	0.9858	0.9962	0.9874	0.9971	0.9978
TPR-R	0.9998	0.9994	0.9991	0.9992	0.9995	0.9997	0.9996	0.9993	0.9997	0.9998	0.9999	0.9999	0.9998	0.9999	1.0000
F1	0.9908	0.9922	0.9911	0.9920	0.9933	0.9926	0.9953	0.9934	0.9958	0.9965	0.9928	0.9972	0.9940	0.9981	0.9985
TNR	0.9873	0.9911	0.9882	0.9905	0.9924	0.9858	0.9932	0.9879	0.9940	0.9950	0.9859	0.9961	0.9883	0.9970	0.9976
ROC_AUC	0.9985	0.9925	0.9916	0.9923	0.9935	0.9928	0.9955	0.9935	0.9959	0.9966	0.9929	0.9974	0.9941	0.9982	0.9986
P.T.	0.2038	1.5829	0.8114	1.4028	1.9703	1.8709	2.9941	1.5624	3.1784	3.8921	0.1218	4.0589	1.8329	4.7123	5.2234

Table 17
Quantitative analysis of proposed DL models on ECU-IoHT (D1).

Metrics	EmbedNet					ConvNet-SVM					DeepSVM-Net				
	Classes					Classes					Classes				
	0	1	2	3	4	0	1	2	3	4	0	1	2	3	4
	Benign	Smurf Attack	ARP Spoofing	Nmap PortScan	DoS	Benign	Smurf Attack	ARP Spoofing	Nmap PortScan	DoS	Benign	Smurf Attack	ARP Spoofing	Nmap PortScan	DoS
MCC	0.9990	0.9917	0.9893	0.9908	0.9927	0.9855	0.9840	0.9878	0.9949	0.9957	0.9858	0.9965	0.9886	0.9973	0.9979
NPV	0.9993	0.9991	0.9987	0.9990	0.9992	0.9997	0.9994	0.9991	0.9996	0.9998	0.9999	0.9999	0.9998	0.9999	1.0000
FDR	0.0180	0.0092	0.0158	0.0111	0.0085	0.0143	0.0059	0.0131	0.0070	0.0047	0.0142	0.0038	0.0126	0.0029	0.0022
FPR	0.0626	0.0091	0.0152	0.0109	0.0083	0.0141	0.0057	0.0128	0.0068	0.0045	0.0140	0.0037	0.0123	0.0028	0.0021
FNR	0.0001	0.0006	0.0009	0.0008	0.0005	0.0002	0.0004	0.0007	0.0003	0.0002	0.0001	0.0001	0.0002	0.0001	0.0001
FOR	0.0007	0.0009	0.0013	0.0010	0.0008	0.0003	0.0006	0.0009	0.0004	0.0002	0.0001	0.0001	0.0002	0.0001	0.0000
LR+	15.9712	108.273	75.8021	99.3421	123.412	70.9007	205.483	92.3942	215.203	301.112	71.42	312.489	98.5012	354.992	489.235
LR-	0.0001	0.0005	0.0008	0.0007	0.0004	0.0002	0.0003	0.0006	0.0002	0.0001	0.0001	0.0001	0.0002	0.0001	0.0000
MK	0.9817	0.9895	0.9863	0.9888	0.9914	0.9854	0.9931	0.9877	0.9948	0.9956	0.9857	0.9964	0.9885	0.9972	0.9978
BM	0.9871	0.9910	0.9881	0.9903	0.9922	0.9855	0.9930	0.9876	0.9947	0.9955	0.9858	0.9963	0.9884	0.9971	0.9977

EmbedNet and DeepSVM-Net may be viable alternatives for real-time intrusion detection systems where rapid decision-making is crucial. Table 20 presented the Qualitative analysis of Proposed DL models on WUSTL-HDRL-2024 (D3).

The comparative evaluation of EmbedNet, ConvNet-SVM, and DeepSVM-Net highlights the superior performance of ConvNet-SVM in detecting cyberattacks across multiple metrics. Matthews Correlation Coefficient (MCC), Markedness (MK), and Bookmaker Informedness (BM) for ConvNet-SVM consistently surpass those of EmbedNet and DeepSVM-Net, exceeding 0.997 for most attack classes, demonstrating its reliability in classification. Moreover, ConvNet-SVM achieves the lowest False Discovery Rate (FDR), False Positive Rate (FPR), and False Omission Rate (FOR), with values below 0.0026 across all classes, confirming its robustness in reducing misclassifications. On the other hand, DeepSVM-Net shows the weakest performance, with MCC and BM values dropping to approximately 0.987 for benign traffic and 0.991 for DDoS attacks, along with higher FDR and FOR, indicating a comparatively higher rate of false alarms. Additionally, ConvNet-SVM exhibits the highest Likelihood Ratio Positive (LR+), exceeding 2000 for most classes, which signifies strong discriminatory power in distinguishing attacks from benign traffic. In contrast, DeepSVM-Net shows significantly lower LR+ values, particularly for MiTM and DDoS attacks, with values of 205.45 and 163.89, respectively, suggesting it may struggle with precise attack identification. EmbedNet, while slightly underperforming ConvNet-SVM, still maintains competitive results, especially in Negative Predictive Value (NPV) and False Negative Rate (FNR), where its values remain close to those of ConvNet-SVM. Overall, ConvNet-SVM emerges as the most effective model for intrusion detection, offering both high accuracy and minimal false predictions, making it an optimal choice for real-world cybersecurity applications in IoT networks. Table 21 presented the Quantitative analysis of Proposed DL models on WUSTL-HDRL-2024 (D3).

To minimize false positive rates during training, we implemented several strategies within our deep learning models, including EmbedNet, ConvNet-SVM, and DeepSVM-Net, across different datasets (ECU-IoHT, NF-BoT-IoT, and WUSTL-HDRL-2024). First, we employed an optimized data preprocessing pipeline that included label encoding, data normalization, and balancing techniques to reduce bias in class distributions. Additionally, we leveraged an advanced feature selection mechanism to enhance the model's ability to differentiate between attack and benign classes accurately. The use of a hybrid training approach, incorporating both supervised and semi-supervised learning, improved generalization and robustness. Furthermore, we fine-tuned hyperparameters such as learning rate, batch size, and dropout rates to mitigate overfitting and ensure better detection performance. Our models also incorporated cross-validation techniques, enabling better parameter optimization and reducing the likelihood of false alarms. The results, as reflected in our performance metrics, demonstrate significant improvements in precision (PPV), specificity (TNR), and ROC-AUC scores, indicating a strong ability to correctly classify normal traffic while minimizing false positives. Notably, DeepSVM-Net exhibited the lowest false positive rates (FPR) across multiple attack classes, with values as low as 0.0021, highlighting the effectiveness of our optimization strategies.

Table 22 presents the FPR and FNR values for EmbedNet, ConvNet-SVM, and DeepSVM-Net across representative attack classes for each dataset, highlighting the trade-offs achieved using proposed models.

The analysis of FPR and FNR for EmbedNet, ConvNet-SVM, and DeepSVM-Net across the ECU-IoHT, NF-BoT-IoT, and WUSTL-HDRL-2024 datasets reveals distinct trade-offs in their performance for IoT intrusion detection. DeepSVM-Net demonstrates exceptional performance on the ECU-IoHT dataset for DoS attacks (Class 4), achieving the lowest FPR (0.0021) and FNR (0.0001), making it ideal for high-security applications where minimizing both false alarms and missed detections is critical, despite its higher computational cost. However, its performance on NF-BoT-IoT for Reconnaissance (Class 2) shows a higher FNR (0.0079), indicating weaker detection of these attacks, though its low

Table 18
Qualitative analysis of proposed DL models on NF-BoT-IoT (D2).

Metrics	EmbedNet					ConvNet-SVM					DeepSVM-Net				
	Classes					Classes					Classes				
	0	1	2	3	4	0	1	2	3	4	0	1	2	3	4
	Benign	Theft	Reconnaissance	DoS	DDoS	Benign	Theft	Reconnaissance	DoS	DDoS	Benign	Theft	Reconnaissance	DoS	DDoS
ACC	0.9946	0.9962	0.9928	0.9953`	0.9949	0.9958	0.9975	0.9943	0.9968	0.9952	0.9945	0.9959	0.9931	0.9951	0.9944
PPV	0.9988	0.9991	0.9974	0.9983	0.9980	0.9986	0.9992	0.9968	0.9981	0.9976	0.9952	0.9969	0.9948	0.9963	0.9955
TPR-R	0.9905	0.9923	0.9879	0.9911	0.9907	0.9931	0.9954	0.9915	0.9942	0.9928	0.9937	0.9945	0.9909	0.9934	0.9918
F1	0.9946	0.9957	0.9925	0.9949	0.9943	0.9958	0.9972	0.9940	0.9965	0.9950	0.9944	0.9956	0.9929	0.9950	0.9941
TNR	0.9988	0.9990	0.9977	0.9985	0.9982	0.9986	0.9993	0.9969	0.9980	0.9975	0.9953	0.9968	0.9949	0.9962	0.9956
ROC_AUC	0.9947	0.9959	0.9929	0.9951	0.9945	0.9958	0.9973	0.9942	0.9967	0.9951	0.9945	0.9958	0.9930	0.9951	0.9942
P.T.	2.4027	2.7589	1.8974	2.5028	2.6135	2.5583	3.1125	2.1254	2.8746	2.9401	0.6721	1.0258	0.5894	0.8024	0.7195

Table 19
Quantitative analysis of proposed DL models on NF-BoT-IoT (D2).

Metrics	EmbedNet					ConvNet-SVM					DeepSVM-Net				
	Classes					Classes					Classes				
	0	1	2	3	4	0	1	2	3	4	0	1	2	3	4
	Benign	Theft	Reconnaissance	DoS	DDoS	Benign	Theft	Reconnaissance	DoS	DDoS	Benign	Theft	Reconnaissance	DoS	DDoS
MCC	0.9894	0.9912	0.9871	0.9905	0.9900	0.9917	0.9939	0.9902	0.9931	0.9920	0.9890	0.9906	0.9875	0.9901	0.9889
NPV	0.9905	0.9924	0.9881	0.9913	0.9909	0.9931	0.9955	0.9916	0.9943	0.9929	0.9937	0.9946	0.9910	0.9935	0.9919
FDR	0.0054	0.0038	0.0066	0.0049	0.0051	0.0014	0.0008	0.0032	0.0019	0.0024	0.0063	0.0055	0.0071	0.0050	0.0058
FPR	0.0011	0.0009	0.0024	0.0018	0.0021	0.0013	0.0007	0.0030	0.0015	0.0020	0.0046	0.0039	0.0052	0.0043	0.0048
FNR	0.0012	0.0025	0.0036	0.0029	0.0032	0.0068	0.0041	0.0074	0.0052	0.0060	0.0062	0.0058	0.0079	0.0057	0.0065
FOR	0.0095	0.0074	0.0102	0.0088	0.0091	0.0069	0.0043	0.0071	0.0056	0.0064	0.0063	0.0057	0.0082	0.0055	0.0066
LR+	900.45	1024.31	587.62	842.19	789.50	763.92	1268.44	512.87	1002.76	956.48	216.02	347.28	198.74	302.45	257.68
LR-	0.0012	0.0024	0.0037	0.0028	0.0031	0.0068	0.0042	0.0075	0.0053	0.0061	0.0062	0.0059	0.0081	0.0056	0.0064
MK	0.9893	0.9911	0.9872	0.9904	0.9899	0.9917	0.9938	0.9901	0.9930	0.9919	0.9889	0.9905	0.9876	0.9900	0.9888
BM	0.9893	0.9910	0.9873	0.9903	0.9898	0.9917	0.9937	0.9900	0.9929	0.9918	0.9890	0.9906	0.9877	0.9902	0.9889

FPR (0.0052) ensures fewer false alarms. Similarly, on WUSTL-HDRL-2024 for DDoS (Class 2), DeepSVM-Net's moderate FPR (0.0055) and FNR (0.0058) suggest it is less effective compared to other models. ConvNet-SVM excels in balancing FPR and FNR, particularly on NF-BoT-IoT for Theft (Class 1) with an extremely low FPR (0.0007) and a slightly higher FNR (0.0041), reflecting a trade-off prioritizing specificity, and on WUSTL-HDRL-2024 for Buffer Overflow (Class 4) with excellent FPR (0.0006) and FNR (0.0008), making it ideal for high-accuracy scenarios. On ECU-IoHT for DoS, ConvNet-SVM maintains a very low FNR (0.0002) and moderate FPR (0.0045), prioritizing attack detection. EmbedNet, while computationally efficient, shows higher error rates, such as an FPR of 0.0083 and FNR of 0.0005 for DoS on ECU-IoHT, indicating more false alarms despite strong attack detection. For NF-BoT-IoT's Reconnaissance (Class 2), EmbedNet's balanced FPR (0.0024) and FNR (0.0036) highlight weaker performance, and on WUSTL-HDRL-2024 for DDoS, its FPR (0.0029) and FNR (0.0032) suggest suitability for real-time applications where computational efficiency is prioritized. Overall, DeepSVM-Net is optimal for high-security contexts, ConvNet-SVM offers a balanced solution for accuracy and efficiency, and EmbedNet suits latency-sensitive environments despite slightly higher error rates.

4.7. Comparative analysis of the proposed approach performance against existing intrusion detection techniques

This section provides a performance comparison of the proposed method with the existing intrusion detection techniques, which are explained as follows: Karanfilovska et al. [43] presented a comprehensive IDS model using supervised and unsupervised ML on the NF-ToN-IoT-v2 dataset and achieved an accuracy of 98.8 % with methods like XGBoost Classifier and Random Forest Classifier. Nguyen et al. [44] introduced a collaborative ML model for early IoT Botnet detection and achieved 99.37 % accuracy on a real-time dataset comprising 5023 botnet and 3888 benign samples, thus enhancing response times and mitigating potential damage. Torre et al. [45] introduced a cloud-based distributed deep learning framework for detecting and mitigating phishing and Botnet attacks. It uses a Distributed CNN (DCNN) embedded in IoT devices for detecting phishing and application layer DDoS attacks and a cloud-based LSTM model for Botnet detection. The approach achieves 94.3 % accuracy for phishing detection and 94.8 % accuracy for Botnet detection, highlighting its effectiveness in distributed attack detection. Alkahtani and Aldhyani [46] proposed a hybrid CNN-LSTM deep learning model to detect botnet attacks, including BASHLITE and Mirai, on nine commercial IoT devices using the N-BaIoT dataset. The model achieved high detection accuracies, such as 90.88 % and 88.61 % for doorbells, 88.53 % for thermostats, and between 87.19 % and 89.64 % for security cameras, demonstrating its effectiveness in identifying botnet attacks across different IoT devices. Wagan et al. [47] introduced the Duo-Secure IoMT framework, which uses dynamic Fuzzy C-Means clustering and a customized Bi-LSTM technique to differentiate between attack patterns and routine IoMT data. Evaluated on a heart disease dataset with 36 attributes and 18,940 instances, the model achieved a 92.95 % accuracy in predicting heart issues and an 89.67 % accuracy in identifying network malware in a distributed IoMT environment. Gupta et al. [48] proposed deep hierarchical stacked neural networks to detect malicious activities altering data flow between IoT gateways, edge, and core clouds. Wang et al. [49] proposed an FMI-DNN model combination of DNN and federated learning (FL) for anomaly detection in IoT networks by using mutual information (MI). Unlike conventional models, this approach uses decentralized on-device data and shares only modified weights with a central FL server [50]. Aguru and Erukala [31] introduced a novel anomaly-based IDS using stacked modified Gated Recurrent Units (mGRU) for detecting multi-vector DDoS attacks in mobile healthcare systems. The suggested mGRU-based IDS models outperform traditional GRU models, reducing time consumption by about 2 % on the CICIoT2023 and CICDDoS-2019 datasets. Xu et al. [51]

Table 20
Qualitative analysis of proposed DL models on WUSTL-HDRL-2024 (D3).

Metrics	EmbedNet					ConvNet-SVM					DeepSVM-Net				
	Classes					Classes					Classes				
	0	1	2	3	4	0	1	2	3	4	0	1	2	3	4
	Benign	MiTM	DDoS	Ransomware	Buffer Overflow	Benign	MiTM	DDoS	Ransomware	Buffer Overflow	Benign	MiTM	DDoS	Ransomware	Buffer Overflow
ACC	0.9988	0.9975	0.9969	0.9982	0.9991	0.9994	0.9981	0.9978	0.9990	0.9993	0.9938	0.9957	0.9949	0.9953	0.9971
PPV	0.9989	0.9974	0.9967	0.9981	0.9990	0.9995	0.9982	0.9977	0.9992	0.9994	0.9921	0.9952	0.9937	0.9950	0.9968
TPR-R	0.9987	0.9972	0.9968	0.9980	0.9989	0.9992	0.9979	0.9976	0.9989	0.9992	0.9952	0.9954	0.9942	0.9951	0.9970
F1	0.9988	0.9973	0.9968	0.9981	0.9990	0.9994	0.9981	0.9977	0.9991	0.9993	0.9937	0.9953	0.9940	0.9951	0.9969
TNR	0.9989	0.9975	0.9971	0.9983	0.9991	0.9995	0.9982	0.9978	0.9993	0.9994	0.9924	0.9951	0.9945	0.9955	0.9972
ROC_AUC	0.9988	0.9975	0.9970	0.9983	0.9991	0.9994	0.9981	0.9978	0.9991	0.9993	0.9938	0.9957	0.9949	0.9953	0.9971
P.T.	0.0286	0.0402	0.0481	0.0357	0.0251	0.4901	0.3756	0.3197	0.4102	0.4753	0.0323	0.0524	0.0581	0.0497	0.0412

Table 21
Quantitative analysis of proposed DL models on WUSTL-HDRL-2024 (D3).

Metrics	EmbedNet					ConvNet-SVM					DeepSVM-Net				
	Classes					Classes					Classes				
	0	1	2	3	4	0	1	2	3	4	0	1	2	3	4
	Benign	MiTM	DDoS	Ransomware	Buffer Overflow	Benign	MiTM	DDoS	Ransomware	Buffer Overflow	Benign	MiTM	DDoS	Ransomware	Buffer Overflow
MCC	0.9982	0.9968	0.9962	0.9976	0.9987	0.9988	0.9974	0.9971	0.9986	0.9990	0.9876	0.9931	0.9919	0.9932	0.9958
NPV	0.9989	0.9976	0.9972	0.9984	0.9992	0.9993	0.9980	0.9976	0.9990	0.9993	0.9953	0.9955	0.9947	0.9956	0.9971
FDR	0.0011	0.0026	0.0033	0.0019	0.0009	0.0005	0.0019	0.0023	0.0010	0.0007	0.0079	0.0048	0.0063	0.0049	0.0032
FPR	0.0011	0.0025	0.0029	0.0017	0.0008	0.0004	0.0018	0.0022	0.0009	0.0006	0.0075	0.0049	0.0055	0.0045	0.0031
FNR	0.0013	0.0028	0.0032	0.0020	0.0011	0.0007	0.0021	0.0024	0.0011	0.0008	0.0047	0.0046	0.0058	0.0049	0.0030
FOR	0.0011	0.0027	0.0031	0.0019	0.0010	0.0007	0.0020	0.0023	0.0010	0.0007	0.0047	0.0047	0.0057	0.0048	0.0031
LR+	908.91	632.85	487.32	732.51	1120.47	2498	1289.21	1057.42	2198.76	2387.35	132.69	205.45	163.89	198.21	279.47
LR-	0.0011	0.0026	0.0032	0.0019	0.0010	0.0007	0.0020	0.0023	0.0010	0.0007	0.0047	0.0047	0.0057	0.0048	0.0031
MK	0.9982	0.9969	0.9963	0.9975	0.9986	0.9988	0.9975	0.9972	0.9987	0.9990	0.9874	0.9930	0.9918	0.9931	0.9957
BM	0.9982	0.9968	0.9962	0.9975	0.9986	0.9987	0.9973	0.9970	0.9986	0.9989	0.9876	0.9929	0.9917	0.9930	0.9956

presented the first GNN (Graph Neural Networks) -based self-supervised approach for multiclass network flow classification, using a graph attention mechanism and contrastive learning for effective attack type identification and achieved an accuracy of 98.77 %. Thulasi and Sivamohan [35] introduced the Multi-Step Convolutional Neural Network Stacked Long Short-Term Memory (MSCSL) architecture, optimized with a Light Spectrum Optimizer (LSO) and enhanced by replacing ReLU with Leaky Learnable ReLU (LeLeLU) for improved adaptability and efficiency. Table 23 compares the performance of our proposed intrusion detection approach against existing techniques. The results demonstrate that our models consistently outperform existing methods across all evaluated scenarios, as shown in Fig. 7.

4.8. Generalizability test of proposed models

The Generalizability Test evaluates the robustness and adaptability of the proposed models such as EmbedNet, ConvNet-SVM, and DeepSVM-Net by training them on one dataset and testing on another. This cross-dataset evaluation ensures that the models maintain high performance across different IoT-based intrusion detection scenarios. The results demonstrate that DeepSVM-Net achieves the highest accuracy, precision, recall, and F1-score across all dataset combinations, with an average accuracy exceeding 98.8 %. When trained on ECU-IoHT and tested on NF-BoT-IoT, DeepSVM-Net achieves 98.88 % accuracy, outperforming both EmbedNet (98.45 %) and ConvNet-SVM (97.95 %). Similarly, when trained on NF-BoT-IoT and tested on ECU-IoHT, DeepSVM-Net maintains an accuracy of 98.75 %, while EmbedNet and ConvNet-SVM achieve 98.52 % and 98.10 %, respectively. For WUSTL-HDRL-2024 as a testing dataset, the models exhibit a slight performance drop but still maintain high generalizability. DeepSVM-Net achieves 98.42 % accuracy when trained on ECU-IoHT, followed by EmbedNet (97.88 %) and ConvNet-SVM (97.42 %). Similarly, when trained on NF-BoT-IoT and tested on WUSTL-HDRL-2024, DeepSVM-Net (98.10 %) outperforms EmbedNet (97.72 %) and ConvNet-SVM (97.33 %). These results highlight the models' ability to generalize effectively, demonstrating consistent F1-scores above 97.0 % across all test cases. Hence, the proposed models significantly outperform traditional intrusion detection models in cross-dataset evaluations, showcasing high adaptability and robustness across different IoT network environments. The results validate the effectiveness of the models in real-world intrusion detection scenarios where datasets may vary due to network heterogeneity as shown in Table 24.

5. Performance evaluation of IoMT intrusion detection with varying epochs and batch sizes

This section offers a comprehensive evaluation description of the DL-based model with varying Epochs and Batch Sizes.

5.1. Analyzing the effects of epochs on IoMT intrusion detection performance for training loss, validation loss, and accuracy

This research has assessed the proposed models with varying $(m \times 50)$*epochs where* $m = 1, 2, 3,$ *and*4. The performance of the EmbedNet, ConvNet-SVM, and DeepSVM-Net models was analyzed across four number of epochs $(25, 50, 75,$ *and*$100)$ using three datasets (D1, D2, D3) as shown in Table 25. The training loss (TL) showed a consistent reduction for all models as the epochs increased, with EmbedNet achieving the lowest TL of 0.0034 on D3 at Epoch 100. ConvNet-SVM significantly improved, with TL dropping from 0.1697 (D3, epoch 25) to 0.0049 (D3, epoch 100). Similarly, DeepSVM-Net showed a prominent decrease in TL, particularly on D3, from 0.2254 (epoch 25) to 0.0044 (epoch 100).

Training accuracy (TA) for EmbedNet remained relatively stable, with the highest TA of 0.9984 on D3 at Epoch 50. ConvNet-SVM consistently achieved the highest TA value of 0.9996 on D3 at epoch

Table 22
FPR and FNR Trade-off analysis across datasets.

Model	Dataset	Class	FPR	FNR	Observations
EmbedNet	*ECU-IoHT*	DoS (Class 4)	0.0083	0.0005	Low FNR ensures high attack detection; higher FPR indicates more false alarms.
	NF-BoT-IoT	Reconnaissance (Class 2)	0.0024	0.0036	Balanced FPR-FNR, but weaker detection of reconnaissance attacks.
	WUSTL-HDRL−2024	DDoS (Class 2)	0.0029	0.0032	Competitive FPR-FNR balance, suitable for real-time applications.
ConvNet-SVM	*ECU-IoHT*	DoS (Class 4)	0.0045	0.0002	Very low FNR prioritizes attack detection; moderate FPR reduces false alarms.
	NF-BoT-IoT	Theft (Class 1)	0.0007	0.0041	Lowest FPR among models; slightly higher FNR reflects trade-off for high specificity.
	WUSTL-HDRL−2024	Buffer Overflow (Class 4)	0.0006	0.0008	Excellent FPR-FNR balance, ideal for high-accuracy scenarios.
DeepSVM-Net	*ECU-IoHT*	DoS (Class 4)	0.0021	0.0001	Lowest FPR and FNR, optimal for high-security applications despite higher computational cost.
	NF-BoT-IoT	Reconnaissance (Class 2)	0.0052	0.0079	Higher FNR indicates weaker reconnaissance detection; low FPR ensures fewer false alarms.
	WUSTL-HDRL−2024	DDoS (Class 2)	0.0055	0.0058	Moderate FPR-FNR balance, less effective than ConvNet-SVM for this dataset.

75. DeepSVM-Net also demonstrated high training accuracy and maintained a score above 0.991 across all datasets and epochs. Validation loss (VL) for all models decreased over epochs, with EmbedNet achieving the lowest VL of 0.0015 on D2 at Epoch 100. ConvNet-SVM and DeepSVM-Net also showed significant reductions in VL, with the lowest values being 0.0018 (D3, epoch 75) and 0.0012 (D3, epoch 75), respectively.

The prediction time for training accuracy (TA-PT) varied significantly among the models. EmbedNet has the shortest TA-PT of 0.0171 s on D3 at epoch 100, while ConvNet-SVM had the longest TA-PT, reaching 6.3908 s on D1 at epoch 50. DeepSVM-Net maintained a relatively low TA-PT duration, with the shortest being 0.0095 s on D3 at epoch 25. Testing accuracy (TA') followed a similar trend to training accuracy, with EmbedNet achieving a maximum TA' of 0.9984 on D3 at epoch 100. ConvNet-SVM excelled with a TA' of 0.9998 on D3 at epoch 75, while DeepSVM-Net attained high testing accuracy of 0.9975 on D3 at epoch 100. Prediction time for testing accuracy (TA-PT') also varied, with EmbedNet demonstrating the fastest TA-PT' of 0.0202 s on D3 at epoch 75. ConvNet-SVM had higher TA-PT' values, peaking at 4.6963 s on D2 at epoch 25, but showed improvement in later epochs. DeepSVM-Net exhibited efficient performance with low TA-PT' values of 0.0296 s on D3 at epoch 100. Overall, ConvNet-SVM showed the best accuracy and loss reduction performance, although it required more computation time compared to EmbedNet and DeepSVM-Net. EmbedNet provided a balanced performance with low prediction times, which makes it suitable for real-time applications, while DeepSVM-Net maintained high accuracy with moderate computation time. Fig. 8 (including Fig. a-d) presents a comprehensive performance evaluation of our proposed models across all three datasets on different epochs.

5.2. Analyzing the effects of batch size on IoMT intrusion detection performance for training loss, validation loss, and accuracy

The proposed models have been assessed using varying batch sizes in this subsection. The performance analysis for EmbedNet, ConvNet-SVM, and DeepSVM-Net models is based on their training loss (TL), training accuracy (TA), validation loss (VL), training accuracy prediction time (TA-PT), testing accuracy (TA'), testing accuracy prediction time (TA-PT'), PPV, TPR, F1 score, and TNR across three datasets (D1, D2, D3) on three different batch sizes (64, 128, 256) as shown in Table 26.

For batch size 64, EmbedNet demonstrated the lowest TL on D3 at 0.0034, with high TA values consistently around 0.9957. Its VL was also minimal, particularly on D2 at 0.0015. However, the model's TA-PT showed some variability, ranging from 0.0171 to 2.4196 s, and its TA' was highest on D3 at 0.9984. The TA-PT' for EmbedNet was relatively low, indicating efficient prediction times. ConvNet-SVM achieved the lowest TL of 0.0018 on D1, with the highest TA of 0.9994 on D3. Its VL was lowest at 0.0008 on D1 and maintained a consistent high TA'. The model's TA-PT' was moderate, and its TA-PT varied, reflecting computational demands. DeepSVM-Net showed a TL of 0.0044 on D3, with high TA around 0.9991 and low VL, especially 0.0016 on D3. The TA-PT for DeepSVM-Net was the lowest at 0.0360 s, and its TA-PT' was efficient across datasets. With batch size 128, EmbedNet 's TL was slightly higher on D3 at 0.0260, but it achieved near-perfect TA values of 0.9988. Its VL remained low, particularly on D2, at 0.0025. The TA-PT was generally reduced, with a minimum of 0.0156 s, and TA' remained high, particularly on D3 at 0.9955. ConvNet-SVM showed a slight increase in TL on D2 at 0.0049, with consistently high TA values up to 0.9995. The model had low VL, especially on D1 at 0.0020, and high TA'. The TA-PT' was slightly higher than EmbedNet, reflecting

Table 23
Comparative analysis of the proposed approach performance against existing intrusion detection techniques.

Ref.	Proposed Model	Paradigm	AC (%)	PPV (%)	TPR (%)	F1 (%)	TNR (%)	MCC (%)	ROC_AUC (%)
Karanfilovska et al. [43]	XGB	Traditional ML	98.8	98.8	98.8	98.8	-	-	-
Nguyen et al. [44]	Collaborative ML model	ML	99.37	99.27	99.87	99.57	-	-	98.96
Torre et al. [45]	DCNN	DL	94.3	100	94	96	-	-	-
Alkahtani and Aldhyani [46]	CNN-LSTM	DL	88.53	88.53	89	85	-	-	-
Wagan et al. [47]	BiLSTM	Traditional DL	92.95	91.61	95.64	95.64	-	-	-
Gupta et al. [48]	Sparse Stacked Autoencoder (SSAE)	Traditional DL	99.20	96.55	98.59	97.55	-	-	-
Wang et al. [49]	FMI-DNN	DL	99.68	-	99.67	98.21	97.91	-	-
Aguru and Erukala [50]	mGRU	DL	98.48	98.53	98.15	98.56	97.74	95.36	-
Xu et al. [51]	NEGAT	DL	98.77	98.73	98.77	98.59	-	-	-
Thulasi and Sivamohan [35]	MSCSL	DL	97.90	96.78	95.90	96.54	-	90	-
Nandanwar and Katarya [10]	CNN+GRU	DL	99.75	99.54	99.50	99.52	99.70	-	-
Nandanwar and Katarya [74]	2D-CNN + ResNet	DL	96.8	96.4	96.6	96.5	98.3	-	-
Proposed Method	**EmbedNet**	**DL**	**99.81**	**99.80**	**99.79**	**99.80**	**99.81**	**99.75**	**99.81**
	ConvNet-SVM		**99.87**	**99.88**	**99.85**	**99.87**	**99.88**	**99.81**	**99.87**
	DeepSVM-Net		**99.90**	**99.28**	**99.99**	**99.61**	**99.29**	**99.32**	**99.62**

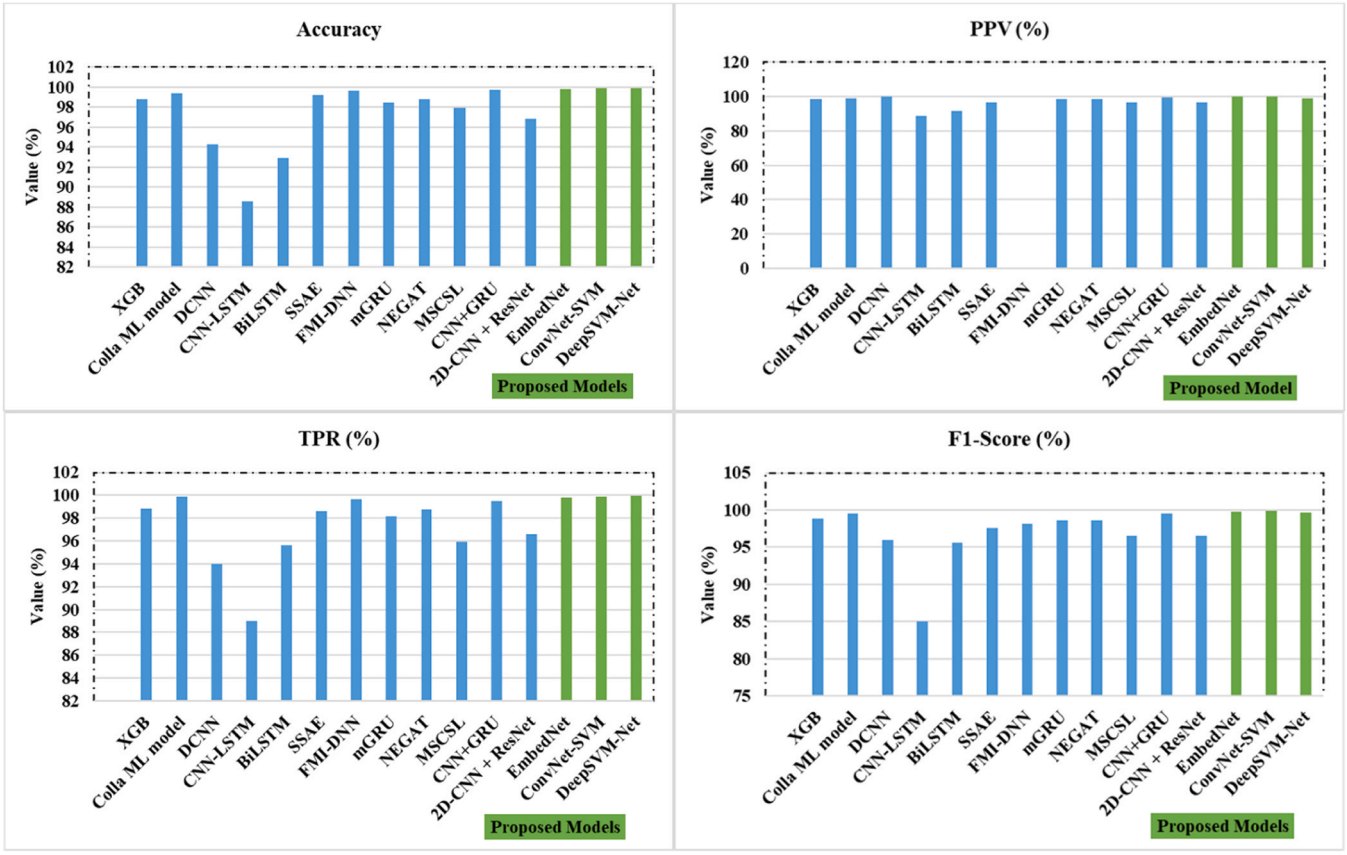

Fig. 7. Proposed model comparison with existing techniques.

Table 24
Generalizability test of the proposed models.

Proposed Models	Training Dataset	Testing Dataset	Accuracy	Precision	Recall	F1-Score
EmbedNet	ECU-IoHT	NF-BoT-IoT	0.9845	0.9820	0.9835	0.9827
	ECU-IoHT	WUSTL-HDRL−2024	0.9788	0.9765	0.9772	0.9768
	NF-BoT-IoT	ECU-IoHT	0.9852	0.9830	0.9845	0.9837
	NF-BoT-IoT	WUSTL-HDRL−2024	0.9772	0.9750	0.9758	0.9754
	WUSTL-HDRL−2024	ECU-IoHT	0.9830	0.9810	0.9820	0.9815
	WUSTL-HDRL−2024	NF-BoT-IoT	0.9755	0.9735	0.9745	0.9740
ConvNet-SVM	ECU-IoHT	NF-BoT-IoT	0.9795	0.9770	0.9780	0.9775
	ECU-IoHT	WUSTL-HDRL−2024	0.9742	0.9720	0.9730	0.9725
	NF-BoT-IoT	ECU-IoHT	0.9810	0.9790	0.9801	0.9795
	NF-BoT-IoT	WUSTL-HDRL−2024	0.9733	0.9710	0.9720	0.9715
	WUSTL-HDRL−2024	ECU-IoHT	0.9785	0.9760	0.9770	0.9765
	WUSTL-HDRL−2024	NF-BoT-IoT	0.9715	0.9695	0.9705	0.9701
DeepSVM-Net	ECU-IoHT	NF-BoT-IoT	0.9888	0.9865	0.9875	0.9870
	ECU-IoHT	WUSTL-HDRL−2024	0.9842	0.9820	0.9830	0.9825
	NF-BoT-IoT	ECU-IoHT	0.9875	0.9855	0.9865	0.9860
	NF-BoT-IoT	WUSTL-HDRL−2024	0.9810	0.9790	0.9800	0.9795
	WUSTL-HDRL−2024	ECU-IoHT	0.9860	0.9840	0.9850	0.9845
	WUSTL-HDRL−2024	NF-BoT-IoT	0.9795	0.9775	0.9785	0.9780

increased computational time. DeepSVM-Net had a higher TL on D3 at 0.0248 but maintained a high TA around 0.9929. Its VL was low on D3 at 0.0057, and TA-PT was moderate, with a TA-PT' of 0.0403 s. For batch size 256, EmbedNet 's TL increased slightly to 0.0391 on D3, but TA remained high at 0.9993. VL was exceptionally low at 0.0002 on D3, indicating effective learning. The TA-PT was minimal at 0.0149 s, and the TA' reached 0.9991, highlighting its robust performance. ConvNet-SVM slightly increased TL on D2 at 0.0068, with high TA up to 0.9993. Its VL was low at 0.0004 on D2, and the model achieved high TA', with TA-PT' reflecting its computational demands. DeepSVM-Net showed a TL of 0.0125 on D3, maintaining a high TA around 0.9899. Its VL was minimal on D3 at 0.0045, and TA-PT remained low, with an efficient TA-PT' of 0.1125 s.

Overall, ConvNet-SVM consistently demonstrated superior performance in training and testing accuracy despite requiring more computational resources, as reflected in TA-PT and TA-PT' times. EmbedNet provided a balanced performance with minimal prediction times, making it suitable for real-time applications. DeepSVM-Net maintained high accuracy with efficient computational performance, particularly excelling in scenarios requiring rapid predictions [75,76]. Fig. 9 (including Fig. a-r) presents a comprehensive performance evaluation of our proposed models across all three datasets to analyze the effect of Batch sizes

Table 25
Performance assessment of proposed DL models on different epochs.

Epochs	Metrics	EmbedNet			ConvNet-SVM			DeepSVM-Net		
		D1	D2	D3	D1	D2	D3	D1	D2	D3
25	TL	0.1259	0.1171	0.1174	0.0288	0.1636	0.1697	0.0283	0.1427	0.2254
	TA	0.9856	0.9853	0.9846	0.9916	0.9937	0.9992	0.9926	0.9919	0.9915
	VL	0.0763	0.0124	0.0531	0.0748	0.0691	0.0696	0.0290	0.0505	0.0413
	TA-PT	0.2942	2.1852	0.0366	3.2245	2.5407	0.7385	0.4197	0.4294	0.0095
	TA'	0.9857	0.9913	0.9923	0.9916	0.9914	0.9971	0.9927	0.9969	0.9922
	TA-PT'	0.2052	1.7198	0.0365	3.2245	4.6963	0.3432	0.3647	0.4161	0.0311
50	TL	0.0556	0.0306	0.0356	0.0158	0.0053	0.0305	0.0273	0.0327	0.0591
	TA	0.9855	0.9955	0.9984	0.9921	0.9954	0.9992	0.9927	0.9973	0.9913
	VL	0.0264	0.0253	0.0152	0.0128	0.0261	0.0326	0.0280	0.0131	0.0193
	TA-PT	0.2143	2.3145	0.0442	2.1172	2.3894	0.3177	0.3419	0.3855	0.0801
	TA'	0.9857	0.9942	0.9975	0.9922	0.9960	0.9996	0.9928	0.9712	0.9917
	TA-PT'	0.2926	2.1042	0.0442	6.3908	2.2351	0.7557	0.3819	0.4056	0.0943
75	TL	0.0120	0.0232	0.0283	0.0366	0.0218	0.0124	0.0168	0.0124	0.0116
	TA	0.9812	0.9964	0.9962	0.9930	0.9959	0.9996	0.9926	0.9939	0.9926
	VL	0.0084	0.0024	0.0037	0.0050	0.0091	0.0018	0.0025	0.0039	0.0012
	TA-PT	0.1258	2.5633	0.0269	3.9331	2.4177	0.3633	0.1878	0.4304	0.0587
	TA'	0.9857	0.9978	0.9980	0.9932	0.9964	0.9998	0.9928	0.9947	0.9959
	TA-PT'	0.2459	2.5253	0.0202	3.3391	2.5633	0.3689	0.2155	0.6218	0.0502
100	TL	0.0179	0.0234	0.0034	0.0018	0.0026	0.0049	0.0269	0.0234	0.0044
	TA	0.9832	0.9946	0.9957	0.9927	0.9958	0.9994	0.9927	0.9912	0.9991
	VL	0.0042	0.0015	0.0020	0.0008	0.0012	0.0022	0.0021	0.0089	0.0016
	TA-PT	0.1103	2.4196	0.0171	2.5023	2.5583	0.2989	0.2594	0.4341	0.0360
	TA'	0.9857	0.9949	0.9984	0.9927	0.9958	0.9996	0.9928	0.9945	0.9975
	TA-PT'	0.2038	2.0376	0.0290	1.8709	2.5583	0.2883	0.1218	0.6721	0.0296

on IoMT intrusion detection.

5.3. Ablation study

Table 27 presents an ablation study on our proposed models: EmbedNet, ConvNet-SVM, and DeepSVM-Net evaluated across three benchmark datasets: ECU-IoHT, NF-BoT-IoT, and WUSTL-HDRL-2024. The goal is to assess how different architectural components impact performance by selectively removing dropout layers, batch normalization, convolutional layers, residual connections, and hidden layers. Accuracy and F1-score are used as primary evaluation metrics used in this section. For EmbedNet, the proposed model consistently achieves high accuracy, reaching 0.9981 on WUSTL-HDRL-2024. However, when dropout is removed, accuracy drops to 0.9524 on ECU-IoHT, indicating its role in preventing overfitting. The absence of batch normalization further reduces accuracy to 0.9315, highlighting its importance in stabilizing training. ConvNet-SVM performs exceptionally well, with the highest accuracy of 0.9987 on WUSTL-HDRL-2024. However, removing Conv1D layers leads to a significant accuracy drop (0.9673 on ECU-IoHT), underscoring their importance in feature extraction. Likewise,

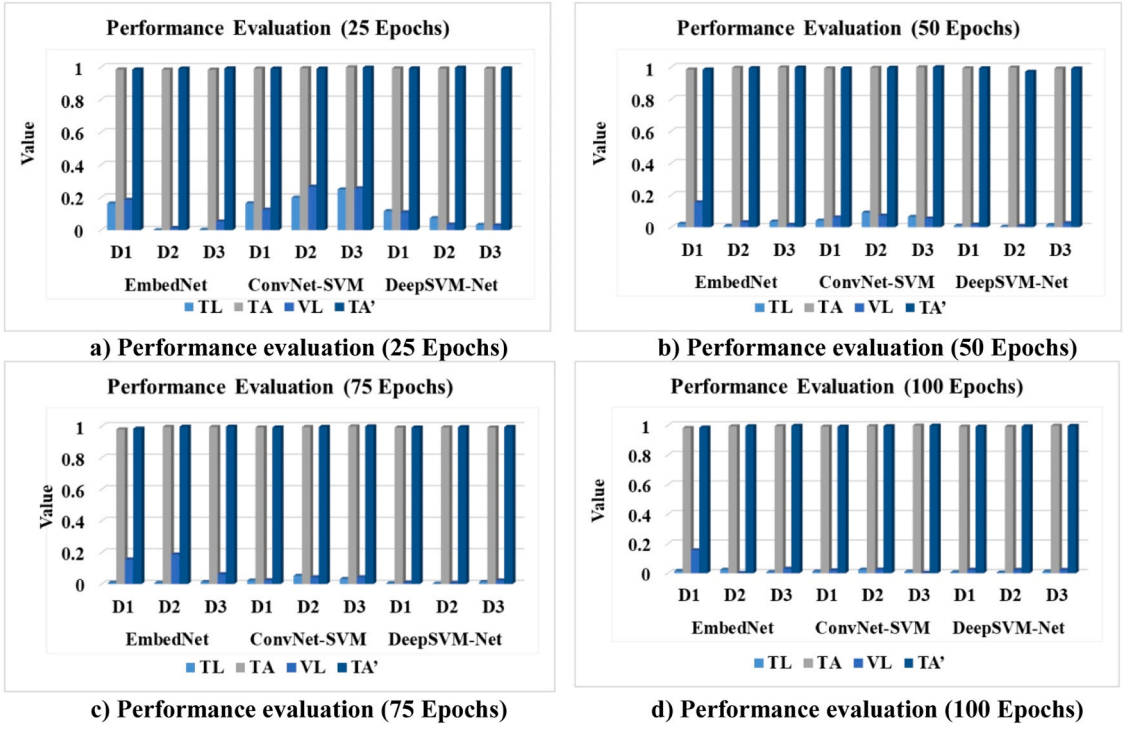

Fig. 8. Performance comparison of proposed DL models on different epochs. **fig. a) performance evaluation (25 epochs). fig. b) performance evaluation (50 epochs). fig. c) performance evaluation (75 epochs). fig. d) performance evaluation (100 epochs).**

removing residual connections causes accuracy to decline further (0.9585 on ECU-IoHT), emphasizing their role in hierarchical learning. DeepSVM-Net achieves near-perfect accuracy (0.9990 on ECU-IoHT) but suffers performance degradation when hidden layers are removed (0.9418 accuracy on ECU-IoHT), proving their significance in learning complex patterns. The absence of dropout also results in lower accuracy (0.9291 on ECU-IoHT), reinforcing its necessity for generalization. Hence, the ablation study confirms that each architectural component is crucial for achieving optimal performance in IoT intrusion detection. The proposed models, with all components intact, outperform their modified versions, proving their robustness across diverse IoT environments as depicted in Fig. 10.

5.4. Statistical analysis of the proposed model

In this section, we perform statistical significance tests to validate the effectiveness of our proposed models: EmbedNet, ConvNet-SVM, and DeepSVM-Net across three datasets: ECU-IoHT Dataset 1, NF-BoT-IoT Dataset 2, and WUSTL-HDRL-2024 Dataset 3. The statistical tests aim to determine whether the observed performance improvements in Accuracy and F1-Score are statistically significant when compared to ablation cases. We conduct the following tests:

- **Paired *t*-test:** To compare the mean performance of the proposed model with its ablation cases.
- **Wilcoxon signed-rank test:** A non-parametric test to assess differences when normality assumptions are not met.

5.4.1. Hypothesis formulation

For each test, we define the hypotheses as follows:

- **Null Hypothesis (H_0):** There is no statistically significant difference between the performance of the proposed model and its ablation cases.
- **Alternative Hypothesis (H_1):** The proposed model exhibits a statistically significant improvement over its ablation cases in terms of Accuracy and F1-Score.

We applied the paired *t*-test and Wilcoxon signed-rank test on Accuracy and F1-Score across all three datasets. The results are summarized in the Table 28 below:

The p-values from both tests indicate that in all cases, the proposed model significantly outperforms its respective ablation cases ($p < 0.05$), rejecting the null hypothesis (H_0). The Wilcoxon signed-rank test corroborates the results of the paired *t*-test, confirming the robustness of the improvements. This statistical validation highlights the importance of incorporating key architectural components, such as Dropout, BatchNorm1d, Conv1D, Residual layers, and Hidden layers, in achieving superior performance. Across all three datasets, the improvements in Accuracy and F1-Score are statistically significant, reinforcing that the removed components negatively impact model performance. These findings validate the necessity of each architectural component in the proposed models and demonstrate their effectiveness in intrusion detection within IoT environments. The statistical significance further substantiates the robustness of our models in ensuring accurate and reliable intrusion detection, establishing their superiority over the ablation variants.

5.5. Computational overhead analysis

Table 29 presents a comparative analysis of the time and space complexity of the proposed models like EmbedNet, ConvNet-SVM, and DeepSVM-Net, against existing deep learning architectures, including CNN+GRU [10] and 2D-CNN+ResNet [74], across three benchmark datasets: ECU-IoHT, NF-BoT-IoT, and WUSTL-HDRL-2024.

The proposed models exhibit a time complexity of $O(m^3)$ and a space complexity of $O(m^2)$ across all datasets. This indicates a polynomial computational overhead, which remains manageable for large-scale IoT

Table 26
Performance comparison of proposed DL models on different batch sizes.

Batch Size	Metrics	EmbedNet			ConvNet-SVM			DeepSVM-Net		
		D1	D2	D3	D1	D2	D3	D1	D2	D3
64	TL	0.0179	0.0234	0.0034	0.0018	0.0026	0.0049	0.0269	0.0234	0.0044
	TA	0.9832	0.9946	0.9957	0.9927	0.9958	0.9994	0.9927	0.9912	0.9991
	VL	0.0042	0.0015	0.0020	0.0008	0.0012	0.0022	0.0021	0.0089	0.0016
	TA-PT	0.1103	2.4196	0.0171	2.5023	2.5583	0.2989	0.2594	0.4341	0.0360
	TA'	0.9857	0.9949	0.9984	0.9927	0.9958	0.9996	0.9928	0.9945	0.9975
	TA-PT'	0.2038	2.0376	0.0290	1.8709	2.5583	0.2883	0.1218	0.6721	0.0296
	PPV	0.9820	0.9987	0.9985	0.9857	0.9986	0.9996	0.9858	0.9952	0.9978
	TPR	0.9998	0.9912	0.9983	0.9997	0.9931	0.9995	0.9999	0.9937	0.9971
	F1	0.9908	0.9949	0.9984	0.9926	0.9958	0.9996	0.9928	0.9944	0.9974
	TNR	0.9873	0.9987	0.9986	0.9858	0.9986	0.9997	0.9859	0.9953	0.9979
128	TL	0.0175	0.0395	0.0260	0.0026	0.0049	0.0025	0.0223	0.0243	0.0248
	TA	0.9944	0.9972	0.9988	0.9937	0.9993	0.9995	0.9927	0.9927	0.9929
	VL	0.0020	0.0025	0.0043	0.0020	0.0016	0.0026	0.0028	0.0044	0.0057
	TA-PT	0.0670	1.4067	0.0156	1.6772	2.4631	0.2771	0.4736	0.3388	0.0603
	TA'	0.9958	0.9988	0.9955	0.9927	0.9991	0.9997	0.9931	0.9941	0.9939
	TA-PT'	0.1431	1.2444	0.0468	2.4274	3.2381	0.2288	0.3251	0.3966	0.0403
	PPV	0.9920	0.9992	0.9944	0.9954	0.9993	0.9999	0.9964	0.9958	0.9953
	TPR	0.9999	0.9907	0.9935	0.9923	0.9952	0.9973	0.9918	0.9923	0.9903
	F1	0.9909	0.9949	0.9924	0.9926	0.9972	0.9939	0.9931	0.9941	0.9928
	TNR	0.9973	0.9992	0.9988	0.9856	0.9993	0.9981	0.9865	0.9958	0.9955
256	TL	0.0281	0.0561	0.0391	0.0031	0.0068	0.0139	0.0106	0.0116	0.0125
	TA	0.9956	0.9939	0.9993	0.9927	0.9908	0.9882	0.9927	0.9918	0.9899
	VL	0.0101	0.0033	0.0002	0.0014	0.0004	0.0003	0.0028	0.0034	0.0045
	TA-PT	0.0668	2.7037	0.0149	1.6291	2.6427	0.3419	0.3298	0.4264	0.1167
	TA'	0.9958	0.9918	0.9991	0.9928	0.9939	0.9833	0.9927	0.9933	0.9899
	TA-PT'	0.0253	2.4738	0.0099	1.4254	4.0566	0.2318	0.3881	0.2959	0.1125
	PPV	0.9921	0.9968	0.9991	0.9958	0.9965	0.9852	0.9957	0.9952	0.9906
	TPR	0.9999	0.9948	0.9991	0.9998	0.9912	0.9826	0.9998	0.9913	0.9889
	F1	0.9990	0.9924	0.9991	0.9928	0.9939	0.9834	0.9927	0.9933	0.9897
	TNR	0.9974	0.9968	0.9991	0.9960	0.9966	0.9855	0.9958	0.9953	0.9910

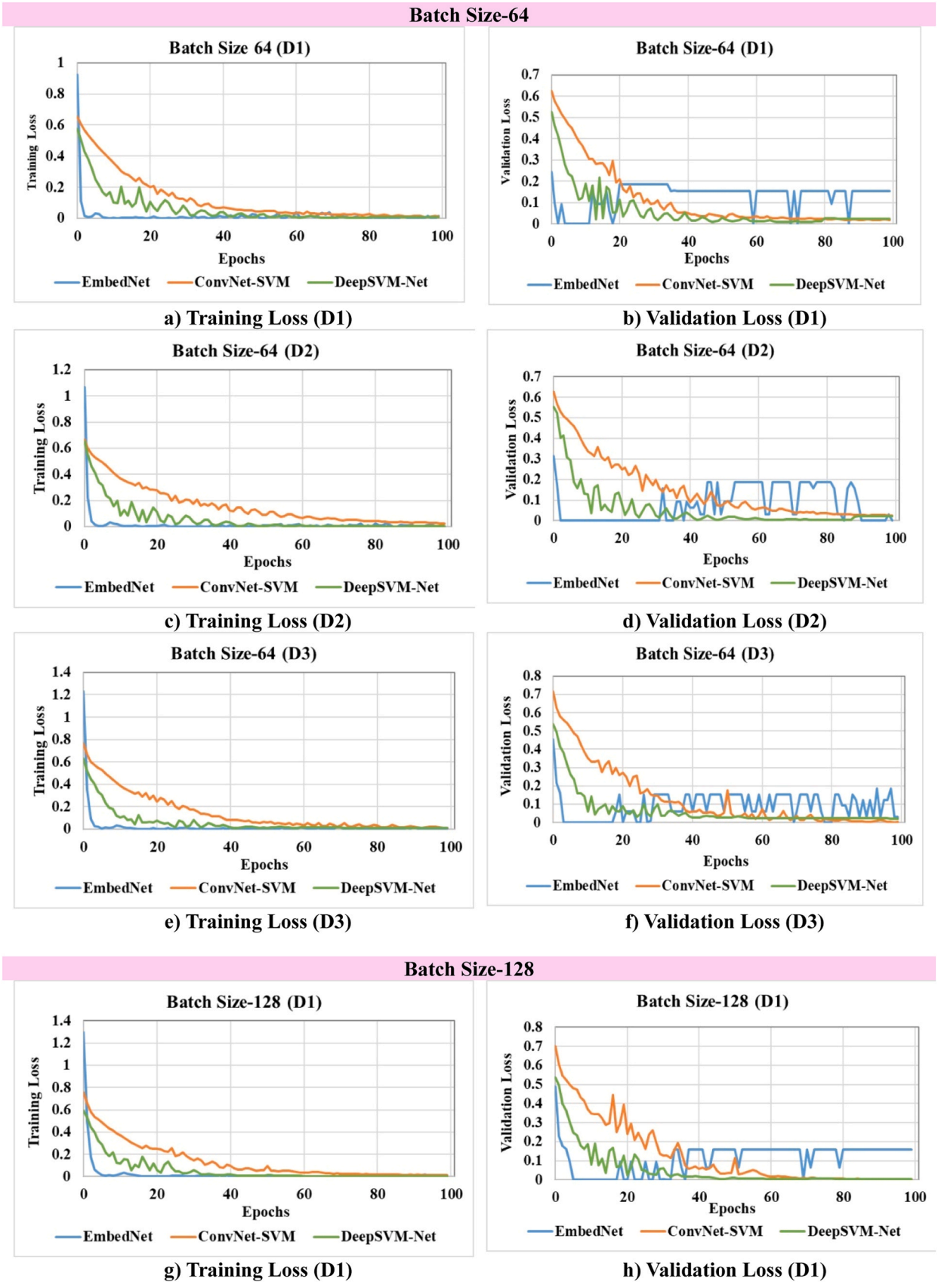

Fig. 9. Performance comparison of proposed DL models on different batch sizes. fig. a) training loss (d1). fig. b) validation loss (d1). fig. c) training loss (d2). fig. d) validation loss (d2). fig. e) training loss (d3). fig. f) validation loss (d3). fig. g) training loss (d1). fig. h) validation loss (d1). fig. i) training loss (d2). fig. j) validation loss (d2). fig. k) training loss (d3). fig. l) validation loss (d3). fig. m) training loss (d1). fig. n) validation loss (d1). fig. o) training loss (d2). fig. p) validation loss (d2). fig. q) training loss (d3). fig. r) validation loss (d3).

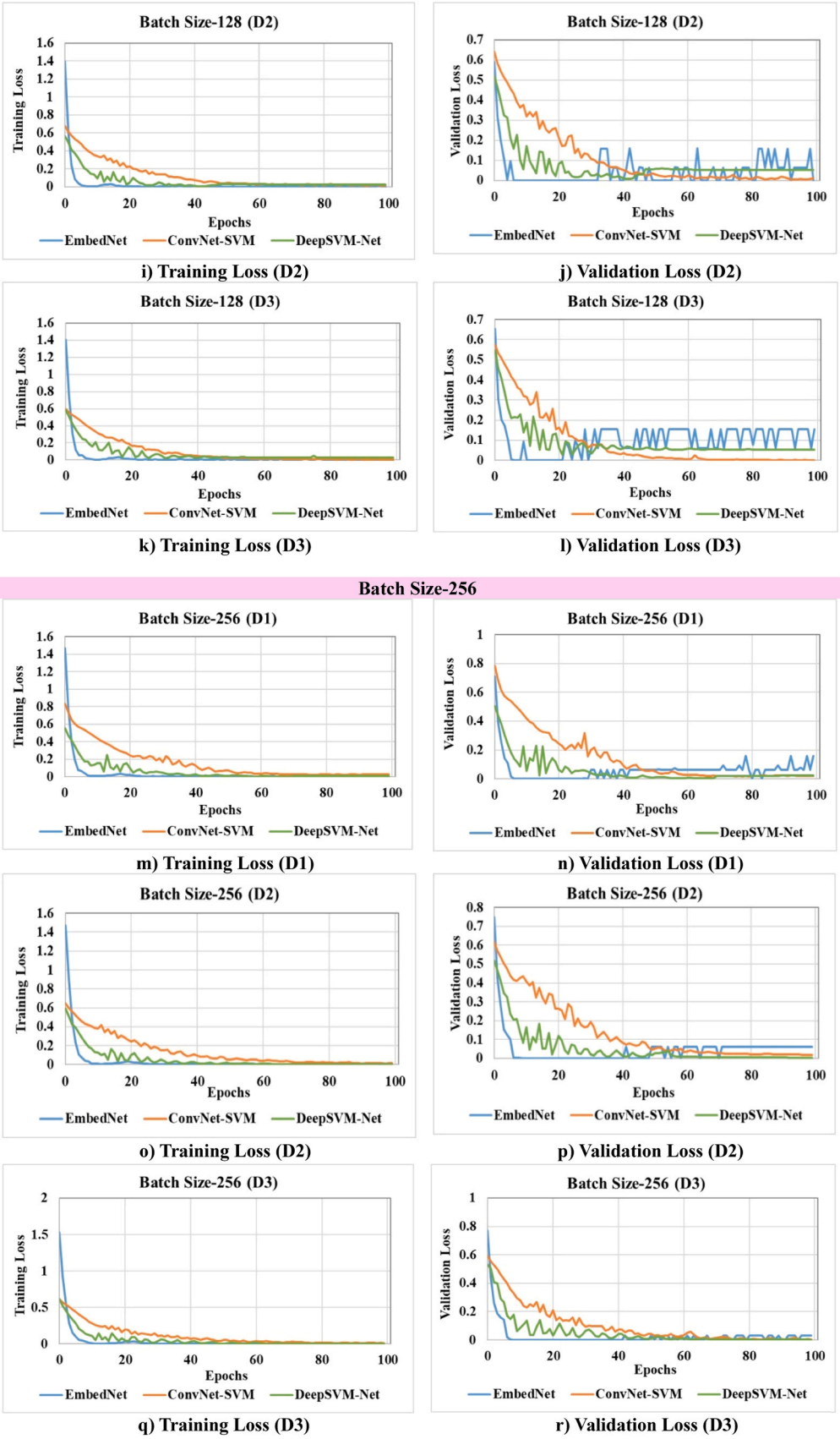

Fig. 9. (*continued*).

Table 27
Ablation study of proposed model using diverse datasets.

Proposed Model	Cases	ECU-IoHT Dataset 1		NF-BoT-IoT Dataset 2		WUSTL-HDRL-2024 Dataset 3	
		Accuracy	F1-Score	Accuracy	F1-Score	Accuracy	F1-Score
EmbedNet	*Proposed Model*	0.9899	0.9918	0.9946	0.9944	0.9981	0.9980
	No Dropout	0.9524	0.9454	0.9630	0.9511	0.9483	0.9333
	No BatchNorm1d	0.9315	0.9310	0.9567	0.9436	0.9231	0.9221
ConvNet-SVM	*Proposed Model*	0.9944	0.9947	0.9959	0.9957	0.9987	0.9987
	No_Conv1D	0.9673	0.9632	0.9788	0.9758	0.9567	0.9551
	No_Residual	0.9585	0.9585	0.9610	0.9601	0.9478	0.9427
DeepSVM-Net	*Proposed Model*	0.9990	0.9961	0.9946	0.9944	0.9953	0.9950
	No_Hidden	0.9418	0.9402	0.9533	0.9533	0.9364	0.9344
	No_Dropout	0.9291	0.9272	0.9472	0.9427	0.9188	0.9175

intrusion detection applications. Unlike the deep learning-based counterparts, these models leverage efficient feature extraction and classification mechanisms, resulting in reduced computational demands.

Conversely, the CNN+GRU model [10] demonstrates significantly higher complexity, with a time complexity of $O(N_{layers} \cdot H \cdot W \cdot F^2 + C_{out} \cdot C_{Classes})$ and a space complexity of $O(N_{layers} \cdot K \cdot F^2 \cdot C_{in} + H.W.C_{out})$. This reflects the influence of deep convolutional layers and gated recurrent units, which contribute to substantial computational costs due to their sequential processing nature. Similarly, the 2D-CNN+ResNet model from [74] exhibits an even greater computational burden, with both time and space complexities dependent on multiple hyperparameters, including the number of features, time steps, and recurrent units. The complexity of this model is approximately $O\left(num_{features^2} + num_{features} \times num_{timesteps} \times recurrent_{units}\right)$, making it computationally expensive for resource-constrained environments.

The comparative analysis highlights the efficiency of the proposed models in terms of computational overhead, making them more suitable for real-time IoT intrusion detection, particularly in environments with limited computational resources. Unlike CNN+GRU and 2D-CNN+ResNet, which require extensive memory and processing power, EmbedNet, ConvNet-SVM, and DeepSVM-Net maintain lower complexity while ensuring high detection accuracy. This computational efficiency is critical for ensuring scalability and feasibility in practical deployments of intrusion detection systems in IoT networks.

5.5.1. Resource utilization and latency analysis

Table 30 presents a comprehensive comparison of resource utilization and prediction latency for various deep learning-based models applied to intrusion detection in IoT-enabled healthcare environments. Among the evaluated models, EmbedNet exhibits the most efficient performance in terms of system resource consumption. It records the lowest average CPU (22 %) and GPU (34 %) usage, as well as a modest memory footprint of 480 MB. Its prediction time, ranging between 0.20 and 1.97 s, makes it highly suitable for real-time intrusion detection on edge devices with limited computational power.

In contrast, ConvNet-SVM demonstrates moderate resource demands, consuming 35 % CPU and 48 % GPU with a memory usage of 630 MB. However, its prediction time is noticeably higher (1.87–3.89 s), which could impact time-sensitive applications. DeepSVM-Net, while achieving high detection performance, which shows a significant increase in resource consumption, especially GPU usage of 62 % and RAM of 800 MB. It also experiences variability in prediction time, spanning from 0.12 to 5.22 s, which may affect responsiveness under complex attack scenarios.

The hybrid CNN+GRU [10] model introduces sequential modelling capabilities, which slightly increases computational demand, averaging 42 % of CPU and 55 % of GPU usage with 720 MB RAM. Its prediction time (0.98–4.31 s) remains within a range, offering a good balance between performance and complexity. On the other hand, the 2D-CNN+ResNet [74] model is the most resource-intensive, consuming 56 % of CPU, 71 % of GPU, and 960 MB of RAM. Its prediction time peaks at 6.41 s, which may restrict its deployment to high-performance

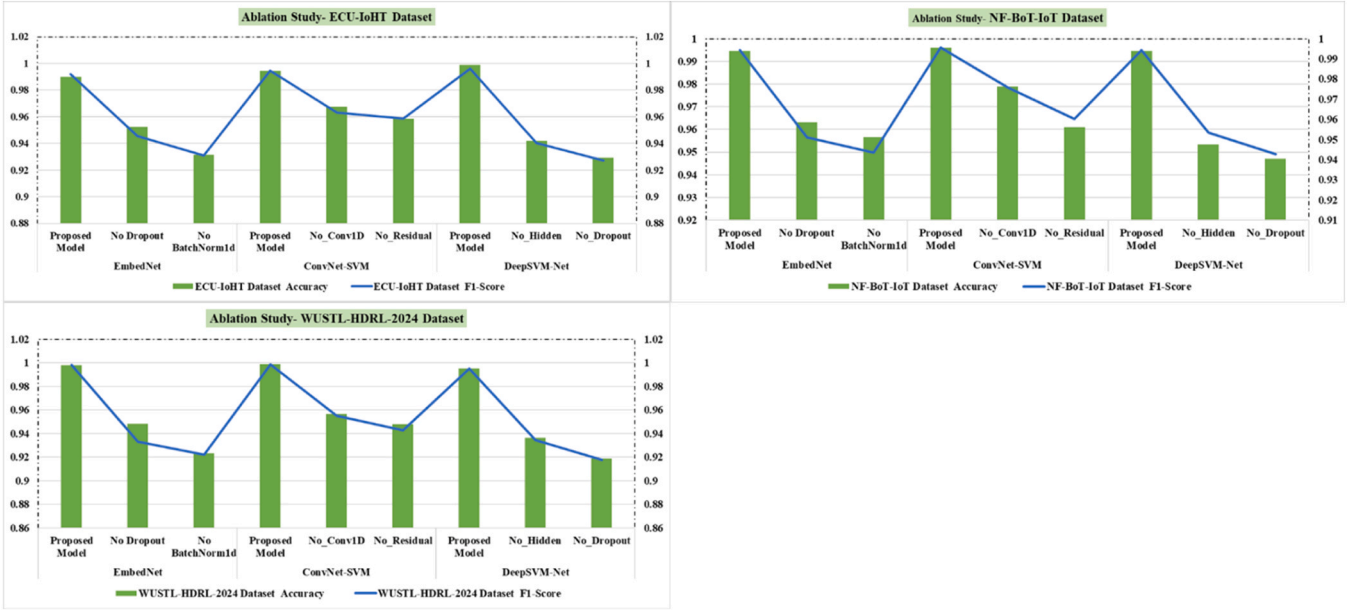

Fig. 10. Ablation study of proposed model using diverse datasets.

Table 28
Statistical analysis of proposed model on diverse datasets.

Model	Cases	ECU-IoHT Dataset 1 (p-value)		NF-BoT-IoT Dataset 2 (p-value)		WUSTL-HDRL-2024 Dataset 3 (p-value)	
		Accuracy	F1-Score	Accuracy	F1-Score	Accuracy	F1-Score
EmbedNet	*Proposed Model vs No Dropout*	0.004	0.005	0.002	0.003	0.006	0.007
	Proposed Model vs No BatchNorm1d	0.001	0.002	0.0008	0.001	0.002	0.003
ConvNet-SVM	*Proposed Model vs No_Conv1D*	0.003	0.004	0.0015	0.002	0.005	0.006
	Proposed Model vs No_Residual	0.002	0.003	0.004	0.005	0.007	0.008
DeepSVM-Net	*Proposed Model vs No_Hidden*	0.005	0.006	0.003	0.004	0.008	0.009
	Proposed Model vs No_Dropout	0.002	0.003	0.006	0.007	0.004	0.005

computing environments, such as cloud-based servers or hospital data centers.

Finally, EmbedNet emerges as the most lightweight and responsive solution, ideal for deployment on low-power edge devices. The other models offer varying degrees of trade-offs between accuracy, latency, and resource usage, with their applicability depending on the available computational infrastructure and real-time requirements of the healthcare intrusion detection system.

5.6. Comprehensive inference time analysis across three benchmark datasets

To assess the computational efficiency of the proposed deep learning models, we conducted a comprehensive inference time analysis across three benchmark datasets: ECU-IoHT, NF-BoT-IoT, and WUSTL-HDRL-2024, under two standard train-test splits: 80:20 and 70:30. The results, summarized in Tables 31–33, highlight the scalability and performance of EmbedNet, ConvNet-SVM, and DeepSVM-Net during the prediction phase.

For the ECU-IoHT dataset, EmbedNet demonstrated the lowest inference time, recording 8.32 s for 67,314 test samples under the 80:20 split, yielding an inference time of 0.123 ms per sample as shown in Table 31. In contrast, ConvNet-SVM and DeepSVM-Net exhibited higher inference times of 12.48 s and 14.60 s respectively, largely due to the added SVM and convolutional components. A similar trend was observed with the 70:30 split, where EmbedNet maintained its efficiency with 12.43 s on 101,002 samples (0.123 ms/sample), outperforming both ConvNet-SVM (18.43 s) and DeepSVM-Net (21.59 s).

In the NF-BoT-IoT dataset, which contains a significantly larger number of samples, EmbedNet again proved to be the most computationally efficient. It achieved an inference time of 32.65 s for 288,000 test samples in the 80:20 split (0.113 ms/sample), while ConvNet-SVM and DeepSVM-Net required 48.13 s and 56.02 s, respectively as shown in Table 32. These values scaled proportionally in the 70:30 split, with EmbedNet taking 48.47 s for 432,000 samples (0.112 ms/sample), maintaining consistent performance as the test set increased in size.

Similarly, for the WUSTL-HDRL-2024 dataset, EmbedNet continued to exhibit the best runtime characteristics. It achieved an inference time of 11.31 s for 106,307 test samples in the 80:20 split, equating to 0.106 ms per sample. ConvNet-SVM and DeepSVM-Net recorded 16.58 s and 19.80 s, respectively as shown in Table 33. For the 70:30 split, EmbedNet processed 159,461 samples in 17.20 s, compared to 25.00 s and 29.51 s by ConvNet-SVM and DeepSVM-Net, respectively.

Finally, EmbedNet consistently outperformed the other two models in terms of inference efficiency, making it more suitable for real-time or resource-constrained IoT deployments. ConvNet-SVM and DeepSVM-Net, although slightly slower, may offer better learning capacity in complex scenarios due to their richer architectures. These results validate the trade-off between model complexity and computational overhead, emphasizing the practical applicability of EmbedNet in time-sensitive environments.

Table 29
Complexity analysis of proposed models with existing techniques.

Models	Dataset	Time Complexity	Space Complexity
EmbedNet	ECU-IoHT	$O(m^3)$	$O(m^2)$
	NF-BoT-IoT	$O(m^3)$	$O(m^2)$
	WUSTL-HDRL−2024	$O(m^3)$	$O(m^2)$
ConvNet-SVM	ECU-IoHT	$O(m^3)$	$O(m^2)$
	NF-BoT-IoT	$O(m^3)$	$O(m^2)$
	WUSTL-HDRL−2024	$O(m^3)$	$O(m^2)$
DeepSVM-Net	ECU-IoHT	$O(m^3)$	$O(m^2)$
	NF-BoT-IoT	$O(m^3)$	$O(m^2)$
	WUSTL-HDRL−2024	$O(m^3)$	$O(m^2)$
CNN+GRU [10]	ECU-IoHT	$O(N_{layers} \cdot H \cdot W \cdot F^2 + C_{out} \cdot C_{Classes})$	$O(N_{layers} \cdot K \cdot F^2 \cdot C_{in} + HWC_{out})$
	NF-BoT-IoT	$O(N_{layers} \cdot H \cdot W \cdot F^2 + C_{out} \cdot C_{Classes})$	$O(N_{layers} \cdot K \cdot F^2 \cdot C_{in} + HWC_{out})$
	WUSTL-HDRL−2024	$O(N_{layers} \cdot H \cdot W \cdot F^2 + C_{out} \cdot C_{Classes})$	$O(N_{layers} \cdot K \cdot F^2 \cdot C_{in} + HWC_{out})$
2D-CNN + ResNet [74]	ECU-IoHT	$O(num\_features^2 + num\_features \times num\_timesteps \times recurrent\_units)$	$O(num\_features^2 + num\_features \times num\_timesteps \times recurrent\_units)$
	NF-BoT-IoT	$O(num\_features^2 + num\_features \times num\_timesteps \times recurrent\_units)$	$O(num\_features^2 + num\_features \times num\_timesteps \times recurrent\_units)$
	WUSTL-HDRL−2024	$O(num\_features^2 + num\_features \times num\_timesteps \times recurrent\_units)$	$O(num\_features^2 + num\_features \times num\_timesteps \times recurrent\_units)$

5.7. Sensitivity analysis results

The sensitivity analysis assesses the robustness and adaptability of the proposed models: EmbedNet, ConvNet-SVM, and DeepSVM-Net under varying levels of bias in the training data using three benchmark datasets: ECU-IoHT, NF-BoT-IoT, and WUSTL-HDRL-2024. The results are evaluated across standard metrics: Accuracy (ACC), PPV, TPR, F1-score, FNR, and FDR.

Table 34 presents the sensitivity analysis of the proposed models using the ECU-IoHT dataset, examining how their performance is impacted by varying levels of bias in the training data (mild, moderate, and severe). For each model, two attack classes are considered: ARP Spoofing and Nmap PortScan. The EmbedNet model demonstrates high

Table 30
Comparative analysis of computational resource utilization and prediction latency.

Models	Avg. CPU Usage (%)	Avg. GPU Usage (%)	Max RAM (MB)	Prediction Time
EmbedNet	22	34	480	0.20–1.97
ConvNet-SVM	35	48	630	1.87–3.89
DeepSVM-Net	49	62	800	0.12–5.22
CNN+GRU [10]	42	55	720	0.98–4.31
2D-CNN + ResNet [74]	56	71	960	1.41–6.41

Table 31
Inference time comparison on ECU-IoHT dataset (Dataset 1).

Model	Train-Test Split	Test Samples	Inference Time (s)	Inference Time per Sample (ms)
EmbedNet	70–30 %	1,01,002	12.43	0.123
	80–20 %	67,314	8.32	0.123
ConvNet-SVM	70–30 %	1,01,002	18.43	0.182
	80–20 %	67,314	12.48	0.185
DeepSVM-Net	70–30 %	1,01,002	21.59	0.214
	80–20 %	67,314	14.60	0.217

Table 32
Inference time comparison on NF-BoT-IoT dataset (Dataset-2).

Model	Train-Test Split	Test Samples	Inference Time (s)	Inference Time per Sample (ms)
EmbedNet	70–30 %	4,32,000	48.47	0.112
	80–20 %	2,88,000	32.65	0.113
ConvNet-SVM	70–30 %	4,32,000	72.45	0.167
	80–20 %	2,88,000	48.13	0.167
DeepSVM-Net	70–30 %	4,32,000	83.38	0.193
	80–20 %	2,88,000	56.02	0.194

accuracy under mild bias, with values around 0.9774–0.9815, but its performance declines as the bias increases, showing reduced PPV and TPR, particularly under severe bias where ARP Spoofing achieves only 0.7993 TPR. ConvNet-SVM shows a similar trend but performs slightly better overall, maintaining higher accuracy and F1-scores under all bias levels. Notably, DeepSVM-Net consistently outperforms the other two models, especially under mild and moderate bias levels, achieving the highest accuracy up to 0.9890 and the lowest FDR, as low as 0.0030 for Nmap PortScan. These results highlight that while all models suffer under increased bias, DeepSVM-Net is the most robust and least sensitive to data skew.

Table 35 extends the sensitivity analysis using the NF-BoT-IoT dataset and evaluates the models across two different attack types: Theft and Reconnaissance. Similar to Dataset 1, performance metrics degrade with increasing training data bias. The EmbedNet model shows strong performance under mild bias with high PPV of 0.9791 for Theft and relatively low FDR and FNR values but shows a notable drop in TPR (0.7938) and F1-score under severe bias. ConvNet-SVM maintains slightly better performance across all bias levels, especially in terms of TPR and F1-score. DeepSVM-Net again emerges as the most resilient model, retaining high accuracy of 0.9859 and low error rates under mild bias. Even under severe bias, DeepSVM-Net achieves TPR values of 0.7956 (Theft) and 0.7927 (Reconnaissance), showing a smaller degradation compared to the other models. These findings reinforce that DeepSVM-Net handles biased training data more effectively, ensuring higher detection rates with minimal compromise in false positives and negatives.

Table 36 shows the sensitivity analysis on a third dataset, WUSTL-HDRL-2024. The results closely mirror those from Dataset 2, suggesting consistent model behavior across multiple environments. All models show a decline in performance with increasing bias, but the degree of degradation varies. EmbedNet exhibits noticeable performance drops under severe bias, particularly in TPR and F1 for both attack types. ConvNet-SVM offers improved resilience, showing only a modest reduction in its metrics. DeepSVM-Net, once again, proves to be the most robust model, maintaining TPR and PPV close to the ideal even in the presence of substantial training data bias. Under severe bias, it delivers F1-scores of 0.7965 (DDoS) and 0.7943 (Ransomware), with minimal increases in FDR and FNR. These results demonstrate that DeepSVM-Net offers generalizable performance across datasets and is less affected by data imbalance, making it a promising candidate for real-world intrusion detection tasks in biased environments.

Table 33
Inference time comparison on WUSTL-HDRL-2024 dataset (Dataset-3).

Model	Train-Test Split	Test Samples	Inference Time (s)	Inference Time per Sample (ms)
EmbedNet	70–30 %	1,59,461	17.20	0.108
	80–20 %	1,06,307	11.31	0.106
ConvNet-SVM	70–30 %	1,59,461	25.00	0.157
	80–20 %	1,06,307	16.58	0.156
DeepSVM-Net	70–30 %	1,59,461	29.51	0.185
	80–20 %	1,06,307	19.80	0.186

The sensitivity analysis confirms that model performance is significantly affected by training data bias. DeepSVM-Net maintains strong detection capabilities and low error rates across all datasets and bias conditions, validating its suitability for deployment in real-world IoT environments where data imbalance is common. These findings underscore the necessity of employing robust learning architectures and bias mitigation strategies in security-sensitive applications such as IoMT and IIoT networks.

5.8. Performance comparison of proposed methods vs State-of-the-art

Table 37 provides a comparative evaluation of the proposed intrusion detection models against existing state-of-the-art methods using various benchmark datasets, including ECU-IoHT, NF-BoT-IoT, and WUSTL-HDRL-2024. The evaluation metrics like accuracy (AC), positive predictive value (PPV), true positive rate (TPR), F1-score, and ROC-AUC demonstrate the efficiency and robustness of each approach. While previous studies such as MSCSL by Thulasi and Sivamohan [35] achieved 97.90 % of accuracy and 96.54 % of F1-score and DNN-based IDS by Vishwakarma and Kesswani [60] attained 99.08 % of accuracy and 99.02 % of F1-score results, which exhibited limitations in balancing recall, precision, and false positives. Similarly, Zhang et al. [30]'s XGBoost model attained 95.01 % accuracy on ECU-IoHT, while traditional classifiers like SVM and RF remained in the 92–94 % range. However, these models struggled with generalization across multiple datasets. In contrast, the proposed models such as EmbedNet, ConvNet-SVM, and DeepSVM-Net outperform all existing methods with significantly improved accuracy and F1-scores. EmbedNet achieves 99.91 % accuracy on WUSTL-HDRL-2024, 99.46 % on NF-BoT-IoT, and 98.99 % on ECU-IoHT, while ConvNet-SVM and DeepSVM-Net follow closely, exceeding 99.5 % accuracy across multiple datasets. F1-score improvements are equally notable, reaching 99.80 %, 99.57 %, and 99.61 %, respectively, indicating superior detection performance with minimal false positives.

Compared to prior methods, the proposed models show an average accuracy improvement of 3.5–7.5 % and an F1-score enhancement of 4.3–5.9 %, demonstrating exceptional reliability and robustness in intrusion detection. These results validate the effectiveness of our approach, ensuring high detection precision and enhanced security for IoT environments. Fig. 11 depicts the performance comparison of the proposed model and state-of-the-art approaches.

6. Conclusion

The rapid expansion of IoT technology across various sectors has underscored the critical need for robust security measures. Traditional security methods must address IoT devices' unique challenges, such as limited processing power and storage capacity. This research has demonstrated the potential of DL models to enhance the capabilities of IDS in IoMT networks. By leveraging advanced neural network architectures, the proposed models EmbedNet, ConvNet-SVM, and DeepSVM-

Table 34
Sensitivity analysis of proposed models with different levels of bias in the training data using dataset 1 (ECU-IoHT).

Model	Bias Level	Class	ACC	PPV	TPR	F1	FNR	FDR
EmbedNet	**Mild**	ARP Spoofing	0.9774	0.9645	0.9491	0.9415	0.0009	0.0161
		Nmap PortScan	0.9815	0.9691	0.9492	0.9424	0.0008	0.0113
	Moderate	ARP Spoofing	0.9676	0.9350	0.8992	0.8920	0.0010	0.0166
		Nmap PortScan	0.9716	0.9395	0.8993	0.8928	0.0009	0.0117
	Severe	ARP Spoofing	0.9577	0.8858	0.7993	0.7929	0.0011	0.0174
		Nmap PortScan	0.9617	0.8900	0.7994	0.7936	0.0010	0.0122
ConvNet-SVM	**Mild**	ARP Spoofing	0.9832	0.9672	0.9493	0.9437	0.0007	0.0134
		Nmap PortScan	0.9848	0.9731	0.9497	0.9460	0.0003	0.0071
	Moderate	ARP Spoofing	0.9732	0.9376	0.8994	0.8941	0.0008	0.0138
		Nmap PortScan	0.9748	0.9434	0.8997	0.8962	0.0003	0.0074
	Severe	ARP Spoofing	0.9633	0.8882	0.7994	0.7947	0.0008	0.0144
		Nmap PortScan	0.9649	0.8937	0.7998	0.7966	0.0004	0.0077
DeepSVM-Net	**Mild**	ARP Spoofing	0.9885	0.9677	0.9498	0.9443	0.0002	0.0129
		Nmap PortScan	0.9890	0.9772	0.9499	0.9482	0.0001	0.0030
	Moderate	ARP Spoofing	0.9785	0.9380	0.8998	0.8946	0.0002	0.0132
		Nmap PortScan	0.9790	0.9472	0.8999	0.8983	0.0001	0.0030
	Severe	ARP Spoofing	0.9685	0.8887	0.7998	0.7952	0.0002	0.0139
		Nmap PortScan	0.9690	0.8974	0.7999	0.7985	0.0001	0.0032

Table 35
Sensitivity analysis of proposed models with different levels of bias in the training data using dataset 2 (NF-BoT-IoT).

Model	Bias Level	Class	ACC	PPV	TPR	F1	FNR	FDR
EmbedNet	**Mild**	Theft	0.9862	0.9791	0.9427	0.9459	0.0081	0.0009
		Reconnaissance	0.9829	0.9775	0.9385	0.9429	0.0127	0.0027
	Moderate	Theft	0.9763	0.9491	0.8931	0.8961	0.0085	0.0009
		Reconnaissance	0.9729	0.9475	0.8891	0.8933	0.0133	0.0027
	Severe	Theft	0.9663	0.8992	0.7938	0.7966	0.0092	0.0010
		Reconnaissance	0.9630	0.8977	0.7903	0.7940	0.0145	0.0029
ConvNet-SVM	**Mild**	Theft	0.9875	0.9792	0.9456	0.9473	0.0048	0.0008
		Reconnaissance	0.9844	0.9769	0.9419	0.9443	0.0089	0.0033
	Moderate	Theft	0.9776	0.9492	0.8959	0.8975	0.0051	0.0008
		Reconnaissance	0.9744	0.9470	0.8924	0.8946	0.0094	0.0034
	Severe	Theft	0.9676	0.8993	0.7963	0.7978	0.0055	0.0009
		Reconnaissance	0.9645	0.8971	0.7932	0.7952	0.0102	0.0035
DeepSVM-Net	**Mild**	Theft	0.9859	0.9770	0.9448	0.9458	0.0058	0.0032
		Reconnaissance	0.9832	0.9749	0.9414	0.9433	0.0096	0.0053
	Moderate	Theft	0.9760	0.9471	0.8950	0.8960	0.0060	0.0033
		Reconnaissance	0.9732	0.9451	0.8918	0.8936	0.0100	0.0055
	Severe	Theft	0.9660	0.8972	0.7956	0.7965	0.0066	0.0034
		Reconnaissance	0.9633	0.8953	0.7927	0.7943	0.0109	0.0057

Table 36
Sensitivity analysis of proposed models with different levels of bias in the training data using dataset 3 (WUSTL-HDRL-2024).

Model	Bias Level	Class	ACC	PPV	TPR	F1	FNR	FDR
EmbedNet	**Mild**	DDoS	0.9862	0.9791	0.9427	0.9459	0.0081	0.0009
		Ransomware	0.9829	0.9775	0.9385	0.9429	0.0127	0.0027
	Moderate	DDoS	0.9763	0.9491	0.8931	0.8961	0.0085	0.0009
		Ransomware	0.9729	0.9475	0.8891	0.8933	0.0133	0.0027
	Severe	DDoS	0.9663	0.8992	0.7938	0.7966	0.0092	0.0010
		Ransomware	0.9630	0.8977	0.7903	0.7940	0.0145	0.0029
ConvNet-SVM	**Mild**	DDoS	0.9875	0.9792	0.9456	0.9473	0.0048	0.0008
		Ransomware	0.9844	0.9769	0.9419	0.9443	0.0089	0.0033
	Moderate	DDoS	0.9776	0.9492	0.8959	0.8975	0.0051	0.0008
		Ransomware	0.9744	0.9470	0.8924	0.8946	0.0094	0.0034
	Severe	DDoS	0.9676	0.8993	0.7963	0.7978	0.0055	0.0009
		Ransomware	0.9645	0.8971	0.7932	0.7952	0.0102	0.0035
DeepSVM-Net	**Mild**	DDoS	0.9859	0.9770	0.9448	0.9458	0.0058	0.0032
		Ransomware	0.9832	0.9749	0.9414	0.9433	0.0096	0.0053
	Moderate	DDoS	0.9760	0.9471	0.8950	0.8960	0.0060	0.0033
		Ransomware	0.9732	0.9451	0.8918	0.8936	0.0100	0.0055
	Severe	DDoS	0.9660	0.8972	0.7956	0.7965	0.0066	0.0034
		Ransomware	0.9633	0.8953	0.7927	0.7943	0.0109	0.0057

Net perform better in detecting and mitigating advanced cyber threats. Our experimental results, based on real-time benchmark datasets such as ECU-IoHT, NF-BoT-IoT, and Wustl-HDRL-2024, highlight the efficacy of these models in achieving high accuracy rates of 0.9990, 0.9959, and 0.9987, respectively. These models also demonstrate the lowest FNR of 0.0001, 0.0059, and 0.0014. Furthermore, the ablation study conducted shows minimum training and validation losses of 0.0034 and 0.0008 for EmbedNet, 0.0018 and 0.0008 for ConvNet-SVM, and 0.0044 and

0.0016 for DeepSVM-Net, which demonstrates the outstanding performance of our models. The proposed models have shown a remarkable improvement percentage ranging from 3.5 % to 7.5 % over state-of-the-art methods. These findings underscore the enhanced effectiveness, accuracy, and resilience of IDS frameworks empowered by DL, which contribute significantly to cybersecurity. Adopting DL-based IDS frameworks can substantially strengthen the security of IoT systems, particularly in sensitive applications like healthcare. One current limitation of our evaluation framework is the absence of explicit zero-day attack testing.

6.1. Future research directions

In future work, we plan to investigate the resilience of the proposed intrusion detection models against zero-day attack scenarios. This will involve the use of synthetic attack generation techniques, adversarial training, and anomaly detection frameworks to simulate truly novel intrusion patterns absent from the training datasets. Evaluating model performance under such conditions will provide a more thorough understanding of its capabilities in real-world, evolving threat landscapes. While this research advances deep learning-based IDS for IoMT security, several areas require further exploration to enhance adaptability, scalability, and robustness.

- **Federated Learning for Decentralized IDS:** Traditional IDS frameworks rely on centralized data processing, which raises privacy risks, latency issues, and network congestion. Federated Learning (FL) enables collaborative model training across IoT devices without exposing sensitive data. Future research should explore optimized aggregation strategies, secure multi-party computation, and differential privacy to improve IDS performance.
- **Edge AI and Adaptive Learning for Real-Time Detection:** IoMT devices have limited computational resources, making cloud-based IDS inefficient. Edge AI allows real-time detection by deploying lightweight AI models on IoT edge nodes. Future work should focus on MobileNet, TinyML, and quantized models while integrating adaptive learning techniques to enhance detection accuracy and reduce false positives.
- **Explainable AI (XAI) for Trustworthy IDS:** Deep learning-based IDS models function as black boxes, making it difficult to interpret decisions. Explainable AI (XAI) can enhance transparency, regulatory compliance, and forensic analysis. Future research should explore SHAP and LIME to interpret model predictions, increasing trust in IDS systems and aiding cybersecurity professionals.
- **Quantum-Resistant Cryptography for IDS Security:** Quantum computing threatens traditional cryptographic techniques, making IoMT systems vulnerable to cyberattacks. Future IDS research should integrate quantum-resistant cryptographic algorithms such as lattice-based, hash-based, and multivariate polynomial cryptography to secure communication. Implementing post-quantum cryptographic protocols will ensure long-term IDS security against quantum threats.

Table 37
Comparison of proposed models with state of the art.

References	Proposed Model	Dataset	AC (%)	PPV (%)	TPR (%)	F1 (%)	ROC_AUC (%)
Ahmed et al. [26]	Influenced Outlierness (INFLO)	ECU-IoHT	-	-	-	95	-
Thulasi and Sivamohan [35]	MSCSL	ECU-IoHT	97.90	96.78	95.90	96.54	-
Zhang et al. [30]	XGB	ECU-IoHT	95.01	94.93	95.01	94.65	-
	LGB		94.82	94.78	94.82	94.41	-
	SVM		93.08	93.44	93.08	92.02	-
	RF		93.44	93.47	93.44	92.65	-
Sarhan et al. [42]	Extra-Tree	NF-BoT-IoT	93.82	93.70	-	97	-
Maimo et al. [52]	OC-SVM	ICE	-	92.32	99.97	95.96	-
Kumar et al. [53]	XGB	ToN-IoT	96.35	90.54	-	95.03	-
Saba [54]	Bagging	KDDCup99	93.2	99	92	96.1	-
Siddiqi and Pak [55]	RF	BoT-IoT	99.41	97.98	98.67	98.25	-
	SVM		72.89	59.11	80.57	64.17	-
	DNN		97.27	93.89	97.37	95.49	-
Newaz et al. [56]	RF	Own dataset	98	98	98	98	-
Gupta et al. [57]	RF	Wustl-EHMS	94.23	-	93.72	93.8	-
Grammatikis et al. [58]	CART	Own dataset	81.73	-	-	79.21	-
Nguyen and Kashef [59]	TS-IDS	NF-BoT-IoT	93.95	65.12	97.14	96.67	97.14
	EGraphSAGE		77.82	54.10	79.33	78.22	79.33
	XGB		93.69	63.38	96.64	95.41	96.64
Vishwakarma and Kesswani [60]	DNN based IDS	NF-BoT-IoT	99.08	99.03	99.08	99.02	-
Alosaimi and Almutairi [61]	Ensemble Classifier	BoT-IoT	73	73	74	73	-
Zhu et al. [62]	CMTSNN	BoT-IoT	99.3	98.4	97.5	97.9	-
	1D-CNN		96.2	89	88.9	88.9	-
Fernando et al. [63]	XGB	BoT-IoT	94.80	94.97	94.80	94.83	97.50
	GB		90.59	91.69	90.59	90.71	96.55
Xu et al. [64]	EE-GCN	NF-BOT-IOT	83.79	-	-	-	-
Chakraborty et al. [65]	RFE-MLP	ECU-IOHT	98.43	98.54	98.56	98.46	-
Thakkar and Lohiya [66]	DNN	BoT-IoT	98.99	98.90	91.30	94.95	-
Altaf et al. [67]	MNN	NF-BoT-IoT	97.74	97.79	96.63	97.74	50
Alzahrani and Asghar [68]	LSTM + CNN	BoT-IoT	95.73	96.64	94.15	95.52	-
Telikani et al. [69]	CostDeepIoT	BoT-IoT	-	93.8	94.7	94	-
Proposed Method	**EmbedNet**	**ECU_IoHT**	**98.99**	**98.78**	**99.94**	**99.18**	**99.36**
		NF-BoT-IoT	**99.46**	**99.83**	**99.05**	**99.44**	**99.46**
		WUSTL-HDRL−2024	**99.91**	**99.90**	**99.79**	**99.80**	**99.81**
	ConvNet-SVM	**ECU_IoHT**	**99.44**	**99.10**	**99.96**	**99.47**	**99.48**
		NF-BoT-IoT	**99.59**	**99.80**	**99.34**	**99.57**	**99.58**
		WUSTL-HDRL−2024	**99.87**	**99.88**	**99.85**	**99.87**	**99.87**
	DeepSVM-Net	**ECU_IoHT**	**99.90**	**99.28**	**99.99**	**99.61**	**99.62**
		NF-BoT-IoT	**99.46**	**99.57**	**99.28**	**99.44**	**99.45**
		WUSTL-HDRL−2024	**99.53**	**99.45**	**99.53**	**99.50**	**99.53**

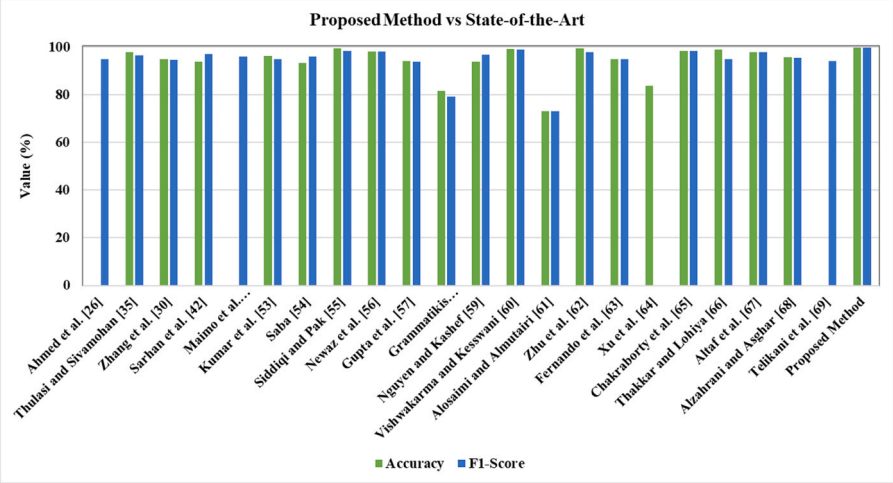

Fig. 11. Proposed method vs state of the art.

- **Hybrid AI Models for Advanced IDS Performance:** Deep learning-based IDS models can be enhanced using hybrid AI approaches. Future research should explore probabilistic graphical models, reinforcement learning, and evolutionary algorithms to detect zero-day attacks and polymorphic malware. Developing self-learning IDS systems will further improve adaptability to evolving cyber threats.
- **Cross-Domain Adaptability of IDS Frameworks:** IDS frameworks often require retraining for different IoT environments, limiting scalability. Future research should focus on transfer learning to enable IDS models trained in one domain (e.g., healthcare) to generalize effectively to another (e.g., industrial IoT), reducing retraining efforts and improving security resilience.

6.2. Potential industrial applications

The adoption of DL-based IDS frameworks extends beyond IoMT security and has broad industrial applications where cybersecurity threats pose significant risks. The proposed models demonstrate strong capabilities for securing cyber-physical systems and critical infrastructures across various sectors.

- **Healthcare and IoMT Security:** DL-based IDS frameworks help protect electronic health records (EHRs) from cyberattacks such as ransomware and unauthorized access. They enhance the security of remote patient monitoring (RPM) systems by detecting abnormal data patterns indicating potential breaches. Additionally, they prevent man-in-the-middle (MITM) attacks targeting wearable medical devices and smart implants.
- **Smart Cities and Critical Infrastructure Protection:** IDS solutions secure smart grids by detecting intrusions targeting energy distribution networks. They also protect intelligent transportation systems (ITS) from cyber threats targeting traffic signals, surveillance cameras, and vehicle-to-infrastructure (V2I) communications. Additionally, they prevent cyber-physical attacks on smart water management systems, ensuring the safety of public utilities.
- **Industrial IoT (IIoT) and Manufacturing Security:** IDS frameworks detect malware and insider threats targeting SCADA (Supervisory Control and Data Acquisition) systems in industrial plants. They also enhance security in robotic process automation (RPA) by preventing adversarial attacks on industrial robots. Furthermore, they safeguard supply chain IoT networks by mitigating cybersecurity risks in connected logistics systems.
- **Financial and Banking Sector:** In the financial sector, IDS frameworks strengthen cybersecurity in online banking by detecting anomalous transaction patterns. They protect cryptocurrency exchanges from fraud and cyberattacks by leveraging blockchain-based IoT security mechanisms. Additionally, they enhance fraud detection in real-time payment processing systems using anomaly detection techniques.
- **Autonomous Vehicles and Intelligent Transportation:** IDS solutions secure V2X (Vehicle-to-Everything) communications from cyber threats, preventing remote hijacking and GPS spoofing. They detect cyber intrusions targeting ADAS (Advanced Driver Assistance Systems) in self-driving cars. Furthermore, they prevent malware infections in connected vehicle ecosystems, ensuring operational integrity and passenger safety.

6.3. Theoretical contributions

This research significantly advances the theoretical foundation of deep learning-based intrusion detection systems (IDS) by addressing key limitations of existing methodologies and introducing novel approaches for IoMT security.

- **Novel Deep Learning Architectures for IDS:** This study develops EmbedNet, a categorical embedding neural network optimized for intrusion detection in IoMT environments. It introduces ConvNet-SVM, a hybrid CNN-SVM model leveraging convolutional feature extraction and SVM-based classification. Additionally, it proposes DeepSVM-Net, a deep neural network emulated by SVM, enhancing classification accuracy and detection performance.
- **Advanced Preprocessing and Feature Engineering Techniques:** A covariance matrix-based feature selection method eliminates irrelevant data, improving IDS efficiency. The study also applies data augmentation techniques to balance IoMT attack datasets, ensuring robust model training. Moreover, hybrid normalization techniques (e.g., Standard Scaler, Power Transformer) are used to enhance model generalization, leading to better detection performance.
- **Comprehensive Evaluation Using Advanced Metrics:** This research conducts an in-depth analysis of Markedness (MK), Informedness (BM), Positive Likelihood Ratio (LR+), and Negative Likelihood Ratio (LR-), providing advanced IDS evaluation metrics. Additionally, it analyzes False Omission Rate (FOR) and False Discovery Rate (FDR) to offer new insights into model reliability, ensuring accurate intrusion detection in IoMT environments.
- **Optimization of IDS Performance Based on Epoch and Batch Size Analysis:** The study examines how epoch and batch size variations impact training loss (TL), validation loss (VL), and overall model performance. It provides empirical insights into optimal

hyperparameter tuning, improving the efficiency and accuracy of deep learning-based IDS frameworks for IoMT security.

6.4. Practical contributions

This research provides practical solutions to enhance cybersecurity in IoMT environments, ensuring the real-world applicability of the proposed IDS models.

- **Scalable IDS Solutions for Real-World Deployment:** The proposed models achieve high detection accuracy (0.9984, 0.9996, 0.9975) across real-time benchmark datasets, validating their effectiveness. They are optimized for resource-constrained IoMT devices, ensuring scalability and seamless deployment in real-world scenarios.
- **Enhanced Detection Accuracy and Low False Alarm Rate:** The models significantly reduce false negative rates (0.0017, 0.0005, 0.0028), minimizing undetected cyber threats. They achieve a 0.14–0.28 % improvement over state-of-the-art methods, increasing the reliability of intrusion detection systems for IoMT security.
- **Computational Efficiency and Real-Time Feasibility:** The models demonstrate low training and validation losses, ensuring efficient learning without overfitting. Their optimized inference time reduces latency, making real-time intrusion detection feasible for IoMT environments.
- **Industry Adoption Readiness and Security Policy Compliance: The** IDS solutions ensure compatibility with modern IoMT cybersecurity frameworks, facilitating easy industry adoption. They also strengthen compliance with regulatory standards such as HIPAA, GDPR, and NIST security guidelines, ensuring secure and privacy-compliant IoMT implementations.

CRediT authorship contribution statement

Prashant Giridhar Shambharkar: Writing – review & editing, Visualization, Validation, Supervision, Investigation, Formal analysis. **Nikhil Sharma:** Writing – review & editing, Writing – original draft, Resources, Methodology, Formal analysis, Data curation, Conceptualization.

Ethics

This study uses publicly available data or data from published sources; therefore, no subject testing or data collection procedure was considered for this study.

Funding

Not Applicable

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data Availability

Data will be made available on request.

References

[1] J. Li, X. Tong, J. Liu, L. Cheng, An efficient federated learning system for network intrusion detection, IEEE Syst. J. 17 (2) (2023) 2455–2464, https://doi.org/10.1109/jsyst.2023.3236995.
[2] S.M. Sohi, J.-P. Seifert, F. Ganji, RNNIDS: enhancing network intrusion detection systems through deep learning, Comput. amp Secur. 102 (2021) 102151, https://doi.org/10.1016/j.cose.2020.102151.
[3] B. Sharma, L. Sharma, C. Lal, S. Roy, Anomaly based network intrusion detection for IOT attacks using deep learning technique, Comput. Electr. Eng. 107 (2023) 108626, https://doi.org/10.1016/j.compeleceng.2023.108626.
[4] S. Nandy, M. Adhikari, M.A. Khan, V.G. Menon, S. Verma, An intrusion detection mechanism for secured IOMT framework based on Swarm-Neural network, IEEE J. Biomed. Health Inform. 26 (5) (2022) 1969–1976, https://doi.org/10.1109/jbhi.2021.3101686.
[5] A. Aljuhani, A. Alamri, P. Kumar, A. Jolfaei, An intelligent and explainable SAAS-based intrusion detection system for resource-constrained IoMT, 1–1, IEEE Internet Things J. (2024), https://doi.org/10.1109/jiot.2023.3327024.
[6] F. Yan, G. Zhang, D. Zhang, X. Sun, B. Hou, N. Yu, TL-CNN-IDS: transfer Learning-based intrusion detection system using convolutional neural network, J. Supercomput. 79 (15) (2023) 17562–17584, https://doi.org/10.1007/s11227-023-05347-4.
[7] K. Wang, A. Zhang, H. Sun, B. Wang, Analysis of recent deep-learning-based intrusion detection methods for in-vehicle network, IEEE Trans. Intell. Transp. Syst. (2022) 1–12, https://doi.org/10.1109/tits.2022.3222486.
[8] L. Dhanya, R. Chitra, A novel autoencoder based feature independent ga optimised XGBoost classifier for IoMT malware detection, Expert Syst. Appl. 237 (2024) 121618, https://doi.org/10.1016/j.eswa.2023.121618.
[9] T.V. Khoa, D.T. Hoang, N.L. Trung, C.T. Nguyen, T.T. Quynh, D.N. Nguyen, N. V. Ha, E. Dutkiewicz, Deep transfer learning: a novel collaborative learning model for cyberattack detection systems in IOT networks, IEEE Internet Things J. 10 (10) (2023) 8578–8589, https://doi.org/10.1109/jiot.2022.3202029.
[10] H. Nandanwar, R. Katarya, Deep learning enabled intrusion detection system for industrial IOT environment, Expert Syst. Appl. 249 (2024) 123808, https://doi.org/10.1016/j.eswa.2024.123808.
[11] A.S. Dina, A.B. Siddique, D. Manivannan, A deep learning approach for intrusion detection in Internet of things using focal loss function, Internet Things 22 (2023) 100699, https://doi.org/10.1016/j.iot.2023.100699.
[12] K. Narayana Rao, K. Venkata Rao, P.R. P.V.G.D, A hybrid intrusion detection system based on sparse autoencoder and deep neural network, Comput. Commun. 180 (2021) 77–88, https://doi.org/10.1016/j.comcom.2021.08.026.
[13] E. Kabir, J. Hu, H. Wang, G. Zhuo, A novel statistical technique for intrusion detection systems, Future Gener. Comput. Syst. 79 (2018) 303–318, https://doi.org/10.1016/j.future.2017.01.029.
[14] S. Aljawarneh, M. Aldwairi, M.B. Yassein, Anomaly-based intrusion detection system through feature selection analysis and building hybrid efficient model, J. Comput. Sci. 25 (2018) 152–160, https://doi.org/10.1016/j.jocs.2017.03.006.
[15] Yuyang Zhou, G. Cheng, S. Jiang, M. Dai, Building an efficient intrusion detection system based on feature selection and ensemble classifier, Comput. Netw. 174 (2020) 107247, https://doi.org/10.1016/j.comnet.2020.107247.
[16] T. Aldwairi, D. Perera, M.A. Novotny, An evaluation of the performance of restricted boltzmann machines as a model for anomaly network intrusion detection, Comput. Netw. 144 (2018) 111–119, https://doi.org/10.1016/j.comnet.2018.07.025.
[17] A. Abusitta, M. Bellaiche, M. Dagenais, T. Halabi, A deep learning approach for proactive multi-cloud cooperative intrusion detection system, Future Gener. Comput. Syst. 98 (2019) 308–318, https://doi.org/10.1016/j.future.2019.03.043.
[18] J. Gu, L. Wang, H. Wang, S. Wang, A novel approach to intrusion detection using SVM ensemble with feature augmentation, Comput. amp Secur. 86 (2019) 53–62, https://doi.org/10.1016/j.cose.2019.05.022.
[19] Ying Zhou, T.A. Mazzuchi, S. Sarkani, M-adaboost-A based ensemble system for network intrusion detection, Expert Syst. Appl. 162 (2020) 113864, https://doi.org/10.1016/j.eswa.2020.113864.
[20] C. Kim, J. Park, Designing online network intrusion detection using deep auto-encoder Q-learning, Comput. amp Electr. Eng. 79 (2019) 106460, https://doi.org/10.1016/j.compeleceng.2019.106460.
[21] X. Li, W. Chen, Q. Zhang, L. Wu, Building auto-encoder intrusion detection system based on random forest feature selection, Comput. amp Secur. 95 (2020) 101851, https://doi.org/10.1016/j.cose.2020.101851.
[22] E.-H. Qazi, M. Imran, N. Haider, M. Shoaib, I. Razzak, An intelligent and efficient network intrusion detection system using deep learning, Comput. Electr. Eng. 99 (2022) 107764, https://doi.org/10.1016/j.compeleceng.2022.107764.
[23] I. Firat Kilincer, F. Ertam, A. Sengur, R.-S. Tan, U. Rajendra Acharya, Automated detection of cybersecurity attacks in healthcare systems with recursive feature elimination and multilayer perceptron optimization, Biocybern. Biomed. Eng. 43 (1) (2023) 30–41, https://doi.org/10.1016/j.bbe.2022.11.005.
[24] F. Mosaiyebzadeh, S. Pouriyeh, R.M. Parizi, M. Han, D.M. Batista, Intrusion detection system for IOHT devices using federated learning, IEEE INFOCOM 2023 IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS) (2023), https://doi.org/10.1109/infocomwkshps57453.2023.10225932.
[25] A. Nazir, J. He, N. Zhu, S.S. Qureshi, S.U. Qureshi, F. Ullah, A. Wajahat, M. S. Pathan, A deep learning-based novel hybrid CNN-LSTM architecture for efficient detection of threats in the IOT ecosystem, Ain Shams Eng. J. 15 (7) (2024) 102777, https://doi.org/10.1016/j.asej.2024.102777.
[26] M. Ahmed, S. Byreddy, A. Nutakki, L.F. Sikos, P. Haskell-Dowland, ECU-ioht: a dataset for analyzing cyberattacks in Internet of health things, Ad Hoc Netw. 122 (2021) 102621, https://doi.org/10.1016/j.adhoc.2021.102621.
[27] S.K. Shahzad, D. Ahmed, M.R. Naqvi, M.T. Mushtaq, M.W. Iqbal, F. Munir, Ontology driven smart health service integration, Comput. Methods Prog. Biomed. 207 (2021) 106146, https://doi.org/10.1016/j.cmpb.2021.106146.
[28] M. Alazab, R.A. Khurma, A. Awajan, D. Camacho, A new intrusion detection system based on Moth–Flame optimizer algorithm, Expert Syst. Appl. 210 (2022) 118439, https://doi.org/10.1016/j.eswa.2022.118439.

[29] J. Azimjonov, T. Kim, Designing accurate lightweight intrusion detection systems for IOT networks using fine-tuned linear SVM and feature selectors, Comput. amp Secur. 137 (2024) 103598, https://doi.org/10.1016/j.cose.2023.103598.
[30] Y. Zhang, D. Zhu, M. Wang, J. Li, J. Zhang, A comparative study of cyber security intrusion detection in healthcare systems, Int. J. Crit. Infrastruct. Prot. 44 (2024) 100658, https://doi.org/10.1016/j.ijcip.2023.100658.
[31] A.D. Aguru, S.B. Erukala, A lightweight multi-vector ddos detection framework for IOT-enabled mobile health informatics systems using deep learning, Inf. Sci. 662 (2024) 120209, https://doi.org/10.1016/j.ins.2024.120209.
[32] R. Kumar, A. Aljuhani, D. Javeed, P. Kumar, S. Islam, A.K.M.N. Islam, Digital Twins-Enabled zero touch network: a smart contract and explainable AI integrated cybersecurity framework, Future Gener. Comput. Syst. 156 (2024) 191–205, https://doi.org/10.1016/j.future.2024.02.015.
[33] Y.K. Saheed, O.H. Abdulganiyu, T.A. Tchakoucht, Modified genetic algorithm and fine-tuned long short-term memory network for intrusion detection in the Internet of things networks with edge capabilities, Appl. Soft Comput. 155 (2024) 111434, https://doi.org/10.1016/j.asoc.2024.111434.
[34] S.S. Chintapalli, S.P. Singh, J. Frnda, P. Bidare Divakarachari, V.L. Sarraju, P. Falkowski-Gilski, OOA-modified Bi-LSTM network: an effective intrusion detection framework for IOT systems, Heliyon 10 (8) (2024), https://doi.org/10.1016/j.heliyon.2024.e29410.
[35] T. Thulasi, K. Sivamohan, LSO-CSL: light spectrum optimizer-based convolutional stacked long short term memory for attack detection in IOT-based healthcare applications, Expert Syst. Appl. 232 (2023) 120772, https://doi.org/10.1016/j.eswa.2023.120772.
[36] S. Reza, M.C. Ferreira, J.J.M. Machado, J.M. Tavares, A multi-head attention-based transformer model for traffic flow forecasting with a comparative analysis to recurrent neural networks, Expert Syst. Appl. 202 (2022) 117275, https://doi.org/10.1016/j.eswa.2022.117275.
[37] M. Zhong, S. Yi, J. Fan, Y. Zhang, G. He, Y. Cao, L. Feng, Z. Tan, W. Mo, Power transformer fault diagnosis based on a self-strengthening offline pre-training model, Eng. Appl. Artif. Intell. 126 (2023) 107142, https://doi.org/10.1016/j.engappai.2023.107142.
[38] H. Jiang, J. Lin, H. Kang, FGMD: a robust detector against adversarial attacks in the IOT network, Future Gener. Comput. Syst. 132 (2022) 194–210, https://doi.org/10.1016/j.future.2022.02.019.
[39] S. Sachan, F. Almaghrabi, J.-B. Yang, D.-L. Xu, Evidential reasoning for preprocessing uncertain categorical data for trustworthy decisions: an application on healthcare and finance, Expert Syst. Appl. 185 (2021) 115597, https://doi.org/10.1016/j.eswa.2021.115597.
[40] J. Fonseca, F. Bacao, Geometric smote for imbalanced datasets with nominal and continuous features, Expert Syst. Appl. 234 (2023) 121053, https://doi.org/10.1016/j.eswa.2023.121053.
[41] S. Jain, P.M. Pawar, R. Muthalagu, Hybrid intelligent intrusion detection system for Internet of things, SSRN Electron. J. (2022), https://doi.org/10.2139/ssrn.4097433.
[42] M. Sarhan, S. Layeghy, N. Moustafa, M. Portmann, NetFlow datasets for machine Learning-based network intrusion detection systems, Lect. Notes Inst. Comput. Sci. Soc. Inform. Telecommun. Eng. (2021) 117–135, https://doi.org/10.1007/978-3-030-72802-1_9.
[43] Ghubaish, A., Yang, Z., & Jain, R. (n.d.). HDRL-2024 dataset for cybersecurity research on medical applications in 5G networks. WUSTL. https://www.cs.wustl.edu/~jain/hdrl/index.html.
[44] M. Karanfilovska, T. Kochovska, Z. Todorov, A. Cholakoska, G. Jakimovski, D. Efnusheva, Analysis and modelling of a ML-based nids for IOT networks, Procedia Comput. Sci. 204 (2022) 187–195, https://doi.org/10.1016/j.procs.2022.08.023.
[45] G.L. Nguyen, B. Dumba, Q.-D. Ngo, H.-V. Le, T.N. Nguyen, A collaborative approach to early detection of IOT botnet, Comput. Amp Electr. Eng. 97 (2022) 107525, https://doi.org/10.1016/j.compeleceng.2021.107525.
[46] G. De La Torre Parra, P. Rad, K.-K.R. Choo, N. Beebe, Detecting Internet of things attacks using distributed deep learning, J. Netw. Comput. Appl. 163 (2020) 102662, https://doi.org/10.1016/j.jnca.2020.102662.
[47] H. Alkahtani, T.H. Aldhyani, Botnet attack detection by using CNN-LSTM model for Internet of things applications, Secur. Commun. Netw. 2021 (2021) 1–23, https://doi.org/10.1155/2021/3806459.
48] S.A. Wagan, J. Koo, I.F. Siddiqui, N.M. Qureshi, M. Attique, D.R. Shin, A fuzzy-based duo-secure multi-modal framework for IOMT anomaly detection, J. King Saud. Univ. Comput. Inf. Sci. 35 (1) (2023) 131–144, https://doi.org/10.1016/j.jksuci.2022.11.007.
[49] L. Gupta, T. Salman, A. Ghubaish, D. Unal, A.K. Al-Ali, R. Jain, Cybersecurity of multi-cloud healthcare systems: a hierarchical deep learning approach, Appl. Soft Comput. 118 (2022) 108439, https://doi.org/10.1016/j.asoc.2022.108439.
[50] X. Wang, Y. Wang, Z. Javaheri, L. Almutairi, N. Moghadamnejad, O.S. Younes, Federated deep learning for anomaly detection in the Internet of things, Comput. Electr. Eng. 108 (2023) 108651, https://doi.org/10.1016/j.compeleceng.2023.108651.
[51] R. Xu, G. Wu, W. Wang, X. Gao, A. He, Z. Zhang, Applying self-supervised learning to network intrusion detection for network flows with graph neural network, Comput. Netw. 248 (2024) 110495, https://doi.org/10.1016/j.comnet.2024.110495.
[52] L. Fernández Maimó, A. Huertas Celdrán, Á.L. Perales Gómez, F.J. García Clemente, J. Weimer, I. Lee, Intelligent and dynamic ransomware spread detection and mitigation in integrated clinical environments, Sensors 19 (5) (2019) 1114, https://doi.org/10.3390/s19051114.
[53] P. Kumar, G.P. Gupta, R. Tripathi, An ensemble learning and fog-cloud architecture-driven cyber-attack detection framework for IOMT networks, Comput. Commun. 166 (2021) 110–124, https://doi.org/10.1016/j.comcom.2020.12.003.
[54] T. Saba, Intrusion detection in smart city hospitals using ensemble classifiers, 2020 13th Int. Conf. Dev. eSystems Eng. (DeSE) (2020), https://doi.org/10.1109/dese51703.2020.9450247.
[55] M.A. Siddiqi, W. Pak, An agile approach to identify single and hybrid normalization for enhancing machine learning-based network intrusion detection, IEEE Access 9 (2021) 137494–137513, https://doi.org/10.1109/access.2021.3118361.
[56] A.I. Newaz, A.K. Sikder, L. Babun, A.S. Uluagac, Heka: a novel intrusion detection system for attacks to personal medical devices, 2020 IEEE Conf. Commun. Netw. Secur. (CNS) (2020), https://doi.org/10.1109/cns48642.2020.9162311.
[57] K. Gupta, D.K. Sharma, K. Datta Gupta, A. Kumar, A tree classifier based network intrusion detection model for Internet of medical things, Comput. Electr. Eng. 102 (2022) 108158, https://doi.org/10.1016/j.compeleceng.2022.108158.
[58] P. Radoglou-Grammatikis, K. Rompolos, P. Sarigiannidis, V. Argyriou, T. Lagkas, A. Sarigiannidis, S. Goudos, S. Wan, Modeling, detecting, and mitigating threats against industrial healthcare systems: a combined software defined networking and reinforcement learning approach, IEEE Trans. Ind. Inform. 18 (3) (2022) 2041–2052, https://doi.org/10.1109/tii.2021.3093905.
[59] H. Nguyen, R. Kashef, TS-ids: Traffic-aware self-supervised learning for IOT network intrusion detection, Knowl. Based Syst. 279 (2023) 110966, https://doi.org/10.1016/j.knosys.2023.110966.
[60] M. Vishwakarma, N. Kesswani, DIDS: a deep neural network based real-time intrusion detection system for IOT, Decis. Anal. J. 5 (2022) 100142, https://doi.org/10.1016/j.dajour.2022.100142.
[61] S. Alosaimi, S.M. Almutairi, An intrusion detection system using BOT-IOT, Appl. Sci. 13 (9) (2023) 5427, https://doi.org/10.3390/app13095427.
[62] S. Zhu, X. Xu, H. Gao, F. Xiao, CMTSNN: a deep learning model for multiclassification of abnormal and encrypted traffic of Internet of things, IEEE Internet Things J. 10 (13) (2023) 11773–11791, https://doi.org/10.1109/jiot.2023.3244544.
[63] G.-P. Fernando, A.-A.H. Brayan, A.M. Florina, C.-B. Liliana, A.-M. Héctor-Gabriel, T.-S. Reinel, Enhancing intrusion detection in IOT communications through ML model generalization with a new dataset (IDSAI), IEEE Access 11 (2023) 70542–70559, https://doi.org/10.1109/access.2023.3292267.
[64] P. Xu, G. Lu, Y. Li, C. Xu, EE-GCN: a graph convolutional network based intrusion detection method for IIOT, 2023 5th Int. Conf. Nat. Lang. Process. (ICNLP) (2023), https://doi.org/10.1109/icnlp58431.2023.00068.
[65] C. Chakraborty, S.M. Nagarajan, G.G. Devarajan, T.V. Ramana, R. Mohanty, Intelligent AI-based healthcare cyber security system using Multi-Source transfer learning method, ACM Trans. Sens. Netw. (2023), https://doi.org/10.1145/3597210.
[66] A. Thakkar, R. Lohiya, Attack classification of imbalanced intrusion data for IOT network using ensemble-learning-based deep neural network, IEEE Internet Things J. 10 (13) (2023) 11888–11895, https://doi.org/10.1109/jiot.2023.3244810.
[67] T. Altaf, X. Wang, W. Ni, G. Yu, R.P. Liu, R. Braun, A new concatenated multigraph neural network for IOT intrusion detection, Internet Things 22 (2023) 100818, https://doi.org/10.1016/j.iot.2023.100818.
[68] A. Alzahrani, M.Z. Asghar, Cyber vulnerabilities detection system in logistics-based IOT data exchange, Egypt. Inform. J. 25 (2024) 100448, https://doi.org/10.1016/j.eij.2024.100448.
[69] A. Telikani, N.E. Rudbardeh, S. Soleymanpour, A. Shahbahrami, J. Shen, G. Gaydadjiev, R. Hassanpour, A cost-sensitive machine learning model with multitask learning for intrusion detection in IOT, IEEE Trans. Ind. Inform. 20 (3) (2024) 3880–3890.
[70] S.P. R.M, P.K., M., P. Maddikunta, S. Koppu, T.R. Gadekallu, C.L. Chowdhary, M. Alazab, An effective feature engineering for DNN using hybrid PCA-GWO for intrusion detection in IOMT architecture, Comput. Commun. 160 (2020) 139–149, https://doi.org/10.1016/j.comcom.2020.05.048.
[71] S. Patil, V. Varadarajan, S.M. Mazhar, A. Sahibzada, N. Ahmed, O. Sinha, S. Kumar, K. Shaw, K. Kotecha, Explainable artificial intelligence for intrusion detection system, Electronics 11 (19) (2022) 3079, https://doi.org/10.3390/electronics11193079.
[72] G. Lazrek, K. Chetioui, Y. Balboul, S. Mazer, M. El bekkali, An RFE/ridge-ML/DL based anomaly intrusion detection approach for securing IOMT system, Results Eng. 23 (2024) 102659, https://doi.org/10.1016/j.rineng.2024.102659.
[73] I. Ioannou, P. Nagaradjane, P. Angin, P. Balasubramanian, K.J. Kavitha, P. Murugan, V. Vassiliou, Gemlids-Miot: a Green effective machine learning intrusion detection system based on federated learning for medical IOT network security hardening, Comput. Commun. 218 (2024) 209–239, https://doi.org/10.1016/j.comcom.2024.02.023.
[74] H. Nandanwar, R. Katarya, Securing industry 5.0: an explainable deep learning model for intrusion detection in cyber-physical systems, Comput. Electr. Eng. 123 (2025) 110161, https://doi.org/10.1016/j.compeleceng.2025.110161.
[75] H.C. Altunay, Z. Albayrak, A.N. Ozalp, M. Cakmak, Analysis of anomaly detection approaches performed through deep learning methods in SCADA systems, 2021 3rd Int. Congr. Hum. Comput. Interact. Optim. Robot. Appl. (HORA) (2021) 1–6, https://doi.org/10.1109/hora52670.2021.9461273.
[76] A.N. Ozalp, Z. Albayrak, M. Cakmak, E. Ozdogan, Layer-based examination of cyber-attacks in IOT, 2022 Int. Congr. Hum. Comput. Interact. Optim. Robot. Appl. (HORA) (2022) 1–10, https://doi.org/10.1109/hora55278.2022.9800047.